Lecture 1

What is Computer Architecture?

- Computer Architecture is split into two main parts:
  - **Instruction-Set Architecture (ISA)**: This defines the set of instructions that the computer understands and can execute. Think of it as the "language" the computer speaks. ISA helps design compilers and operating systems to interact with the hardware, especially focusing on the **Control Unit**.
  - **Hardware-System Architecture (HSA)**: This is all about the design of physical hardware components like the **CPU**, **Memory**, and **Interfaces**.

Computer Family:

- A group of computers that share the same ISA is called a "computer family." This means they can run the same programs and work similarly at the instruction level.

Compatibility:

- **Upward Compatibility**: High-performance computers can run programs designed for lower-performance ones within the same family. This means you can run older software on newer computers without problems.
- **Downward Compatibility**: The opposite of upward compatibility. Lower-performance computers can still run some programs from higher-performance ones, but there might be limits.
- **Forward Compatibility**: Software designed for one computer family can work on future versions of that family.

Historical Perspectives

- **Charles Babbage (1792-1871)**: Known as the father of modern computers.
  - Created the **Difference Engine** in the early 1820s.
  - Later, he worked on the **Analytical Engine**, a more complex machine.

Analytical Engine

- The Analytical Engine was a foundational concept for today's computers. It included ideas like a mechanical processor and could be programmed using punch cards.

Historical Perspectives

- **Eckert and Mauchly**: Developers of the **ENIAC** (Electronic Numerical Integrator and Computer) in the 1940s.
    - ENIAC was built at the University of Pennsylvania.
    - It was not a "stored-program" computer, meaning instructions weren't stored in the same way as modern computers.

ENIAC

- **ENIAC** was one of the first electronic general-purpose computers. It performed thousands of calculations faster than any previous technology.

1944: Von Neumann's Influence

- Joined ENIAC project
- Led to EDVAC development
- Introduced stored program concept

1949: First Working Stored Program Computer

- Maurice built EDSAC(Electronic Delay Storage Automatic Calculator)
- Used mercury delay lines for memory
- Based on Von Neumann's ideas

1946: IAS Machine

- Designed by Von Neumann & Julian at Princeton
- Features:
    - 1024 40-bit words memory
    - 10x faster than ENIAC

Significance:

- Marked transition from hardwired to stored program computers
- Set foundation for modern computer architecture
- Established basic computer memory and processing concepts

- **Colossus**: Developed by the British to break encrypted German messages during World War II.
    - First operational in 1943, but it remained a secret until 1975.

The Manchester Mark I (First Stored Program Computer):

- Operational date: June 21, 1948
- Historical significance: World's first electronic stored program computer

---

- Computers in the late 1940s used vacuum tubes. These machines were unique and mainly found in research labs.
- In the **1960s**, computers advanced to use **transistors**, making them more reliable and compact. These were the **second-generation computers**.

IBM Stretch Computer:

- Date: 1961
- Name: IBM 7704/5
- Key features:
    - Designed to "stretch" technology/performance to its limits
    - Used transistor-based technology
    - Much faster than earlier tube-based systems
    - Processing speed comparison: 1 microsecond add operation compared to 84 microsecond add operation in previous systems

---

. **Number Systems**

- **Number Systems**: Systems used to represent values using positional notation.
- The four commonly used number systems are:
    1. **Decimal Number System**: Base 10, uses digits 0 to 9.
    2. **Binary Number System**: Base 2, uses digits 0 and 1.
    3. **Hexadecimal Number System**: Base 16, uses digits 0-9 and letters A-F.
    4. **Octal Number System**: Base 8, uses digits 0 to 7.

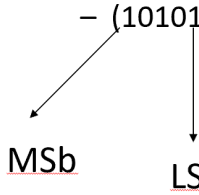
Decimal Number System

Decimal System: The most familiar system, with a base of 10.

- The **decimal** number system has **ten unique symbols**: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9
- It is also called **Base 10** number system
  - the system of counting has ten symbols
  - Its **radix** or **base** is said to be 10
- For example
  - $(3472)_{10} = 3 * 10^3 + 4 * 10^2 + 7 * 10^1 + 2 * 10^0$
    $= 3000 + 400 + 70 + 2$
  - $(536.15)_{10} = 5 * 10^2 + 3 * 10^1 + 6 * 10^0 + 1 * 10^{-1} + 5 * 10^{-2}$

Binary Number System

- **Binary System**: Uses only two symbols, 0 and 1. The base is 2.
- For example
  - $(10101)_2 = 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0$
    $= 16 + 0 + 4 + 0 + 1$
    $= 21$ (the **decimal equivalent** of 101101_2)



MSb: Most Significant bit
LSb: Least Significant bit

- **MSb and LSb**:
  - **MSb**: Most Significant Bit (the leftmost bit).
  - **LSb**: Least Significant Bit (the rightmost bit).

Binary to Decimal

11001.011_2

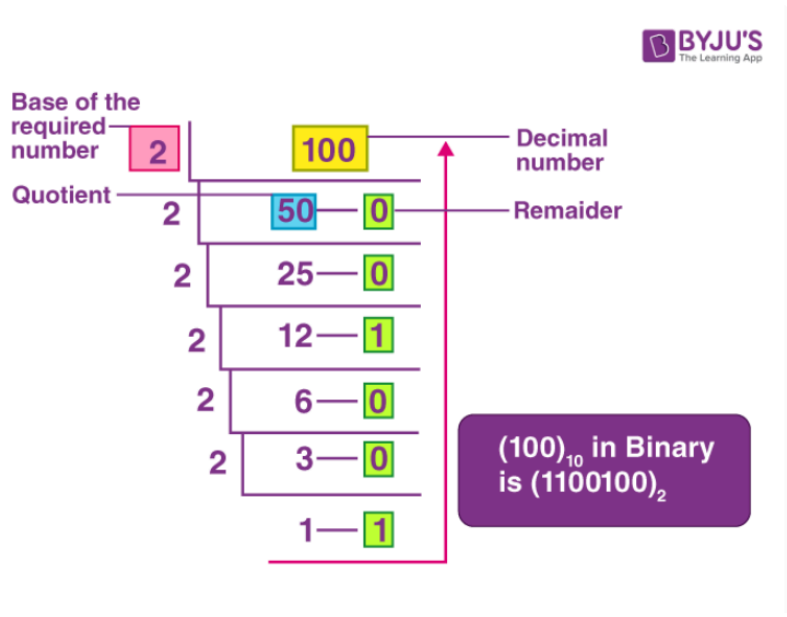
$= 1* 2^4 + 1*2^3 + 0*2^2 + 0*2^1 + 1*2^0 + 0*2^{-1} + 1*2^{-2} + 1*2^{-3}$

$= 16 + 8 + 0 + 0 + 1 + 0 + 0.25 + 0.125$

$= 25.375$

Decimal to Binary Conversion

- **Decimal to Binary Conversion Steps**:
  - Divide the decimal number by 2.
  - Write down the remainder (0 or 1).
  - Repeat this until the quotient is 0.
  - The binary number is the remainders read in reverse order.
- **Examples**:



Method for Converting Decimal Fraction to Binary:

1. Take 0.71 and repeatedly multiply by 2
2. Collect the integer parts (before decimal) as binary digits
3. Continue until we get our 4 bits after decimal point

$0.71 \times 2 = 1.42 \rightarrow 1$

$0.42 \times 2 = 0.84 \rightarrow 0$

$0.84 \times 2 = 1.68 \rightarrow 1$

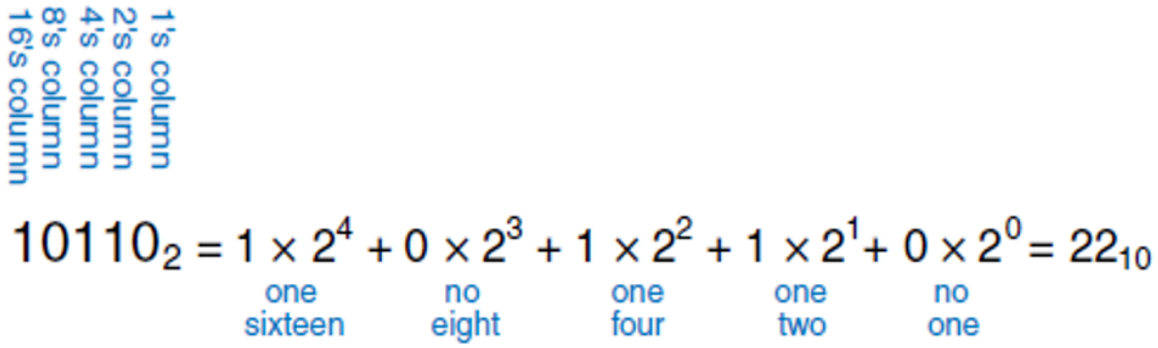
$0.68 \times 2 = 1.36 \rightarrow 1$

Therefore:

$0.71 \approx 0.1011$ in binary

Binary to Decimal Conversion

- **Binary to Decimal Conversion Steps**:
  - Multiply each bit by 2^n, where n is the position of the bit from right to left, starting at 0.
  - Add all the results to get the decimal equivalent.
- **Examples**:



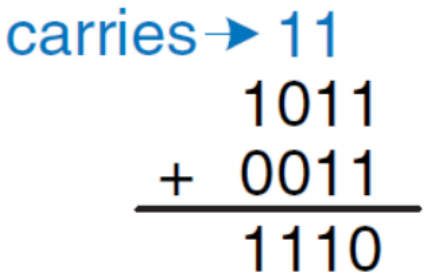
Binary Addition

- **Binary Addition Rules**:

Binary Addition Rules:

- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$ (which means 0, carry 1)

- When adding larger binary numbers, carry values forward just like in decimal addition.



Binary Subtraction

- **Binary Subtraction Using Two's Complement**:
  - Subtraction in binary is performed using the concept of **two's complement**.
  - **Two's Complement**: Invert all bits (1's complement) and add 1 to the result

Example: Subtract 7 from 5 Using Binary (5 - 7)

1. Write the binary numbers:
   - $5 = 0101_2$
   - $7 = 0111_2$
2. Find the Two's Complement of 7 (B):
   - **1's complement of 0111**: Invert all bits to get 1000.
   - **Add 1 to 1's complement**: $1000 + 1 = 1001$.
   - The two's complement of 7 is 1001.
3. Add 5 and the Two's Complement of 7:
   - $0101 + 1001 = 1110$.
   - Since the result starts with a 1, it is a negative number.
4. Interpret the Result:
   - 1110 in two's complement is -2 in decimal.

Key Points to Remember:

- **1's Complement**: Flip all bits (0 becomes 1, and 1 becomes 0).
- **Two's Complement**: Add 1 to the 1's complement.
- If there is an extra carry after addition, ignore it.
- A leading 1 in the result means the number is negative.

What is Hexadecimal?

- **Hexadecimal** is a **base-16** number system.
- It uses **16 symbols**: **0-9** and **A-F**.
  - **A = 10**
  - **B = 11**
  - **C = 12**
  - **D = 13**
  - **E = 14**
  - **F = 15**

What is 4-Bit Binary?

- **4-bit binary** means the binary number has **4 digits**.
- Example: 1010 is a 4-bit binary number.

Convert Hexadecimal to Binary

1. **Understand the Hexadecimal Digit:**
    a. **Digits 0-9:** Their decimal value is the same as the digit.
    b. **Letters A-F:** Represent decimal values **10-15**.
        i. **A = 10**
        ii. **B = 11**
        iii. **C = 12**
        iv. **D = 13**
        v. **E = 14**
        vi. **F = 15**
2. **Convert Each Hex Digit to 4-Bit Binary**
3. Use a **Hex to Binary** chart
4. **Combine All Binary Groups :** Put the binary groups together to get the final binary number.

Example

Convert hexadecimal 2F to binary:

1. **Hexadecimal Number:** 2F
2. **Convert Each Digit:**
    - 2 = 0010
    - F = 1111
3. **Combine Binary Groups:**
    - 2F = 0010 1111
4. **Result:**
    - 2F in Hexadecimal is **00101111** in Binary

Hexadecimal number system

Hexadecimal Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

How to Convert Decimal to 4-Bit Binary:

- **Find how many 8s, 4s, 2s, and 1s fit into the decimal number.**
- **Write 1** for each value that fits and **0** for those that don't.

Example 1: Convert Hex A to Binary

1. **Hex Digit:** A
2. **Decimal Value:** 10 (since A = 10)
3. **Convert 10 to 4-Bit Binary:**
   - **8** fits into 10: Yes → **1**
   - **10 - 8 = 2**
   - **4** fits into 2: No → **0**
   - **2** fits into 2: Yes → **1**
   - **2 - 2 = 0**
   - **1** fits into 0: No → **0**
   - **Binary:** 1010

Result: A in Hexadecimal is **1010** in Binary

Hexadecimal to Decimal Conversion

- Convert $A59C_{16}$ or A59CH into decimal

$$A59CH = A * 16^3 + 5 * 16^2 + 9 * 16^1 + C * 16^0$$
$$= 40960 + 1280 + 144 + 12$$
$$= 42396$$

Convert Decimal to Hexadecimal

1. Divide the Decimal Number by 16

- **Why 16?** Because Hexadecimal is base 16.

2. Keep Track of the Remainders

- **Remainder:** The leftover part after division.
- Remainders will be used to form the Hexadecimal number.

3. Continue Dividing the Quotient by 16

- **Quotient:** The result of the division.
- Repeat the division until the quotient is **0**.

4. Read the Remainders from Bottom to Top

- The first remainder is the **rightmost** digit.
- The last remainder is the **leftmost** digit.

5. Convert Remainders Greater Than 9 to Letters

- **10 = A**
- **11 = B**
- **12 = C**
- **13 = D**
- **14 = E**
- **15 = F**

Example 1: Convert 255_{10} to Hexadecimal

Problem: Convert **255** (in Decimal) to Hexadecimal.

Step 1: Divide by 16

Step 3: Read the Remainders from Bottom to Top

- **First Remainder:** 15
- **Second Remainder:** 15

Step 4: Convert Remainders to Hexadecimal Letters

- **15 = F**

Step 5: Combine the Hexadecimal Digits

- **First Remainder (bottom):** F
- **Second Remainder (top):** F
- **Hexadecimal Number: FF_{16}**

Result: $255_{10} = FF_{16}$

4 Types of Flags

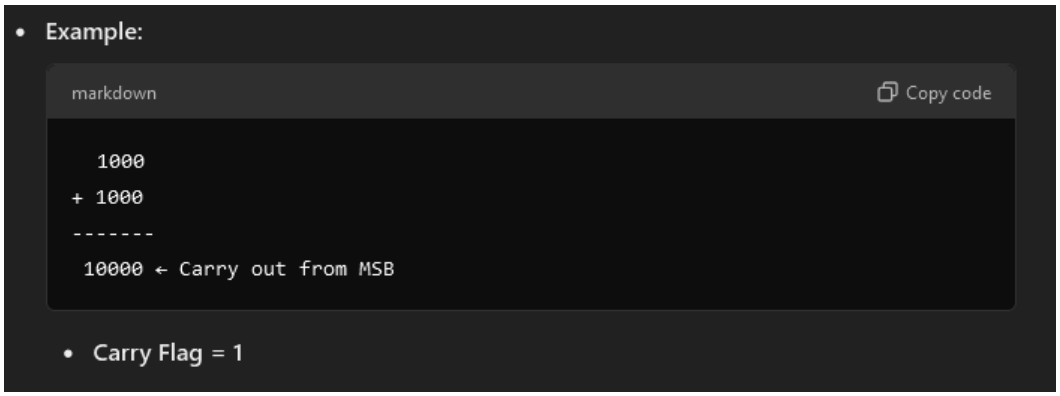
Carry Flag (C)

What is the Carry Flag?

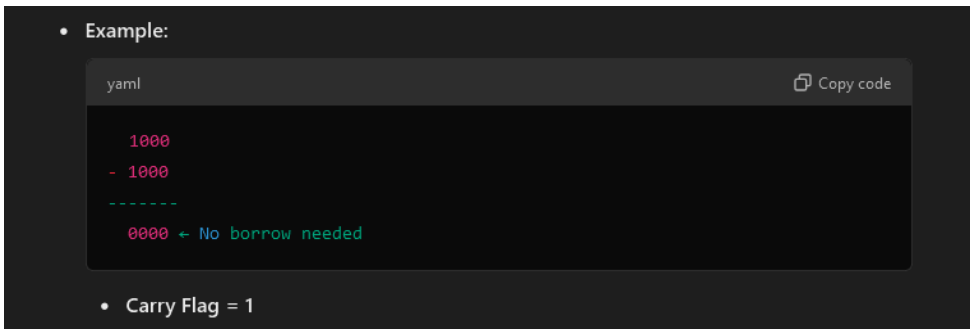
- Purpose: The Carry Flag helps the computer handle unsigned numbers (numbers that are always positive).
- When is it Used: It keeps track of extra information during addition and subtraction.

When is the Carry Flag Set?

- **In Addition:**
  - Set to 1 when there is a carry out from the Most Significant Bit (MSB) (the leftmost bit).



- In Subtraction:
  - Set to 1 when there is no borrow needed.



Zero Flag (Z)

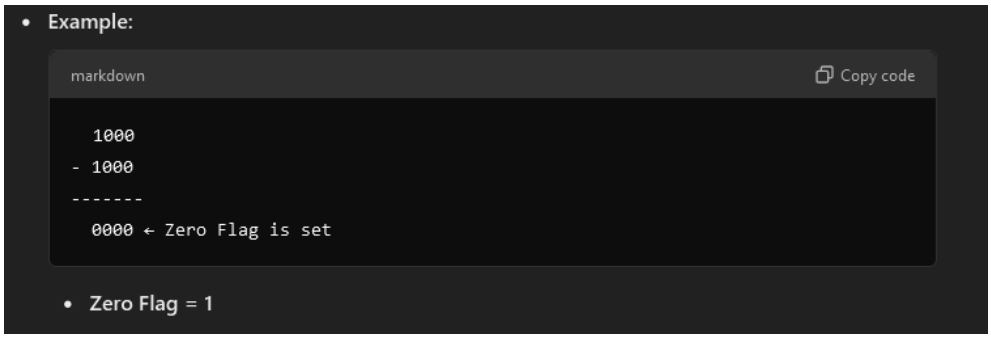
What is the Zero Flag?

- Purpose: The Zero Flag checks if the result of an operation is exactly zero.

- When is it Used: It's useful for comparisons and checking if a value is zero.

When is the Zero Flag Set?

- Set to 1 when the result of an operation is zero.



Sign Flag (S)

What is the Sign Flag?

- Purpose: The Sign Flag shows if the result of an operation is negative.
- How it Works: It copies the Most Significant Bit (MSB) of the result.

When is the Sign Flag Set?

- Set to 1 if the result is negative (MSB = 1).
- Set to 0 if the result is positive (MSB = 0).

Overflow Flag (V)

What is the Overflow Flag?

- Purpose: The Overflow Flag handles signed numbers (numbers that can be positive or negative).
- When is it Used: It checks if the result is too big or has the wrong sign for the given numbers.

When is the Overflow Flag Set?

- **In Addition:**
  - Set to 1 when adding two positive numbers gives a negative result.
  - Set to 1 when adding two negative numbers gives a positive result.
- **In Subtraction:**
  - Set to 1 when subtracting a negative number from a positive number results in a negative.
  - Set to 1 when subtracting a positive number from a negative number results in a positive.

Bytes and Bits:

1. Byte (Basic Unit)

- Group of 8 bits together
- Can store either:
  - One single character
  - One small number

2. Nibble

- -Half of a byte (4 bits)
- -Also called a "half byte"

3. Important Terms for Bits:

For single bits:

- LSB means "Least Significant Bit" - it's the last number on the right
- MSB means "Most Significant Bit" - it's the first number on the left
- 

For whole bytes:

- LSB means "Least Significant Byte" - the right part
- MSB means "Most Significant Byte" - the left part
- 

Data Type Representations:

Sign and Magnitude

- Leftmost bit shows if number is positive/negative (sign bit)
- Remaining bits show the actual value (magnitude)
- Has a drawback : zero has both positive and negative forms

Two's Complement

- **Made by:**
  - Take one's complement (flip all bits)
  - Add 1 to result
- **Benefits**:
  - Only one way to represent zero
  - Most commonly used in computers
  - Makes subtraction easier

Sign Extension :

- **Definition**: Sign extension is the process of increasing the number of bits in a binary number while keeping the value the same.
- **When Used**: It's often used when converting a smaller bit number (like 8-bit) to a larger bit number (like 16-bit) in signed numbers.
- **How It Works**: The most significant bit (leftmost) is copied to fill the new higher bits. This keeps the sign (positive or negative) correct.
- **Example**:
    - If you have an 8-bit number 1111 1101 (which is -3 in two's complement), and you want to extend it to 16 bits:
        - You copy the leftmost bit (1) to all new higher bits, so it becomes 1111 1111 1111 1101.

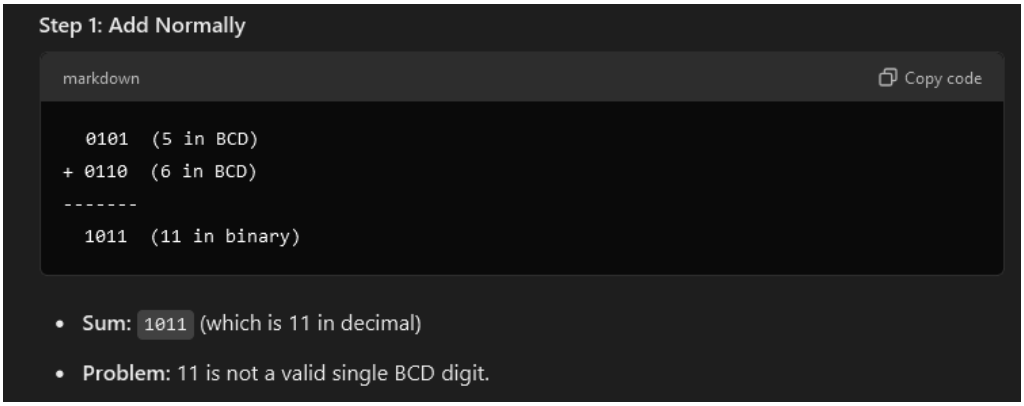
Binary-Coded Decimal (BCD)

What It Is: BCD is a way to represent each decimal digit (0-9) separately in binary.

- **Structure**: Each decimal digit is stored in 4 bits (also called a nibble). For example, decimal 9 is 1001 in BCD.
- **Usage**: Useful in digital displays and calculators where decimal values are directly displayed.
- **Example**:
    - To represent 345 in BCD:
        - 3 becomes 0011
        - 4 becomes 0100
        - 5 becomes 0101
    - So, 345 in BCD is 0011 0100 0101.

Decimal Adjustment :

- **Why It's Needed**: In BCD addition, sometimes the result may go beyond the value for one decimal digit (like getting a 9+1 = 10).
- **How It Works**:
    - If the sum in a BCD operation for a 4-bit segment goes over 9, you add 0110 (which is decimal 6) to adjust it back into BCD range.
- **Steps for Adjustment**:
    - If a 4-bit segment is more than 9 after an addition, add 0110.
    - This keeps the result valid in BCD.
- **Example: Add Two BCD Numbers**
    - Add: 0101 (5) + 0110 (6)

- 

- **Step 2: Check if Adjustment is Needed**
  - Is 1011 > 1001 (9)? Yes.
  - Adjustment Needed: Add 0110 (6 in binary).
- 

- Final BCD Result: 0001 0001 (1 and 1)
- **Step 4: Interpret the Result**
  - 0001 = 1
  - 0001 = 1
  - Combined: 11 in decimal.
- **So, 5 + 6 in BCD equals 11 in decimal, correctly represented as 0001 0001 in BCD.**

Units of Data

1. **Byte**:
   - **Definition**: A byte is a basic unit of data that is made up of 8 bits.
   - **Purpose**: It's used to represent a single character (like a letter or a small integer).
   - **Example**: The letter "A" in ASCII is represented by the byte 01000001.
2. **Half Word**:
   - **Definition**: A half word is a 16-bit data unit (which is 2 bytes).
   - **Use Case**: This term is sometimes used in systems where 16 bits (2 bytes) are often processed together.
3. **Word**:
   - **Definition**: A word is the natural unit of data for a computer, typically based on the CPU's register size.

    - **Common Sizes**:
        - A 32-bit computer has a word size of 4 bytes (32 bits).
        - A 64-bit computer has a word size of 8 bytes (64 bits).
    - **Purpose**: Words are processed as a single unit by the CPU and are used to determine how much data the CPU can handle at once.
4. **Double Word**:
    - **Definition**: A double word is double the size of a word.
    - **Example**: For a 32-bit computer, a double word is 64 bits (8 bytes).
    - **Use**: Double words allow the CPU to work with larger numbers or more complex data types.
5. **Quad Word**:
    - **Definition**: A quad word is four times the size of a word.
    - **Example**: On a 32-bit system, a quad word would be 128 bits (16 bytes).
    - **Use**: Quad words are used in specialized computing where large data sizes are necessary.

Precision in Data Units

1. **Single Precision**:
    - **Definition**: Uses one word (the computer's natural word size) to represent data.
    - **Example**: On a 32-bit computer, single precision would mean storing data in 32 bits.
2. **Double Precision**:
    - **Definition**: Uses two words (double the computer's word size) to represent more detailed or accurate data.
    - **Example**: On a 32-bit computer, double precision would use 64 bits for higher accuracy.
3. **Character Representation**:
    - **ASCII (American Standard Code for Information Interchange)**: A common encoding for characters where each character (like letters and symbols) is represented by a unique byte.
    - **EBCDIC (Extended Binary Coded Decimal Interchange Code)**: Another encoding standard, mostly used in IBM mainframes.

Floating-Point Data

1. **What is Floating-Point Data?**
    - Floating-point numbers are used to represent very large or very small numbers in a compact form, similar to scientific notation.
    - A floating-point number has four main parts:
        - **Sign**: Indicates if the number is positive or negative.

- **Mantissa (or significant)**: The main part of the number.
- **Radix (or base)**: The base for the exponent (usually 2 for binary systems).
- **Exponent**: Represents the power to which the radix is raised.

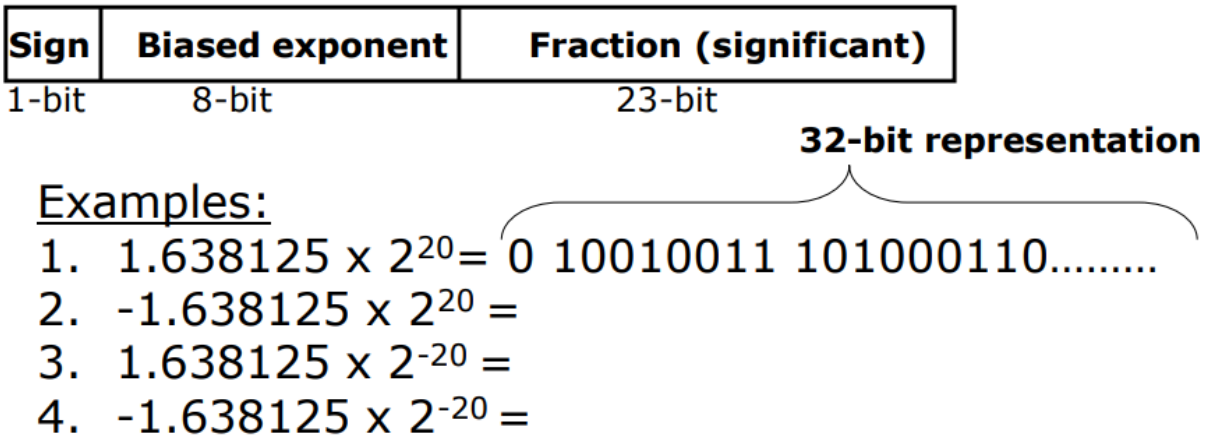
Floating-point number **F**, let **S** be the value of sign bit (0 if positive, 1 if negative), **M** the mantissa, **E** the Exponent and **R** the radix

Value F is $(S) \times M \times R^E$

2. **How Floating-Point Works**:
   - A large or small number, like 976,000,000,000,000, can be written in scientific notation as 9.76 x 10^14.
   - Similarly, in floating-point format, it's broken down into a sign, mantissa, radix, and exponent to save space.
3. **How Number Normalization Works**
   - **Steps to Normalize a Number:**
     1. **Start with the Original Number:**
        - Example: 0.000123
     2. **Shift the Decimal (Binary Point) Left or Right:**
        - Move the decimal point **left** until the **most significant digit** (first non-zero digit) is just before the decimal.
        - Each shift changes the **exponent** accordingly.
     3. **Adjust the Exponent:**
        - **Shifting the point left → exponent goes up** (i.e., is more positive).
        - **Shifting the point right → exponent goes down** (i.e., becomes more negative).
     4. **Result:**
        - A **normalized number** where the first digit is non-zero.
        - Example: 1.23×10^{-4}
4. **Binary Floating-Point**:
   - In computing, the radix (or base) is usually 2, not 10, so numbers are expressed in powers of 2.
   - Example: +18.5 can be written in binary floating-point form as +1.15625 x 2^4.
5. **Components in Memory**:
   - The **bits of a floating-point number** store the sign, mantissa, and exponent.
   - The computer assumes the radix (usually base 2) in floating-point arithmetic.
6. **32-Bit Floating-Point Format**
   - A **32-bit floating-point number** is divided into three parts:
     - **Sign Bit (1 bit):**
       1. **0** for **positive** numbers.
       2. **1** for **negative** numbers.
     - **Exponent (8 bits):**
       1. Stores the **power** to which the base (**2**) is raised.

2. **Biased Exponent:** To handle both positive and negative exponents, a **bias** is added.

- **Fraction (23 bits):**
  1. Also known as the **Mantissa** or **Significant**.
  2. Represents the **precision** of the number



IEEE Floating-Point Standard

The IEEE (Institute of Electrical and Electronics Engineers) Floating-Point Standard is a common system for representing floating-point numbers in computers.

1. **Why IEEE Standard?**
   - To ensure that floating-point numbers are represented consistently across different computer systems.
   - It sets rules for how floating-point numbers are stored, making calculations more reliable.
2. **IEEE 32-bit Floating-Point Format (Single Precision)**:
   - **Sign bit**: 1 bit, which shows if the number is positive (0) or negative (1).
   - **Exponent**: 8 bits, biased by 127 (the value 127 is added to the actual exponent to store it as a positive number).
   - **Mantissa (or significant)**: 23 bits, representing the main part of the number.
   - **Example**: In the 32-bit format, +1.638125 x 2^20 would be represented with its sign, biased exponent, and mantissa stored in these three sections.
3. **IEEE 64-bit Floating-Point Format (Double Precision)**:
   - **Sign bit**: 1 bit.
   - **Exponent**: 11 bits, with a bias of 1023.
   - **Mantissa**: 52 bits, giving more precision than single precision.

4. **Normalization**:
    - The mantissa is usually normalized, meaning the binary point is placed just after the first significant bit. This helps to maintain precision.
    - Normalizing ensures a unique representation of numbers by adjusting the exponent and shifting the mantissa.
5. **Exponent Bias**:
    - To handle both positive and negative exponents, the IEEE standard uses a **bias** (127 for 32-bit and 1023 for 64-bit).
    - For example, if an exponent is 5, the stored exponent is 5 + 127 = 132 in the 32-bit format.

Example :

Steps to Convert 1.638125 × 2^{20} into 32-Bit Format

Step 1: Identify the Sign Bit

- **Number:** 1.638125 × 2^{20}
- **Is it positive or negative?**
- **Answer: Positive**
- **Sign Bit: 0**

Step 2: Normalize the Number

- We need to express the number in the form:

- But our number is already in this form:
- **Normalized Number:** 1.638125 × 2^{20}
- So, the **mantissa** is **1.638125**, and the **exponent** is **20**.

Step 3: Calculate the Exponent

- Use the Exponent Bias
- Bias for 32-bit floats: **127**
- Calculate the Biased Exponent

```mathematica
Biased Exponent = Actual Exponent + Bias
```

- Actual Exponent (E): 20
- Biased Exponent: 20 + 127 = 147

Step 4: Convert the Biased Exponent to Binary

- 147 in Decimal to Binary
- Binary Exponent: **10010011**

Step 5: Find the Mantissa (Fraction)

- Remove the Leading 1
  - In IEEE 754 format, the **leading 1** before the binary point is **implicit** (not stored).
  - So we only need the fractional part after the binary point.
- Convert the Fractional Part to Binary
  - Fractional Part to Convert**:** 0.638125
  - We'll convert 0.638125 to binary
  - Steps to Convert Fractional Decimal to Binary
    1. Multiply the Fraction by 2:

```yaml
0.638125 x 2 = 1.27625 → Bit: 1
```

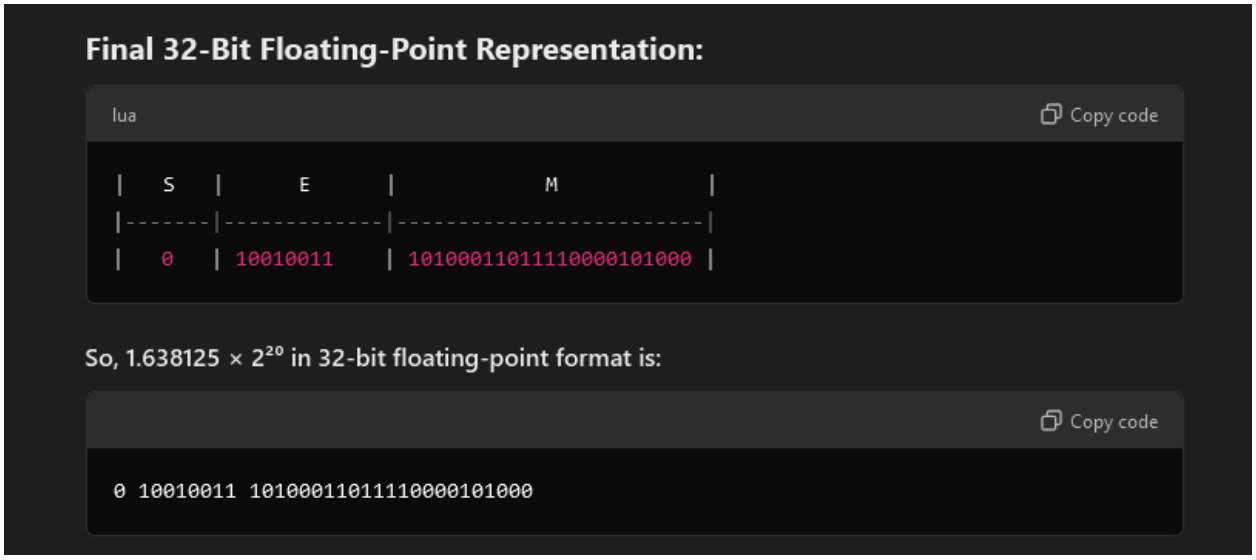
- Take the integer part (1) as the next bit.
- New Fraction: 0.27625

2. Repeat the Process
3. Collect the Bits in Order
4. **First 23 Bits (since mantissa is 23 bits)**
5. Mantissa (Fractional Part): **10100011011110000101000**

Step 6: Putting It All Together

- Now, we have all three parts:
  - **Sign Bit (S)**: 0
  - **Exponent (E)**: 10010011
  - **Mantissa (M)**: 10100011011110000101000

Final 32-Bit Floating-Point Representation:



Logic Gates

Basic Logic Gates:

- Logic gates are the building blocks of digital circuits. They process binary inputs (0s and 1s) and produce a single binary output.
- Common gates include **AND, OR, NOT, NAND, NOR, XOR, and XNOR**. Each gate has a unique function in how it manipulates the binary inputs to give an output.

AND Gate:

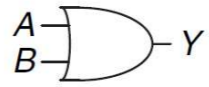
- An AND gate gives an output of 1 only if **all** its inputs are 1.
- For example, if both inputs are 1, the output is 1; otherwise, it's 0.

$Y = AB$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate:

- An OR gate gives an output of 1 if **at least one** of its inputs is 1.
- If either or both inputs are 1, the output will be 1; if both are 0, then the output is 0.



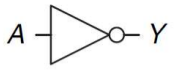
$Y = A + B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

OR gate

NOT Gate:

- A NOT gate has only one input. It simply inverts the input.
- If the input is 1, the output is 0, and if the input is 0, the output is 1.



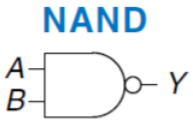
$Y = \overline{A}$

A	Y
0	1
1	0

NOT gate

NAND Gate:

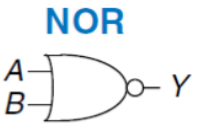
- The NAND gate is the opposite of the AND gate.
- It gives an output of 0 only when **all** inputs are 1. For other input combinations, the output is 1.



$Y = \overline{AB}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

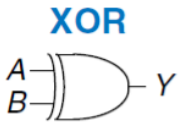
NOR Gate:

- The NOR gate is the opposite of the OR gate.
- It gives an output of 1 only when **all** inputs are 0.
- 

$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

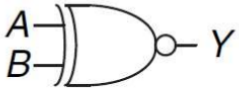
XOR Gate (Exclusive OR):

- The XOR gate gives an output of 1 if **only one** of its inputs is 1.
- If both inputs are the same (both 0 or both 1), the output is 0.
- 

$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XNOR Gate (Exclusive NOR):

- The XNOR gate is the opposite of XOR.
- It gives an output of 1 when both inputs are the same (either both 0 or both 1), and 0 when inputs are different.
- 

$$Y = \overline{A \oplus B}$$

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

Multiple Input Logic Gates:

- Logic gates, like AND, OR, and others, can have more than two inputs. This lets them process more complex logic.
- **Multiple-input AND Gate**:
  - An AND gate with multiple inputs will output 1 only if **all** inputs are 1.
  - For example, with inputs A, B, and C, the output is 1 only if A = 1, B = 1, and C = 1.
- **Multiple-input OR Gate**:
  - An OR gate with multiple inputs will output 1 if **any one** of the inputs is 1.
  - If even one of A, B, or C is 1, the output is 1.
- **Multiple-input XOR Gate**:
  - An XOR gate with multiple inputs is more complex. Typically, it outputs 1 if an **odd number** of inputs are 1.
  - For example, with inputs A, B, and C, the output is 1 if exactly one or all three of the inputs are 1.

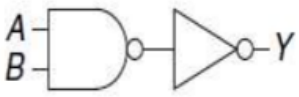


Figure 1.37 Two-input AND gate schematic

- **Problem**: Directly building an AND gate with CMOS is difficult.
- **Solution**: Use a **NAND gate** followed by a **NOT gate** to simulate an AND gate.

Logic Explanation

1. **NAND Gate**: Outputs 0 only if **both inputs (A and B) are 1**. For any other input, it outputs 1.
2. **NOT Gate**: Inverts the output of the NAND gate.

Result

- The combination of NAND and NOT gates produces the same output as an AND gate:
  - Output Y is 1 only when **both inputs A and B are 1**.
  - For any other combination, Y is 0.

Lecture 2

Introduction to Circuits

1. **What is a Circuit?**
   - A digital circuit is a network that processes discrete-valued (binary) variables.
   - It can be represented as a **black box** with:
     - **Inputs**: One or more discrete-valued terminals where data enters.
     - **Outputs**: One or more discrete-valued terminals where results exit.
     - **Functional Specification**: Describes how inputs are related to outputs mathematically or logically.
     - **Timing Specification**: Describes the time delay between changes in inputs and corresponding changes in outputs.
2. **Internal Structure of Circuits**
   - Inside the black box, circuits consist of **nodes** and **elements**:
     - **Nodes**: Connection points, classified as:
       - **Input Nodes**: Where input data is received.
       - **Output Nodes**: Where results are output.
       - **Internal Nodes**: Points between components for internal processing.
     - **Elements**: Small, self-contained circuits that process inputs and provide outputs.

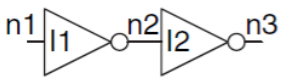
Classification of Circuits

1. **Combinational Circuits**
   - Outputs depend only on the current values of inputs.
   - These circuits compute outputs directly from inputs without any memory of past states.
   - **Example**: A logic gate, such as an OR gate:
     - Outputs "1" if at least one input is "1".
   - **Key Property**: Memoryless (no history of past inputs).

   Rules for Combinational Circuits

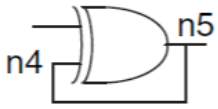
   A circuit is considered combinational if:

- Each component within the circuit is itself combinational.
- Every node is either:
  - An input to the overall circuit, or
  - Connected to exactly one output terminal of a circuit element.
- The circuit has no feedback loops (cyclic paths where signals loop back into the input).
- This is Combinational :



(a)

-
- This is not (because there is a loop):



(b)

-

2. **Sequential Circuits**
   - Outputs depend on both current and previous input values.
   - These circuits "remember" past input sequences, making them useful for systems requiring memory.
   - **Example**: Flip-flop (used to store binary data).
   - **Key Property**: Has memory to retain past inputs.

Boolean Equations

Functional Description

- Combinational circuits are often described using **truth tables** or **Boolean equations**.
- Boolean equations express logical relationships between variables using logical operators (AND, OR, NOT).
- **Key Terminology**
  - **Complement**:
    - "Opposite" of a variable.
    - Example: If A is 1, the complement (written as $\overline{\mathbf{A}}$) is 0.
  - **Literal**:
    - A variable (like A) or its complement (like $\overline{A}$).
  - **Product (AND)**:

        - Multiply literals together using the **AND** rule.
        - Example: A · B means "A AND B" (both A and B must be 1 for the result to be 1).
    - **Sum (OR)**:
        - Add literals together using the **OR** rule.
        - Example: A + B means "A OR B" (if A or B is 1, the result is 1).
    - **Minterm**:
        - A product (AND) that includes **all input variables**.
        - Example: A · B · C means "A AND B AND C".
    - **Maxterm**:
        - A sum (OR) that includes **all input variables**.
        - Example: A + B + C means "A OR B OR C".

Order of Operations

- Boolean equations follow a strict precedence, similar to algebra:
    - **NOT** (highest priority) → **AND** → **OR** (lowest priority).
- Example:
    - Y = A + BC means Y=A OR (B AND C).

Sum-of-Products (SOP) Form

Definition

- It's called "sum-of-products" because it sums (ORs) multiple products (ANDed terms). (**SOP means ORing (adding) multiple ANDed terms.**)

Steps to Create SOP:

1. Look at a **truth table** and find rows where the **output (Y) = 1**.
2. Write **a minterm (AND rule)** for each of these rows:
    - Use the variable directly if it's 1.
    - Use its complement if it's 0.
3. OR (add) all the minterms together.

Examples

-

A	B	Y	minterm	minterm name
0	0	0	$\overline{A}\,\overline{B}$	m_0
0	1	1	$\overline{A}\,B$	m_1
1	0	0	$A\,\overline{B}$	m_2
1	1	1	$A\,B$	m_3

- For this table , the **minterms** are $\overline{A}B$ and AB. **SOP** for this table is : $Y = \overline{A}B + AB$.

Sigma Notation

- SOP form can be compactly written using **Σ** notation:
  - Y= Σ(m1,m3) where m1 and m3 are rows in the truth table where Y=1.

Product-of-Sums (POS) Form

Definition

- It's called "product-of-sums" because it multiplies (ANDs) multiple sums (ORed terms). (**POS means ANDing (multiplying) multiple ORed terms**.)

Steps to Create POS:

1. Look at a **truth table** and find rows where the **output (Y) = 0**.
2. Write a maxterm (OR rule) for each of these rows:
3. Use the variable's complement if it's 1.
4. Use the variable directly if it's 0.
5. AND (multiply) all the maxterms together.

Examples

A	*B*	*Y*	maxterm	maxterm name
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	1	$\overline{A} + \overline{B}$	M_3

-
- For this table , the **maxterms** are $A + B$ and $\overline{A} + B$.
- **POS** for this table is : $Y = (A + B)(\overline{A} + B)$

Pi Notation

- POS form can also be compactly written using Π notation:
  - Y=Π(M0,M2) , where M0 and M2 are rows in the truth table where Y=0.

Comparison of SOP and POS

1. **When to Use SOP**:
    - SOP is more efficient when there are fewer 1′s (TRUE rows) in the truth table.
2. **When to Use POS**:
    - POS is more efficient when there are fewer 0′s (FALSE rows) in the truth table.

Boolean Algebra and Digital Circuit

Boolean Algebra : Boolean algebra is used to simplify Boolean equations, which are essential for designing efficient digital circuits. Simplification reduces the size and cost of the circuit.

Axioms of Boolean Algebra

Basic Axioms:

1. **A1: Binary Field**
    - Rule: B=0 if B≠1 :
        - This means in Boolean algebra, a variable B can only be 0 or 1.
        - There's no other option.
    - Dual: B=1 if B≠0
        - Similarly, if B isn't 0, it must be 1.

---

2. **A2: NOT (Complement Rule)**
    - Rule: $0^{-}=1$
        - The NOT of 0 is 1. (If something is "false," NOT makes it "true.")
    - Dual: $1^{-}=0$
        - The NOT of 1 is 0. (If something is "true," NOT makes it "false.")

---

3. **A3: AND/OR Rule for Zero**
    - Rule: $0 \cdot 0=0$
        - In AND, multiplying 0 with 0 always gives 0. (Both inputs must be 1 for AND to give 1.)
    - Dual: 1+1=1
        - In OR, adding 1 with 1 gives 1. (If either input is 1, OR gives 1.)

---

4. **A4: AND/OR Rule for One**
    - Rule: $1 \cdot 1=1$
        - In AND, multiplying 1 with 1 gives 1. (Both inputs are true, so the result is true.)

    - Dual: 0+0=0
        - In OR, adding 0 with 0 gives 0. (Both inputs are false, so the result is false.)

- **A5: Mixing AND and OR with Zero and One**
    - Rule: $0 \cdot 1 = 0$ and $1 \cdot 0 = 0$
        - If either input in AND is 0, the result is 0.
    - Dual: 1+0=1 and 0+1=1
        - If either input in OR is 1, the result is 1.

- **Axioms theorems**
  - **Identity**: A + 0 = A and A · 1 = A.
  - **Null**: A + 1 = 1 and A · 0 = 0.
  - **Idempotent**: A + A = A and A · A = A.
  - **Complement:** A + A' = 1 and A · A' = 0.
  - **Commutative:** A + B = B + A and A · B = B · A.
- **Principle of Duality:**
  - Every Boolean equation has a dual form.
  - Example: Original: A + 0 = A. Dual: A · 1 = A.

Theorems of Boolean Algebra

- **Theorems for a Single Variable:**
  - A + A' = 1 **(Complementation).**
  - A · A' = 0 **(Exclusion).**
  - A'' = A **(Double Negation)**.
- **Theorems for Multiple Variables:**
  - **Distributive Law:** A · (B + C) = (A · B) + (A · C).
  - **Absorption**: A + (A · B) = A.
  - **Consensus:** AB + A'C + BC = AB + A'C.

Additional Theorems

T6: Commutativity
Rule: B · C = C · B (AND)
Dual: B + C = C + B (OR)
Explanation: The order of inputs doesn't matter in both AND and OR operations.

T7: Associativity
Rule: (B · C) · D = B · (C · D)
Dual: (B + C) + D = B + (C + D)

Explanation: Grouping doesn't matter in both AND and OR operations.

T8: Distributivity
Rule: (B · C) + (B · D) = B · (C + D)
Dual: (B + C) · (B + D) = B + (C · D)
Explanation: Similar to factoring in algebra, distribution works for Boolean operations.

T9: Covering
Rule:B · (B + C) = B
Dual: B + (B · C) = B
Explanation: If B is true, adding or multiplying with C doesn't change the result.

T10: Combining
Rule:(B · C) + (B · ¬C) = B
Dual:(B + C) · (B + ¬C) = B
Explanation: If B is true, the result doesn't depend on whether C is true or false.

T11: Consensus
Rule: (B · C) + (¬B · D) + (C · D) = (B · C) + (¬B · D)
Dual: (B + C) · (¬B + D) · (C + D) = (B + C) · (¬B + D)
Explanation: Redundant terms can be removed without changing the logic.

T12: De Morgan's Theorem
Rule: ¬(B0 · B1 · B2 …) = ¬B0 + ¬B1 + ¬B2 …
Dual: ¬(B0 + B1 + B2 …) = ¬B0 · ¬B1 · ¬B2 …
Explanation: NOT of AND is OR of NOTs, and NOT of OR is AND of NOTs.

De Morgan's Theorems

- De Morgan's theorems provide a way to transform AND operations into OR operations and vice versa by inverting inputs and outputs:

$$Y = \overline{AB} = \overline{A} + \overline{B} \qquad Y = \overline{A + B} = \overline{A}\,\overline{B}$$

Simplification and Optimization of Digital Circuits

Simplifying Boolean equations reduces the complexity of digital circuits. This leads to smaller, faster, and more cost-effective circuits.

- **Example:**
  Consider a truth table-derived sum-of-products equation:

$Y = A \cdot B + A \cdot B'$.

Simplification process:

1. Apply the distributive property: $Y = A \cdot (B + B')$.
2. Use the complementation rule: $B + B' = 1$.
3. Simplify further: $Y = A \cdot 1 = A$.

Idempotency Theorem allows duplication of terms if needed. For example: B = B + B + B + ...

Minimizing Equations: Identify and eliminate redundant terms using Boolean theorems. For example: AB + A'C + BC simplifies to AB + A'C using the Consensus Theorem.

Multi-Level Combinational Logic

Multi-level logic involves circuits with more than two levels of logic gates. These are common in real-world designs where single-level designs (sum-of-products or product-of-sums) are inefficient.

Example: XOR Gate

A 3-input XOR gate can be implemented using multi-level logic.
Equation: $Y = A \oplus B \oplus C$ or $Y = A \cdot B' \cdot C' + A' \cdot B \cdot C$.

Multi-Level Logic Benefits:

- Reduces the number of gates and connections.
- Optimizes overall performance for larger circuits.

Karnaugh Maps (K-Maps)

What are Karnaugh Maps?

- **Karnaugh Maps (K-Maps)** provide a graphical method to simplify Boolean equations.
- Invented by **Maurice Karnaugh** in 1953, they are widely used for minimizing logic functions with up to four variables.

Why Use K-Maps?

1. Simplifies Boolean equations visually.
2. Helps identify opportunities to combine terms.
3. Reduces circuit complexity by combining **implicants** efficiently.

How K-Maps Work

Structure

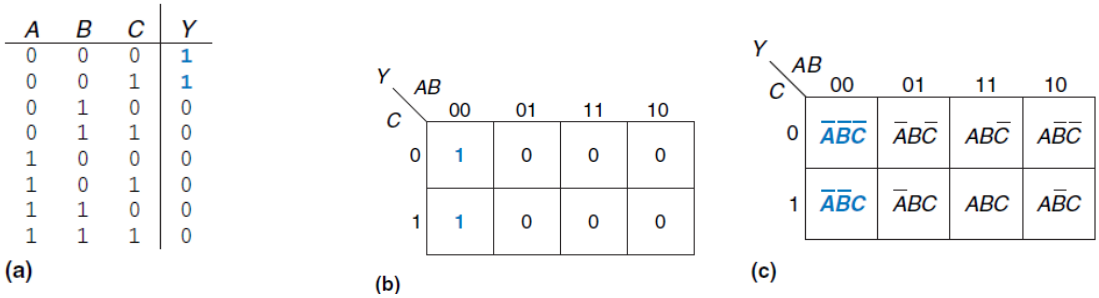
- K-maps are grids where each square corresponds to a **minterm** (a row in the truth table).
  - Rows and columns represent variable combinations.
  - Each cell corresponds to a truth table row.
  - The cell value represents the output for the given input combination.

Variable Representation

- Input variables are split between rows and columns.
- **Example for 3 Variables (A, B, C)**:
  - Row values: C (0 or 1).
  - Column values: AB (Gray code: 00, 01, 11, 10).

Filling the Map

- Each cell is filled with the output value (0 or 1) for the corresponding input combination.



A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

(a)

Y \ AB	00	01	11	10
C = 0	1	0	0	0
C = 1	1	0	0	0

(b)

Y \ AB	00	01	11	10
C = 0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
C = 1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

(c)

Truth table K-map K-map showing minterms

Minimizing Logic with K-Maps

Circle Adjacent 1's

- Look for groups of 1's in the K-Map.
- Combine these into implicants (terms) that cover the 1's efficiently.

Example :

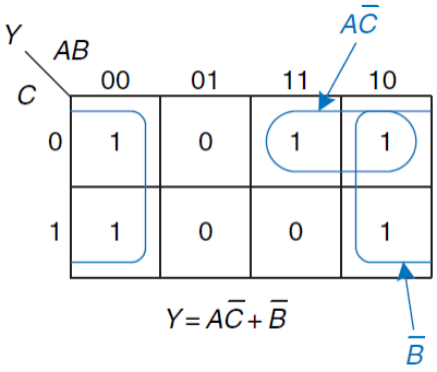


Figure 2.46 Solution for Example 2.9

Step 1: Group the 1's

1. **Rules for Grouping:**
   - Groups must be rectangular and include 1, 2, 4, or 8 squares (powers of 2).
   - Groups can wrap around the edges of the K-map.
   - A square containing 1 can be part of multiple groups if needed.
2. **Identify Groups:**
   - **Group 1**: The two 1's in the leftmost column (A=0,B=0 and C=0,C = 1).
     - These form a vertical rectangle of size 2×1, so C varies but A and B are constant ($A^{-}B^{-}$).
   - **Group 2**: The two 1's in the rightmost column (A=1,B=0 and C=0,C=1).
     - These form another vertical rectangle of size 2×1, so A and C varies but B remains constant (B^{-}).
   - **Group 3**: The two 1's in the top row (C=0, A,B=11 and A,B=10).
     - These form a horizontal rectangle of size 1×2, so B varies but A and C is constant (AC^{-}).

Step 2: Write the Simplified Expression

1. For each group, write the **prime implicant** by including:
   - Variables that **don't change** within the group (those that are constant for all 1's in the group).
   - Exclude variables that **do change**.
2. **Result:**
   - Group 1: $A^{-}B^{-}$ (from the leftmost vertical group).
   - Group 2: B^{-} (from the rightmost vertical group).
   - Group 3: AC^{-}(from the horizontal group).
3. Combine the terms:
   $Y=AC^{-}+B^{-}$

Prime Implicants

- The largest possible groups of adjacent 1's that cannot be combined further.
- Prime implicants reduce the Boolean equation to its simplest form.

Don't Care Conditions

What are Don't Care Conditions?

- Sometimes, certain input combinations are not possible or irrelevant.
- These cases are represented by **X** in the truth table or K-Map.
- Designers have the freedom to treat XXX as 000 or 111, whichever helps simplify the circuit.

How to Use Don't Cares

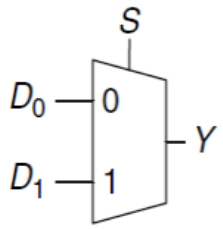
1. Include XXX in groups of 1's if it simplifies the equation.
2. Ignore XXX if it does not contribute to minimization.
3. Example:
    - XXX allows a rectangle to expand, combining more 1's.

What is a Multiplexer (MUX)?

- **A multiplexer (or MUX)** is a device that selects one input signal from multiple input signals and forwards it to the output. The selection is controlled by control signals (also called select signals).
    - For example, if you have two inputs, **D0** and **D1**, and a select signal **S**, the multiplexer will send either **D0** or **D1** to the output **Y**, depending on the value of **S**.

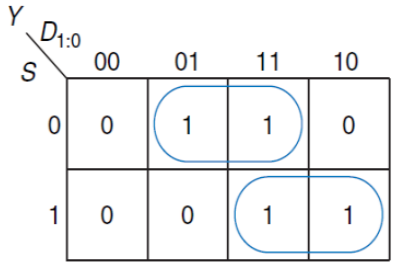
The 2:1 Multiplexer

- This is the simplest type of multiplexer. It has:
    - 2 inputs (D0 and D1),
    - 1 select signal (S), and
    - 1 output (Y).



S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- *When S=0, Y=D0*
- *When S=1, Y=D1*



$Y = D_0\overline{S} + D_1S$

Logic Expression

The output Y is given by: $Y = D_0 \cdot \overline{S} + D_1 \cdot S$

This means:

- $D_0 \cdot \overline{S}$: D_0 is selected when $S = 0$.
- $D_1 \cdot S$: D_1 is selected when $S = 1$.

The 4:1 Multiplexer

- A 4:1 multiplexer works similarly but has:
- **4 inputs** (D0,D1,D2,D3),

- **2 select signals** (S1 and S0),
- **1 output** (Y).

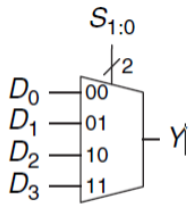


Figure 2.57 4:1 multiplexer

What is a Decoder?

A **decoder** is a combinational circuit with:

1. **N inputs**
2. **2^n outputs**

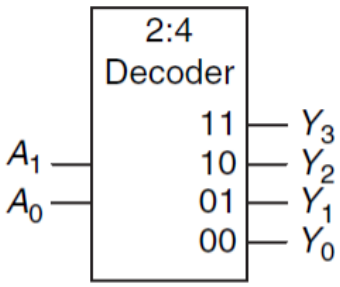
- **Purpose**: It takes a binary input and activates exactly one output while keeping all other outputs low.

It is called "binary to one-hot converter." Only one output is active (HIGH or 1) for any given input combination.

The 2:4 Decoder

This specific decoder has:

- **2 inputs** (A1,A0)
- **4 outputs** (Y3,Y2,Y1,Y0)



A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

- *The input A1, A0 is a binary number.*
- *Depending on the input value (00, 01, 10, or 11), the corresponding output is HIGH (1), while all others remain LOW (0).*

Each output has a unique logic expression:

1. $Y_0 = \overline{A_1} \cdot \overline{A_0}$
2. $Y_1 = \overline{A_1} \cdot A_0$
3. $Y_2 = A_1 \cdot \overline{A_0}$
4. $Y_3 = A_1 \cdot A_0$

Decoders are used in digital systems to enable one output out of many, like activating a specific memory location.

Lecture 3

Combinational vs Sequential Logic

1. **Combinational Logic**:
    - The output depends only on the current input values.
    - It does **not have memory**. For example, circuits like multiplexers and decoders fall under combinational logic.
2. **Sequential Logic**:
    - The output depends on both **current inputs** and **past inputs** (previous states).

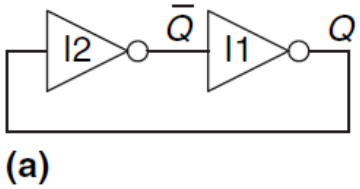
- It has **memory** and stores information about prior inputs using elements like latches and flip-flops.

*Sequential logic remembers its past inputs through **state variables**, which hold the information needed for the system's behavior.*

Memory in Digital Circuits

The simplest form of memory in sequential logic is a **bistable element**. This is a circuit with:

- **Two stable states** (can represent 0 or 1).
- The ability to maintain its state until explicitly changed.



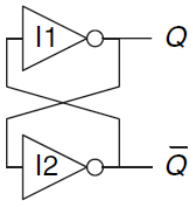
(a)

Bistable Element (Figure 3.1)

1. **What is it?**
   - A bistable element is created using **two cross-coupled inverters.**
   - It stores **one bit of information.**
   - The two states of the bistable element are represented by:
     - $Q = 1, Q' = 0$
     - $Q = 0, Q' = 1$
   - The two outputs, Q and Q', are always complements of each other.

Figure 3.1(a): Cross-Coupled Inverter Pair

- Two inverters (I_1 and I_2) are connected in a loop:
  - The **output of** I_1 is the **input of** I_2.
  - The **output of** I_2 is the **input of** I_1.
- This creates a feedback loop that locks the circuit into one of its two stable states:
  - $Q = 1, Q' = 0$ (one state)
  - $Q = 0, Q' = 1$ (another state)



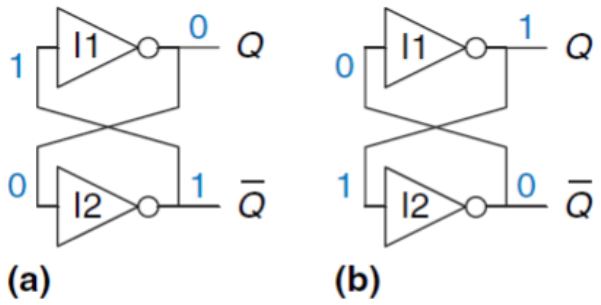
(b)

Bistable elements are used to construct **latches** and **flip-flops**:

- **Latches**: Can store data as long as they are enabled.
- **Flip-Flops**: Edge-triggered devices that store data on clock signals.

They are essential for building:

- Registers
- Counters
- Memory units in CPUs and digital systems.



Cross-Coupled Inverters and Bistable States

The circuit shown in **Figure 3.2** is made of two **inverters** connected in a feedback loop. This creates a **bistable circuit**, meaning the circuit has two stable states:

- $Q = 0$ and $Q' = 1$
- $Q = 1$ and $Q' = 0$

Key Idea:

- The circuit stores **one bit of information** in its stable states.
- It stays in the same state until something external changes it.

Key Characteristics

1. **Bistable Nature**:
   - The circuit has **two stable states**: Q=0,Q′=1 or Q=1,Q′=0 These states represent the binary values **0** and **1**.
2. **Memory**:
   - The circuit **remembers its state** (value of Q) indefinitely.
   - For example, if Q=0, it will stay 0 until something external changes it.
3. **State Variable**:
   - Q is the **state variable**. It stores all the information about the circuit's past that is needed to determine its future behavior.
   - Knowing Q also tells us Q′ because Q′is always the complement of Q

Limitations of Cross-Coupled Inverters

While this circuit can store a bit of information, it has some problems:

1. **No Control Inputs**:
    - There's no way for the user to set or reset the state (Q).
2. **Unpredictable Initialization**:
    - When the circuit is powered on, its initial state (Q) is unknown and might vary each time.
3. **Not Practical**:
    - Although it demonstrates the concept of bistable memory, this circuit is not suitable for real-world applications.

What is an SR Latch?

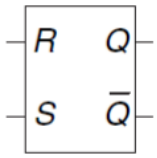
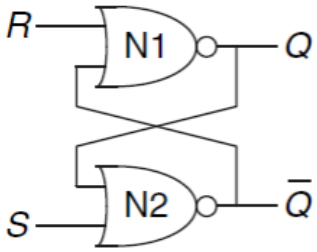


Figure 3.6 SR latch symbol

The **SR latch** is a simple memory circuit used in sequential logic. It:

1. **Stores one bit of information**.
2. Has:
    - **Two inputs**: S (Set) and R (Reset).
    - **Two outputs**: Q (the stored value) and Q' (complement of Q).
3. Uses two **cross-coupled NOR gates**, as shown in **Figure 3.3**.



The SR latch can:

- **Set Q=1** using the S input.
- **Reset Q=0** using the R input.
- **Remember the previous state** when both inputs are 0.

4 Possible cases with SR latches :

Case I: $R = 1, S = 0$

- $R = 1$ forces the output $Q = 0$ (Reset state).
- The second NOR gate ($N2$) sees both inputs as 0 ($Q = 0$ and $S = 0$), so it outputs $\overline{Q} = 1$.

Result:

- $Q = 0, \overline{Q} = 1$ (Reset State).

Case II: $R = 0, S = 1$

- $S = 1$ forces $\overline{Q} = 0$ (Set state).
- The first NOR gate ($N1$) now sees both inputs as 0 ($\overline{Q} = 0$ and $R = 0$), so it outputs $Q = 1$.

Result:

- $Q = 1, \overline{Q} = 0$ (Set State).

Case III: $R = 1, S = 1$

- Both inputs are 1. Both NOR gates see at least one TRUE input, so:
  - $Q = 0$
  - $\overline{Q} = 0$

This case is **invalid** because Q and $\overline{Q}$ are supposed to be complementary, but here they are both 0. This is called the **forbidden state**.

Case IV: $R = 0, S = 0$

- Both inputs are 0. The outputs depend on the **previous state** of the latch (Q_{prev}):
  - If $Q_{prev} = 0$, then $Q = 0, \overline{Q} = 1$.
  - If $Q_{prev} = 1$, then $Q = 1, \overline{Q} = 0$.

This means the latch **remembers** its last state when $S = 0$ and $R = 0$.

The truth table for the SR latch is shown in Figure 3.5. It summarizes the behavior:

Case	S	R	Q	$\overline{Q}$
IV	0	0	Q_{prev}	$\overline{Q}_{prev}$
I	0	1	0	1
II	1	0	1	0
III	1	1	0	0

Figure 3.5 SR latch truth table

Memory:

- When S=0 and R=0 , the SR latch remembers its last state.
- This makes it a basic building block of memory in digital systems.

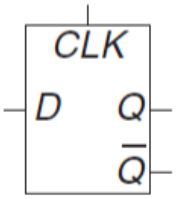
Set and Reset:

- S=1, R=0 : Set the latch (Q=1).
- S=0, R=1: Reset the latch (Q=0).

Invalid State:

- S=1,R=1 : Both outputs become 0, which is invalid for a latch.

What is a D Latch?



(c)

The **D latch** is a sequential circuit designed to store one bit of information. It is an improvement over the SR latch because it eliminates the **invalid state** caused by setting both S and R to 1.

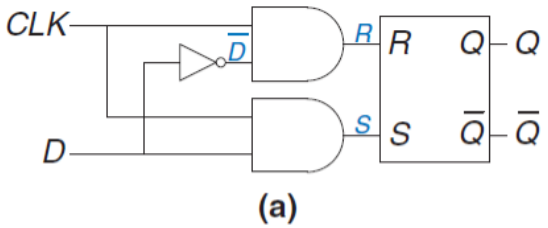
The D latch has:

- **One data input (D)**: This determines the value to be stored.
- **One clock input (CLK)**: This controls when the data is stored.
    - The D input specifies **what** value to store.
    - The CLK input specifies **when** to store the value.

How Does the D Latch Work?

The circuit for the D latch is shown in **Figure 3.7(a)**. It includes:

1. Two **AND gates** that control the S (Set) and R (Reset) inputs of the SR latch.
2. A NOT gate to ensure R and S are complements when CLK=1



(a)

Key States of the D Latch

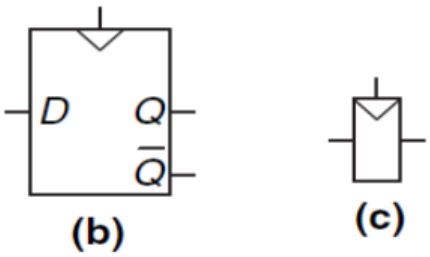
1. When $CLK = 0$:
   - Both AND gates output 0, so:
     - $S = 0$
     - $R = 0$
   - The SR latch is in the **memory state**: Q retains its previous value (Q_{prev}).
2. When $CLK = 1$:
   - The AND gates are enabled, and their outputs depend on D:
     - If $D = 0$, $R = 1$ and $S = 0$: The latch resets ($Q = 0$).
     - If $D = 1$, $S = 1$ and $R = 0$: The latch sets ($Q = 1$).

CLK	D	$\overline{D}$	S	R	Q	$\overline{Q}$
0	X	$\overline{X}$	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

- CLK=0: The latch does nothing (memory state).
- CLK=1: The latch updates its output (Q) to match the input (D).

*D latch is also called a **level-sensitive latch**.*

What is a D Flip-Flop?



The **D flip-flop** is a sequential circuit that:

1. **Stores one bit of information**.
2. Updates its output **Q only on the rising edge of the clock** (when the clock signal goes from 0 to 1).

It is built using two **D latches** arranged in a master-slave configuration:

- **Master latch (L1)**: Captures data when the clock (CLK) is low.
- **Slave latch (L2)**: Captures data when the clock (CLK) is high

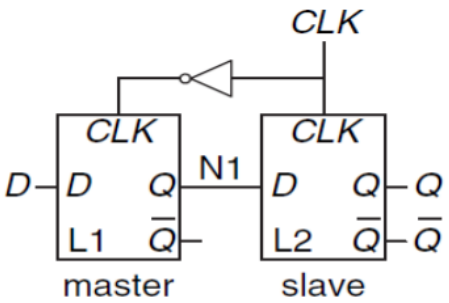
Edge-Triggered Behavior:

- Unlike the D latch, the flip-flop updates its output Q only on the **rising edge of the clock**.
- This makes it easier to synchronize with other circuits.

Prevents Glitches:

- The master-slave design ensures that D cannot directly affect Q except during the clock transition, avoiding errors.

How Does It Work?



- **Two Latches**:
  - **L1 (Master)**: Transparent when CLK=0
  - **L2 (Slave)**: Transparent when CLK=1
  - **Node N1** : The intermediate connection between L1 and L2.

When CLK=0:

1. **L1 (Master)** is **transparent**:
    - This means the first part (L1) lets the input D pass through it.
    - The value of D moves to the middle point **N1**.
2. **L2 (Slave)** is **opaque**:
    - This means the second part (L2) is "closed" and does not change.
    - The output Q stays the same as before.

When CLK=1:

1. **L1 (Master)** is **opaque**:
    - The first part (L1) is "closed" and does not take any new input.
    - The middle point **N1** is now locked with the value it had when CLK changed.
2. **L2 (Slave)** is **transparent**:
    - The second part (L2) is "open" and lets the value at the middle point (N1) pass to the output Q.

- The flip-flop **copies the value of D** to Q on the **rising edge of the clock**:
    - Right before the clock transitions from 0 to 1, N1 holds the value of D.
    - After the clock transitions to 1, Q is updated to match N1.

At all other times, Q retains its previous value.

What is a Register?

A **register** is a group of flip-flops used to store multiple bits of data. It's like a "memory box" that holds more than one bit at a time.

N-bit Register: If a register can store N bits, it contains N flip-flops.

- Example: A 4-bit register has 4 flip-flops, one for each bit.

Shared Clock (CLK): All flip-flops in the register share the same clock signal. This means they all update their stored values **at the same time**.

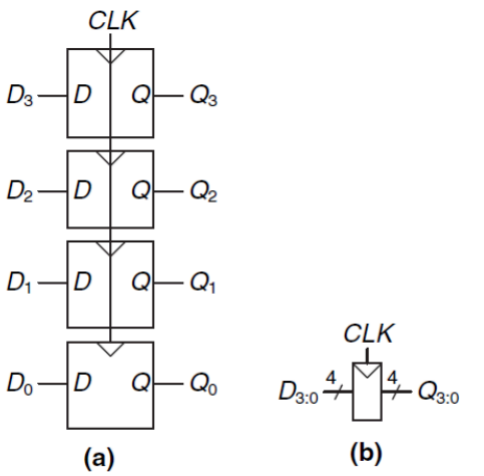
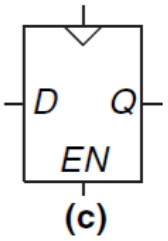
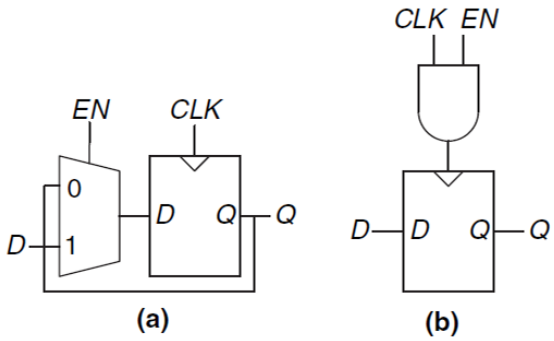


Figure 3.9 A 4-bit register: (a) schematic and (b) symbol

What is an Enabled Flip-Flop?



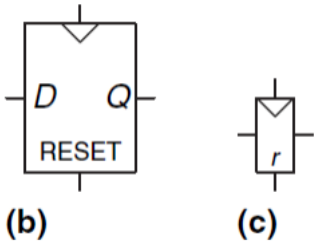


An **enabled flip-flop** is a modified version of the D flip-flop. It has an extra input called **ENABLE (EN)** that decides whether the flip-flop should store new data or keep its current value.

1. **When EN=1**:
   - The flip-flop behaves like a normal D flip-flop.
   - It updates its output Q on the clock edge based on the input D.
2. **When EN=0**:

- The flip-flop ignores the clock signal and **retains its previous value**.
- This is useful when you want the flip-flop to "pause" and hold its current state.

What is a Resettable Flip-Flop?



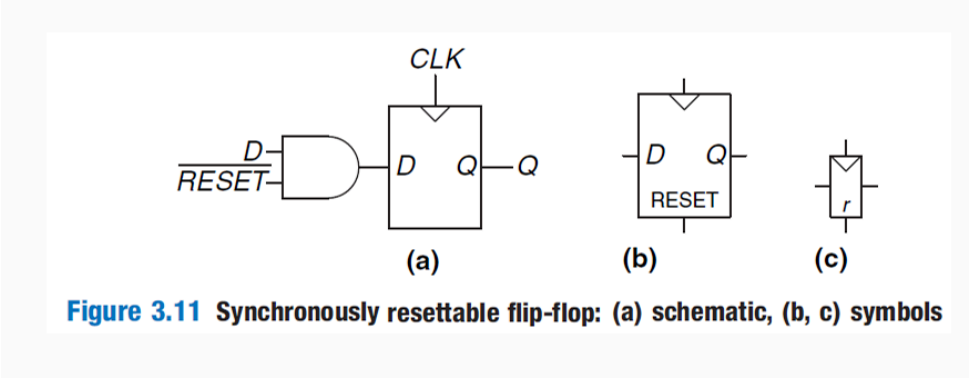


Figure 3.11 **Synchronously resettable flip-flop: (a) schematic, (b, c) symbols**

A **resettable flip-flop** is a modified version of a D flip-flop. It has an extra input called **RESET** that forces the output (Q) to 0, regardless of the input (D).

- **When RESET=0:**
  - The flip-flop works like a normal D flip-flop.
  - The output Q follows the input D on the clock's rising edge.
- **When RESET=1:**
  - The flip-flop **ignores D** and **forces Q=0** (reset state).
  - This is useful to initialize the system to a known state when powered on.

- **Latches and flip-flops** are the foundation of all sequential circuits.
- The D latch is **simple** and **level-sensitive**.
- The D flip-flop is **precise** and **edge-triggered**, making it more reliable for synchronization.
- Registers and resettable flip-flops add more functionality for storing and controlling data.

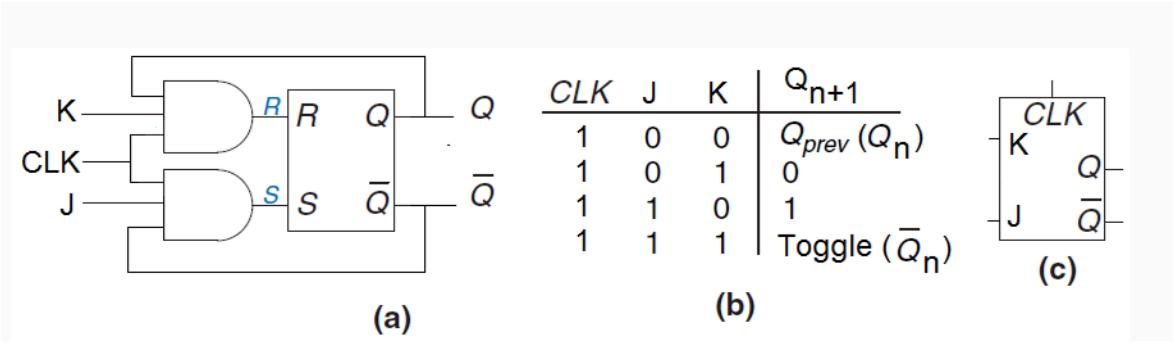
Key Differences

Feature	D Latch	D Flip-Flop	Register
Triggered By	CLK (level-sensitive)	Rising edge of CLK (edge-triggered)	Common clock for all bits
Purpose	Stores one bit temporarily	Stores one bit permanently until next clock edge	Stores multiple bits
Reset Function	Not available	Can be added (resettable flip-flop)	Can include resettable flip-flops

---

J-K Latch Explanation

What is a J-K Latch?



- A **J-K latch** is an improved version of the SR (Set-Reset) latch. It fixes the problem of the forbidden state in the SR latch.
- In an SR latch, when both inputs (S and R) are **1**, the outputs Q and Q^- become **0**, which is invalid.
- The J-K latch solves this by modifying the circuit so that it works properly when J=K=1

How Does a J-K Latch Work?

1. **Inputs:**
   - J: Similar to the "Set" input in an SR latch.
   - K: Similar to the "Reset" input in an SR latch.
2. **Outputs:**
   - Q: Main output.
   - Q^-: Complement of Q.
3. **Behavior:**
   - When J=0 and K=0 : The latch keeps its previous state (Q stays the same).
   - When J=0 and K=1: Q=0, Q^-=1 (Reset).
   - When J=1 and K=0 : Q=1, Q^-=0 (Set).
   - When J=1 and K=1: Q toggles (switches between 0 and 1).

Truth Table of J-K Latch

Clock (CLK)	J	K	Next Output Q_{n+1}
1	0	0	Q_{prev} (No Change)
1	0	1	0 (Reset)
1	1	0	1 (Set)
1	1	1	Toggle ($\overline{Q_{\text{prev}}}$)

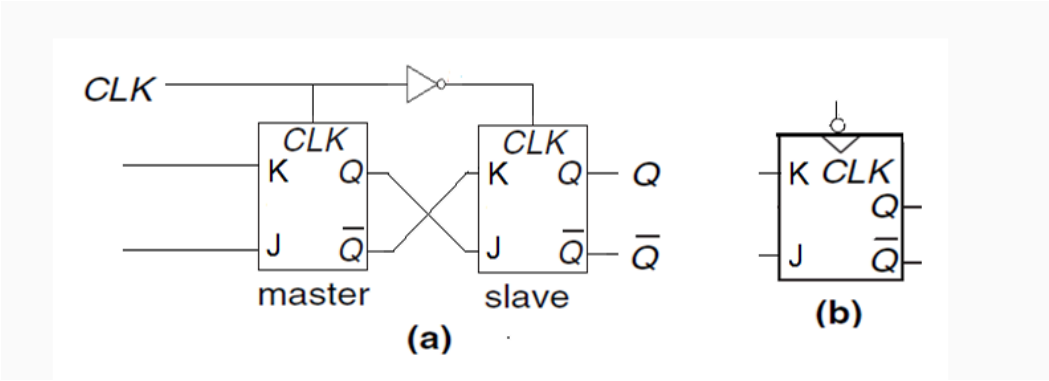
How Does it Prevent Invalid State?

- When both inputs are **1**, the J-K latch toggles the output instead of creating an invalid state.
- This ensures the latch is always in a valid condition.

Race Around Condition

- **What is it?**
  - If the clock signal is **high for too long**, the output toggles repeatedly in one clock cycle. This is called a **race around condition**.
- **Solution:**
  - Convert the J-K latch into a **J-K Flip-Flop** by adding a clock edge-triggering mechanism. This ensures the latch works properly during a clock cycle.

J-K Flip-Flop



A **J-K flip-flop** is an advanced flip-flop based on the J-K latch. It removes the race-around condition and ensures stable operation.

Key Features:

- **Master-Slave Design**:
  - The circuit has two parts:
    - **Master flip-flop**: Active during the clock's HIGH phase.
    - **Slave flip-flop**: Active during the clock's LOW phase.
  - This ensures only one state change per clock pulse, solving the race-around issue.
- **Operation**:
  - When J=K=1 , the output toggles (changes from 0 to 1 or 1 to 0).
  - When J=0,K=0, the state is maintained.
  - When J=1,K=0, the output is set (Q = 1).
  - When J=0,K=1, the output is reset (Q = 0).

How is it Used in Counting?
If you always set J and K to 1, then every time the clock tells the flip-flop to update, it will toggle (switch) its state. If it was 0, it would become 1. If it was 1, it would become 0. This behavior is useful for building counters.

Asynchronous Counters (Ripple Counters) Using J-K Flip-Flops
When you connect several J-K flip-flops one after another, you can make a counter. A counter is a device that goes through a sequence of numbers with each clock pulse. In an **asynchronous** counter (often called a ripple counter):

- The first flip-flop receives the main clock signal directly.
- Each following flip-flop uses the output of the previous flip-flop as its "clock."

Since each flip-flop toggles its value when triggered, the first flip-flop will flip its state on every clock pulse. The second flip-flop will flip only when the first one changes from 1 to 0 (because it sees that as a clock pulse), and so on. This creates a pattern that counts up in binary.

Key Points:

1. **Design**:
   - The output of one flip-flop acts as the clock input for the next.
   - This cascading connection creates a ripple effect.
2. **How It Works**:
   - Each flip-flop toggles its state on the **negative edge** (falling edge - from 1 to 0) of the clock.
   - For N flip-flops, the counter can count 2^N states (e.g., 4 flip-flops = 16 states).
3. **Frequency Division**:
   - Each flip-flop divides the clock frequency by 2.
   - For example:
     - Flip-flop 1: f/2
     - Flip-flop 2: f/4

        - Flip-flop 3: f/8, and so on.

Example: A 4-Bit Counter
By connecting 4 J-K flip-flops in this "chain," each with J and K set to 1:

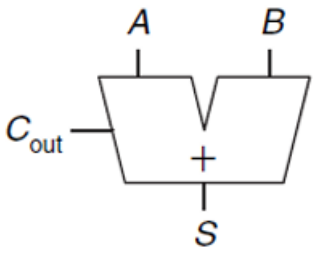
- The first flip-flop (called FF0) changes its output from 0 to 1 or 1 to 0 on every clock pulse.
- The second flip-flop (FF1) toggles its output whenever the first flip-flop goes from 1 to 0. This effectively makes it change every 2 clock pulses, so its frequency of toggling is half of the first one.
- The third flip-flop (FF2) toggles its output whenever the second one goes from 1 to 0, meaning it changes every 4 clock pulses.
- The fourth flip-flop (FF3) toggles its output whenever the third one goes from 1 to 0, meaning it changes every 8 clock pulses.

Together, these four outputs (Q0, Q1, Q2, Q3) form a 4-bit binary number that counts upward from 0000 (0) to 1111 (15) as the clock pulses come in. After reaching 1111, it goes back to 0000 and starts over, making a cycle of 16 steps. This is why it can be called a "divide-by-16" counter: the output completes one full cycle for every 16 clock pulse.

Lecture 4

Half Adder

What is a Half Adder?



A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$
$$C_{out} = AB$$

- A half adder is a basic circuit used to add two binary digits (A and B).
- It has two outputs:
    - **S (Sum):** The result of adding A and B.
    - **Cout (Carry Out):** The carry bit if the result is more than 1.

How It Works:

- Binary addition follows these rules:

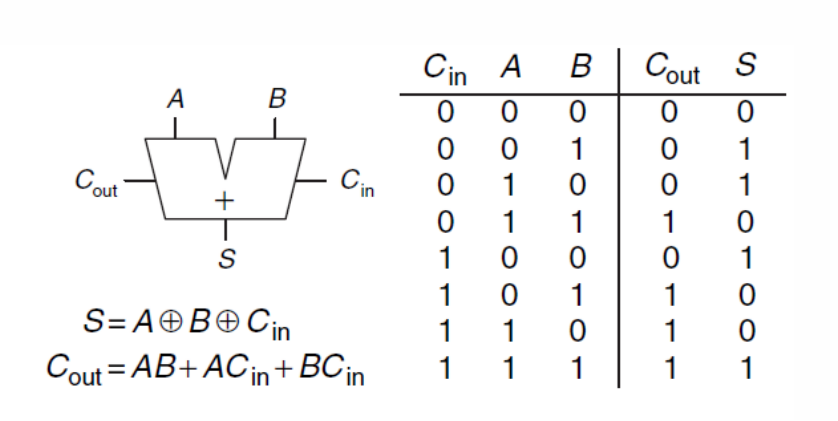
        - 0 + 0 = 0 (no sum, no carry).
        - 0 + 1 = 1 (sum is 1, no carry).
        - 1 + 0 = 1 (sum is 1, no carry).
        - 1 + 1 = 0 (sum is 0, carry is 1).
    - The **Sum (S)** is calculated using an XOR gate: S = A XOR B.
    - The **Carry Out (Cout)** is calculated using an AND gate: Cout = A AND B.

Limitations of Half Adder:

- The half adder cannot handle a carry from a previous addition because it does not have a carry-in input (Cin).

Full Adder

What is a Full Adder?



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- A full adder improves on the half adder by allowing an additional input called **Cin (Carry In)**.
- It adds three inputs: A, B, and Cin.
- It produces two outputs:
    - **S (Sum):** The result of the addition.
    - **Cout (Carry Out):** The carry bit for the next stage.

How It Works:

- Binary addition with three inputs (A, B, Cin) follows these rules:
    - If A, B, and Cin are all 0, then S = 0, Cout = 0.
    - If one of them is 1, then S = 1, Cout = 0.
    - If two of them are 1, then S = 0, Cout = 1.
    - If all three are 1, then S = 1, Cout = 1.

- The **Sum (S)** is calculated as:
  S = A XOR B XOR Cin
- The **Carry Out (Cout)** is calculated as:
  Cout = (A AND B) OR (A AND Cin) OR (B AND Cin)

Why Full Adder is Better:

- It includes the carry-in (Cin), which makes it possible to handle carries from previous stages in multi-bit addition.

Carry Propagate Adder

What is a Carry Propagate Adder (CPA)?

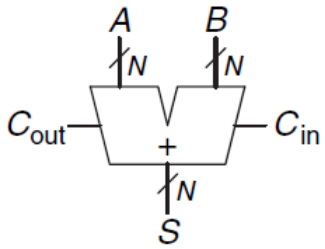


Figure 5.4 Carry propagate adder

- A CPA is used to add two multi-bit binary numbers (A and B) along with a carry-in (Cin).
- It produces:
  - **S (Sum):** The multi-bit result.
  - **Cout (Carry Out):** The final carry out.

How It Works:

- Each bit of A and B is added using a **full adder**.
- The carry out (Cout) from each full adder is passed as the carry in (Cin) to the next full adder in the chain.
- For example:
  - Bit 0: Add A0, B0, and Cin (carry in is 0 for the first bit).
  - Bit 1: Add A1, B1, and the Cout from Bit 0.
  - Bit 2: Add A2, B2, and the Cout from Bit 1.
  - This process continues for all bits.

Why is it Called Carry Propagate?

- The carry generated by one bit propagates to the next bit, and so on.

Ripple-Carry Adder

What is a Ripple-Carry Adder?

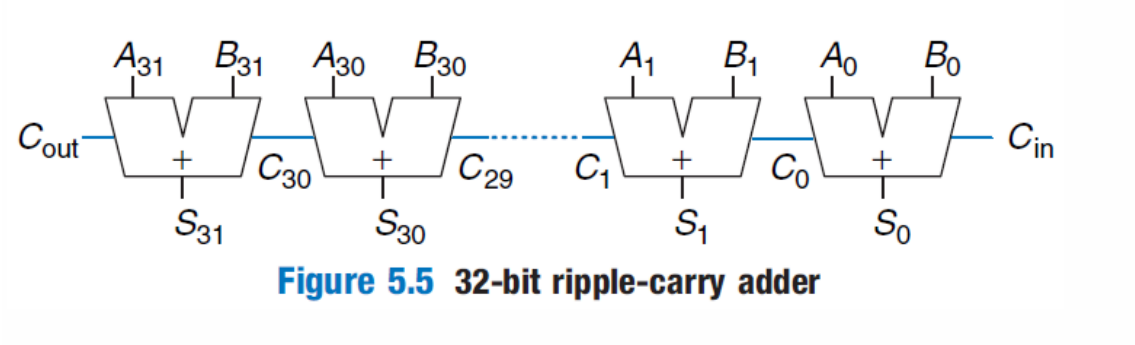


Figure 5.5 32-bit ripple-carry adder

- It is the simplest way to build a multi-bit carry propagate adder.
- It chains multiple full adders together, where:
  - The **Cout** of one full adder becomes the **Cin** of the next full adder.
- Example: A 32-bit ripple-carry adder adds two 32-bit binary numbers by connecting 32 full adders in sequence.

How It Works:

- Start with the least significant bit (LSB):
  - Add A0, B0, and Cin (Cin is 0 for the first adder).
  - Produce Sum S0 and Cout0.
- Move to the next bit:
  - Add A1, B1, and Cin = Cout0.
  - Produce Sum S1 and Cout1.
- Repeat this process for all bits until the most significant bit (MSB).

Disadvantage:

- The carry must ripple through each full adder in the chain.
- This makes it **slow for large numbers** because the addition cannot finish until the carry reaches the last bit.
- The time delay increases with the number of bits, calculated as:
  Total Delay = Number of Bits × Delay of One Full Adder.

Carry-Lookahead Adder

What is a Carry-Lookahead Adder (CLA)?

- A CLA improves the speed of addition by calculating carries more quickly.
- Instead of waiting for the carry to ripple through all bits, it "looks ahead" and calculates all carries in advance.

How It Works:

- The adder is divided into smaller blocks (e.g., 4-bit blocks in a 32-bit adder).
- Additional circuitry predicts the carry for each block as soon as the carry-in (Cin) is known.
- This allows the sum for all blocks to be calculated simultaneously, rather than waiting for the carry to propagate.

Advantages:

- Faster than ripple-carry adders.
- The delay is reduced significantly because carry propagation is eliminated within blocks.

Subtractor

What is a Subtractor?

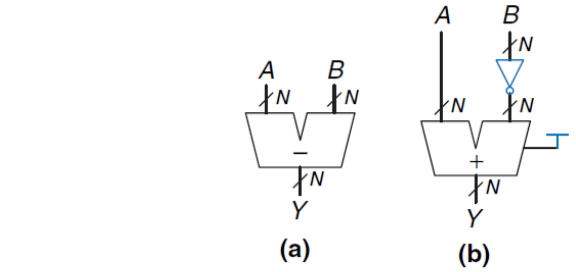


Figure 5.9 Subtractor: (a) symbol, (b) implementation

- A subtractor is a circuit that calculates the subtraction of two binary numbers, A and B.
- Subtraction is done using **two's complement**, which converts subtraction into addition.

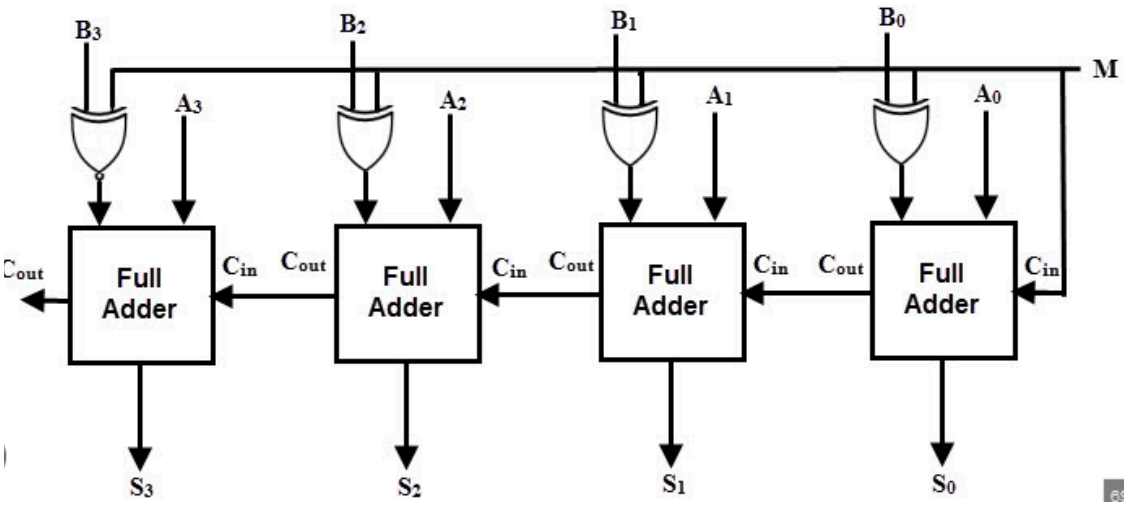
How It Works:

- To calculate Y = A - B:
  1. **Invert** all bits of B (flip 1s to 0s and 0s to 1s).
  2. **Add 1** to the inverted B to get its two's complement.

3. Add A + (-B) using a normal adder circuit.
4. Set the carry-in (Cin) to 1 in the adder to include the +1.

Components of a Subtractor:

- Inverters are used to flip the bits of B.
- A carry propagate adder (CPA) is used to perform the addition.



Comparators

What is a Comparator?

- A comparator is a circuit that compares two binary numbers, A and B.
- It determines whether:
  - A is equal to B (A == B)
  - A is greater than B (A > B)
  - A is less than B (A < B)

Types of Comparators:

a. Equality Comparator :

- This checks if two binary numbers, A and B, are equal.
- **How It Works:**
  - For each bit of A and B, compare them using an XOR gate.
  - XNOR outputs 1 if the bits are the same, and 0 if they are different.
  - Use an AND gate to combine all XOR outputs:
    - If all XNOR outputs are 1, A and B are equal, and the comparator outputs 1 (True).

        - If any XNOR output is 0,, A and B are not equal, and the comparator outputs 0 (False).

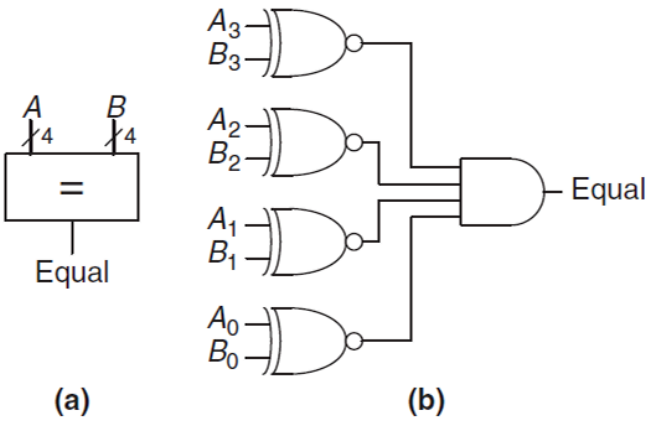


Figure 5.11 4-bit equality comparator: (a) symbol, (b) implementation

- **b. Magnitude Comparator :**
    - This checks whether A is less than B (A < B).
    - **How It Works:**
        - Perform the subtraction A - B using a subtractor circuit.
        - Look at the **sign bit** (most significant bit) of the result:
            - If the sign bit is 1, then A < B.
            - If the sign bit is 0, then A ≥ B.

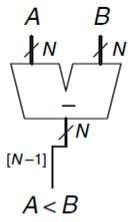


Figure 5.12 *N*-bit magnitude comparator

Arithmetic Logic Unit (ALU)

- **What is an ALU?**
    - The ALU (Arithmetic Logic Unit) is the part of a computer that performs mathematical (arithmetic) and logical operations.

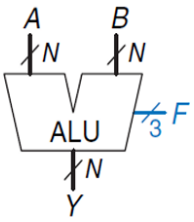


Figure 5.14 ALU symbol

- **What Operations Does It Perform?**
  - The ALU can perform:
    - Logical operations like AND, OR
    - Arithmetic operations like addition, subtraction
    - Special operations like SLT (Set Less Than)
- **How Does It Work?**
  - The ALU takes two binary inputs, A and B, and a control signal F that tells it which operation to perform.
  - The outputs are:
    - A result (Y), which depends on the operation.
    - Optional "flags" like Zero or Overflow to give additional information about the result.
- **Control Signals :**
  - The control signal F2:0 determines the operation:
    - 000: A AND B
    - 001: A OR B
    - 010: A + B (Addition)
    - 110: A - B (Subtraction)
    - 111: SLT (Set Less Than)
- **Set Less Than (SLT) Operation :**
  - SLT determines if A is less than B.
  - **How It Works:**
    - Subtract B from A: S = A - B
    - Look at the **sign bit** of the result S:
      - If the sign bit is 1, A < B, and the ALU sets Y = 1.
      - Otherwise, Y = 0.
- **ALU Components :**
  - **Adder/Subtractor Circuit:**
    - Adds or subtracts A and B depending on the control signal F2.
  - **Logic Gates (AND/OR):**
    - Perform logical operations like AND or OR.
  - **Multiplexer (MUX):**
    - Chooses the output based on the control signal F1:0.
- **How Subtraction Works in the ALU:**
  - Subtraction is done by adding A + (-B):

        - Flip the bits of B using inverters.
        - Set F2 = 1 to add the inverted B with a carry-in of 1 (to account for the +1 in two's complement).
- **Extra ALU Features:**
    - Some ALUs provide additional "flags":
        - **Overflow Flag:** Indicates if the result exceeds the allowed range.
        - **Zero Flag:** Indicates if the result is 0.

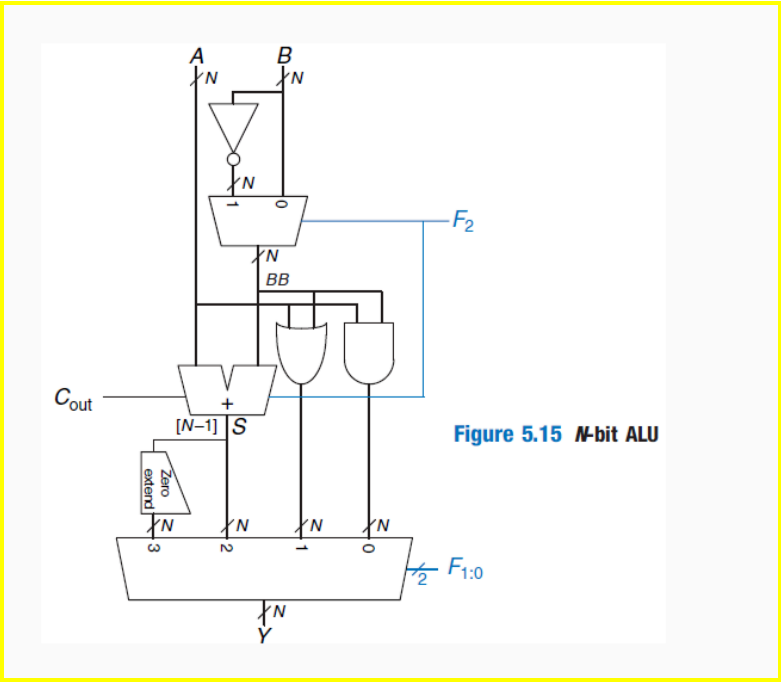


Figure 5.15 *N*-bit ALU

Shifters and Rotators

What are Shifters and Rotators?

- Shifters move binary numbers left or right by a specific number of positions.
- They are useful for **multiplying or dividing** numbers by powers of 2.
- Rotators are similar, but instead of losing bits when shifting, they wrap bits around to the other side.

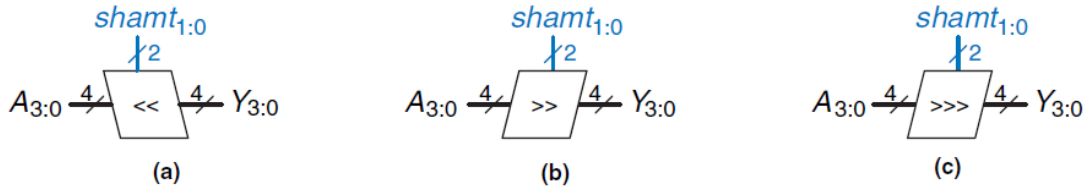
Types of Shifters

1. **Logical Shifter:**
    - Shifts bits left or right and **fills the empty spaces with 0s**.
    - **Example:**
        - 11001 Logical Shift Right (LSR) by 2 → 00110

        - 11001 Logical Shift Left (LSL) by 2 → 00100
2. **Arithmetic Shifter:**
    - Similar to a logical shifter, but **preserves the sign bit** during right shifts.
    - When shifting right, the **most significant bit (MSB)** is copied into the empty spaces.
    - This is useful for signed numbers (positive and negative).
    - **Example:**
        - 11001 Arithmetic Shift Right (ASR) by 2 → 11110
        - 11001 Arithmetic Shift Left (ASL) by 2 → 00100
    - **Note:** Arithmetic Shift Left (ASL) is the same as Logical Shift Left (LSL).
3. **Rotator:**
    - Moves bits in a circular fashion.
    - The bits shifted out on one end are brought back in on the other end.
    - **Example:**
        - 11001 Rotate Right (ROR) by 2 → 01110
        - 11001 Rotate Left (ROL) by 2 → 00111

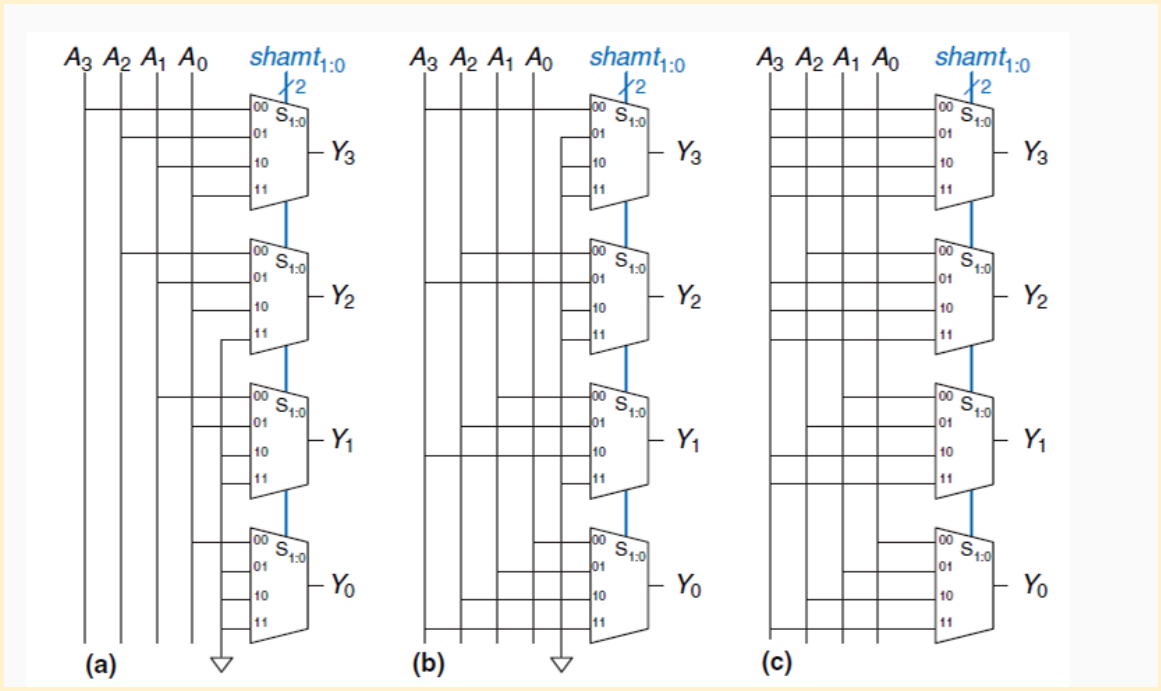
Hardware Implementation of Shifters

1. **Shifter Operators:**
    - << → Shift Left
    - >> → Logical Shift Right
    - >>> → Arithmetic Shift Right



2. **Shifting Logic:**
    - Depending on the shift amount (shamt), the shifter shifts the input number (A) by 0 to N-1 bits.
    - For instance, in a **4-bit shifter** (Figures 5.16):
        - shamt1:0 = 00 means no shift, so output Y = input A.
        - shamt1:0 = 01 shifts A by 1 bit.
        - shamt1:0 = 10 shifts A by 2 bits.
        - shamt1:0 = 11 shifts A by 3 bits.
3. **How Shifters Are Built:**
    - A shifter is made using multiplexers.
    - For an N-bit shifter:

- Each bit of the input is passed through a multiplexer.
- The multiplexer decides whether to shift the bit or leave it unchanged, based on the value of the shift amount.



Examples of Shifters

1. **Left Shift:**
   - Left shifting a binary number multiplies it by a power of 2.
   - **Example:**
     - 000011 Left Shift (<<) by 4 → 110000
     - This is equivalent to multiplying 3 (decimal) by 2^4 = 16, resulting in 48.
2. **Arithmetic Right Shift:**
   - Right shifting a signed binary number divides it by a power of 2.
   - **Example:**
     - 11100 Arithmetic Right Shift (>>>) by 2 → 11111
     - This is equivalent to dividing -4 (decimal) by2^2 = 4, resulting in -1.

Operation	Binary	Decimal	Calculation
Left Shift	`000011 << 4` → `110000`	3 → 48	$3 \times 2^4 = 48$
Arithmetic Right Shift	`11100 >>> 2` → `11111`	-4 → -1	$-4 \div 2^2 = -1$

Multiplication in Binary and Decimal

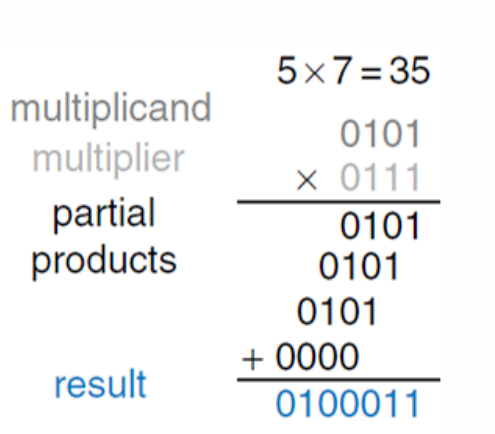
Binary Multiplication Example:

Multiply 5 × 7:

- Represent 5 as 0101 and 7 as 0111 in binary.
- Multiply each bit of the multiplier (7) with the multiplicand (5), shifting left for each step:
  - Step 1: Multiply 0101 by 1 (least significant bit of 7). Result = 0101.
  - Step 2: Multiply 0101 by 1 (next bit), shifted left. Result = 01010.
  - Step 3: Multiply 0101 by 1 (next bit), shifted left. Result = 010100.
  - Step 4: Multiply 0101 by 0 (most significant bit). Result = 000000.

Add the partial products: 0101 +01010 +010100 +000000 = 0100011 (binary result)

- Convert `0100011` to decimal: The result is 35.



Key Concepts:

1. Binary multiplication only involves **1s and 0s**.
2. If a multiplier bit is **1**, the multiplicand is added to the partial product.
3. If a multiplier bit is **0**, the partial product is all zeros.
4. The **AND operation** is used for binary multiplication:
   - 1 AND 1 = 1 → Same as multiplying 1 × 1.
   - 1 AND 0 = 0, 0 AND 1 = 0, 0 AND 0 = 0.

N × N Binary Multiplication

What is an N × N Multiplier?

- Multiplies two N-bit binary numbers to produce a **2N-bit result**.

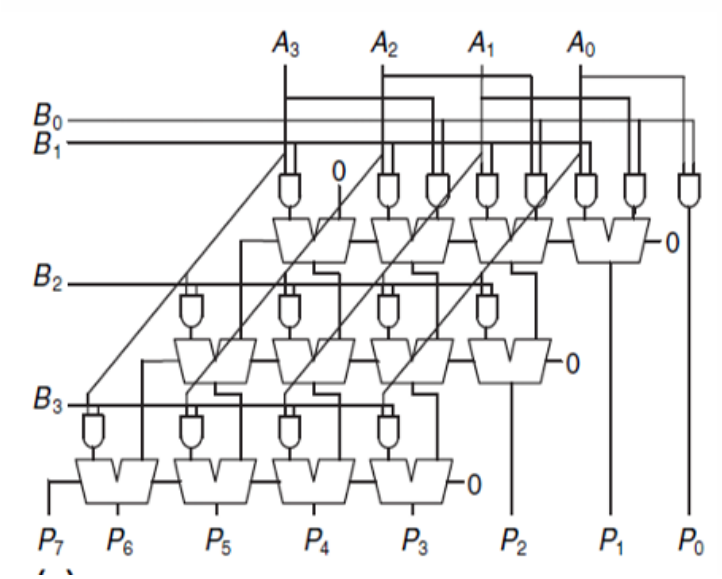
- For example, a 4 × 4 multiplier multiplies two 4-bit numbers and outputs an 8-bit result.

Steps for Binary Multiplication:

- **Generate Partial Products:**
  - Each bit of the multiplier is ANDed with all bits of the multiplicand to form partial products.
  - Example for a 4 × 4 multiplier:
    - First partial product: B0 AND (A3, A2, A1, A0).
    - Second partial product: B1 AND (A3, A2, A1, A0), shifted left by 1.
    - This continues for all bits of the multiplier (B2, B3).
- **Add the Partial Products:**
  - Use binary addition to sum the shifted partial products.

Example for a 4 × 4 Multiplier:

1. Multiply A = 1011 (11 in decimal) and B = 1101 (13 in decimal):
   - Partial Product 1: B0 AND (A3, A2, A1, A0) = 1011.
   - Partial Product 2: B1 AND (A3, A2, A1, A0), shifted left = 10110.
   - Partial Product 3: B2 AND (A3, A2, A1, A0), shifted left = 101100.
   - Partial Product 4: B3 AND (A3, A2, A1, A0), shifted left =1011000.
   - Add all partial products: 1011 + +10110 +101100 +1011000 = 10001111 (binary result, which is 143 in decimal).



Memory Arrays

- Digital systems use memories to **store data**.
- **Registers** (made from flip-flops) are small memory units for small amounts of data.
- **Memory arrays** are designed to store **large amounts** of data efficiently.

Memory Structure

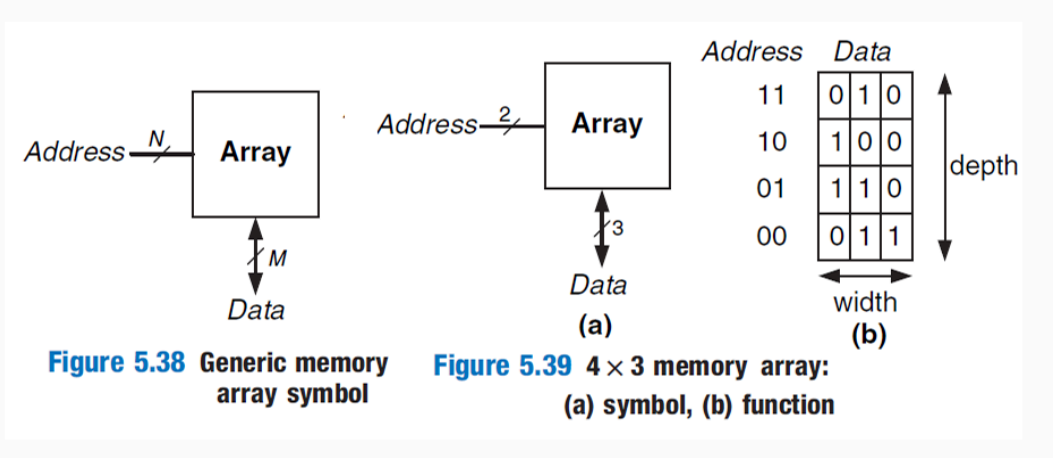


Figure 5.38 Generic memory array symbol

Figure 5.39 4 × 3 memory array: (a) symbol, (b) function

- **Memory is a 2D array** of memory cells.
- Rows in the array represent **data words**, and columns represent **bits in each word**.
- Example: If an array has **N-bit addresses** and **M-bit data**, it will have:
  - **2^N rows** (number of words).
  - **M columns** (word size).
  - **Total size = depth × width**.

Example Array:

- A (4 × 3) memory has:
  - 4 rows (words).
  - 3 columns (bits in each word).

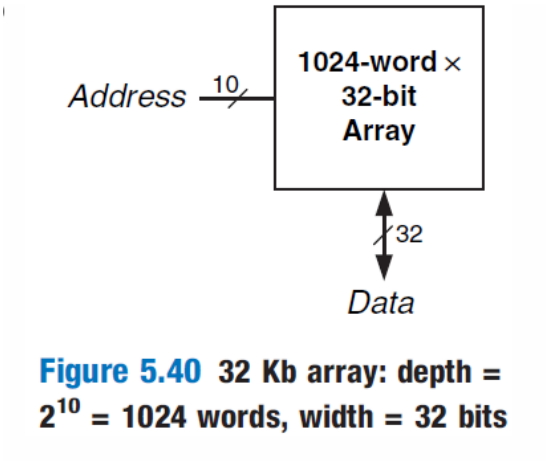


Figure 5.40 32 Kb array: depth = 2^{10} = 1024 words, width = 32 bits

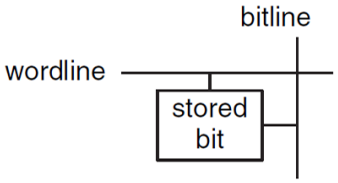
---

Memory Operations

1. **Read**:
   - Selects one row (word) using the **address**.
   - Data from the selected row appears on the output.
2. **Write**:
   - Data to be written is set on the bitlines.
   - The corresponding wordline (row) is activated to store the new data.

Example:

- To **read address 10**:
  - Address decoder activates the wordline for row 10.
  - Row data (e.g., 100) is sent to the output.
- To **write value 001 to address 11**:
  - Bitlines are set to 001.
  - Wordline for row 11 is activated to save the data.



- **Figure 5.41** Bit cell

Memory Array Organization

- Memory cells are organized as:

    - **Wordlines** (rows): Activate a specific row.
    - **Bitlines** (columns): Carry data bits to/from the bit cells.
- When a **wordline is HIGH**:
    - Data flows between bit cells and bitlines.

Memory Ports

- **Ports** allow read or write access to the memory.
- **Single-Port Memory**:
    - Only one address can be accessed at a time.
- **Multi-Port Memory**:
    - Can access multiple addresses at the same time.
    - Example: A memory with 3 ports:
        - **2 read ports**: Read data from 2 different addresses.
        - **1 write port**: Writes data to a specific address.

Memory Types

1. **What is Memory in Digital Systems?**
    - Memory is used to store data as an array of **bit cells**.
    - It is classified based on how the bits are stored and accessed.
2. **Types of Memory:**
    - **RAM (Random Access Memory):**
        - **Volatile**: Loses data when power is turned off.
        - Data is accessed with the same delay for any location.
    - **ROM (Read Only Memory):**
        - **Nonvolatile**: Retains data even without power.
        - Historically, it could only be read, not written.
        - ROM also allows random access but is different from RAM because of its permanence.
3. **Major RAM Types:**
    - **Dynamic RAM (DRAM):** Stores data as a charge on a capacitor.
    - **Static RAM (SRAM):** Stores data using a pair of inverters (flip-flops).

Dynamic Random Access Memory (DRAM)

- **How DRAM Works :**
    - DRAM stores a single bit of data as a charge on a **capacitor**.
    - It uses a single **nMOS transistor** to act as a switch:

        - **Wordline**: Turns the transistor ON or OFF.
        - **Bitline**: Transfers data to or from the capacitor.

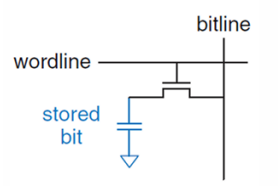
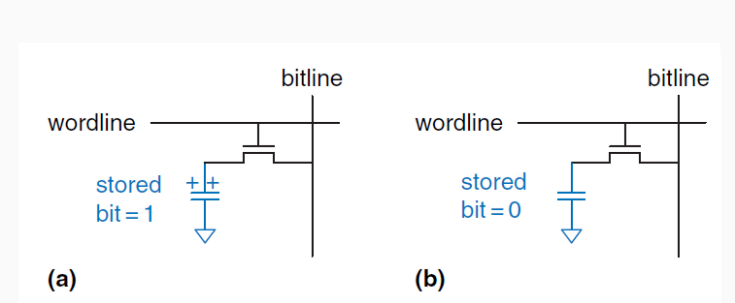


Figure 5.44 DRAM bit cell

        -
- **Storing Data in DRAM :**
    - **Bit = 1**
        - The capacitor is **charged** (voltage = VDD).
    - **Bit = 0 :**
        - The capacitor is **discharged** (voltage = GND).



- **Dynamic Nature of DRAM:**
    - The capacitor does not actively maintain its state.
    - Over time, the charge on the capacitor leaks away.
    - To maintain data, the capacitor must be **refreshed** periodically.

How DRAM Reads and Writes Data

Reading Data:

- When the **wordline** is activated, the transistor turns ON.
- The stored charge on the capacitor is transferred to the **bitline**.
- **Key Point:** Reading destroys the data stored in the capacitor.

Writing Data:

- The bitline transfers data (1 or 0) to the capacitor.
- If the bitline is high (VDD), the capacitor charges (bit = 1).
- If the bitline is low (GND), the capacitor discharges (bit = 0).

Challenges with DRAM

1. **Data Destruction During Read:**
    - Reading a DRAM cell removes the charge from the capacitor.
    - After every read, the original data must be **restored** (rewritten).
2. **Data Leakage:**
    - Even when not being read, the charge on the capacitor slowly leaks away.
    - To prevent data loss, DRAM requires periodic **refresh cycles**:
        - The data is read and rewritten every few milliseconds.

Static Random Access Memory (SRAM)

What is SRAM?

- SRAM is a type of **volatile memory** (it loses data when power is off).
- It is called "static" because the stored data does **not need refreshing**, unlike DRAM.

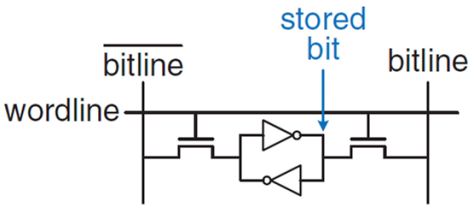


Figure 5.46 SRAM bit cell

How SRAM Stores Data :

- **Storage Mechanism:**
    - Each bit of data is stored using **cross-coupled inverters** (like in latches).
    - These inverters maintain the bit value (0 or 1) as long as power is supplied.
- **Reading and Writing Data:**
    - **Wordline:** Activates the SRAM cell.
    - When the wordline is asserted:
        - The two **nMOS transistors** turn ON.
        - Data can flow to or from the bitlines.
    - The two outputs of the SRAM cell are:
        - **Bitline** (carries data).
        - **Complemented Bitline** (carries the opposite value).
- **Error Correction:**

    - If noise degrades the stored value, the cross-coupled inverters automatically restore it to the correct value.

Comparison of Memory Types

- **Flip-Flop:**
    - **Transistors per Bit Cell:** ~20 (largest area).
    - **Speed:** Fastest.
    - **Notes:**
        - Flip-flops provide immediate data output but require more area, power, and cost due to the high number of transistors.
- **SRAM:**
    - **Transistors per Bit Cell:** 6.
    - **Speed:** Medium.
    - **Notes:**
        - SRAM offers a balance between speed and size.
        - Faster than DRAM but slower than flip-flops.
- **DRAM:**
    - **Transistors per Bit Cell:** 1 (smallest area).
    - **Speed:** Slowest.
    - **Notes:**
        - DRAM requires fewer transistors, making it compact and cost-efficient.
        - Slower due to its reliance on a capacitor for data storage and the need for periodic refreshing.

Why SRAM is Faster Than DRAM:

1. **Actively Driven Data:**
    - In SRAM, the stored bit is actively driven by the cross-coupled inverters.
    - In DRAM, data is stored as a charge on a capacitor, which takes longer to read and write.
2. **No Refresh Needed:**
    - SRAM does not need periodic refreshing, unlike DRAM.
3. **Throughput Advantage:**
    - SRAM can read or write data without delays caused by refresh cycles.

How DRAM Tries to Overcome Its Limitations

- **Synchronous DRAM (SDRAM):**
  - Uses a clock signal to **pipeline memory accesses**.
  - This allows simultaneous operations and reduces delays.
- **Double Data Rate (DDR) SDRAM:**
  - Accesses data on both the **rising** and **falling edges** of the clock signal, effectively doubling throughput.
  - **DDR Generations:**
    - **DDR:** Ran at 100–200 MHz.
    - **DDR2, DDR3, DDR4:** Increased clock speeds, reaching over 1 GHz by 2012.

Register Files

- **What Are Register Files?**
  - **Registers:** Small memory units in a CPU used to store temporary variables.
  - **Register File:** A group of registers in a CPU, used for efficient processing.
- **How They Work:**
  - Register files are typically built as **multiported SRAM arrays**:
    - **Why SRAM?**
      - More compact than flip-flops.
      - Provides faster access compared to DRAM.
  - **Use Case:**
    - Store temporary values during CPU operations (e.g., intermediate results of calculations).

Read-Only Memory (ROM)

What is ROM?

- ROM is a **nonvolatile memory** that retains its data even when the power is turned off.
- It stores a bit (0 or 1) as the **presence or absence of a transistor**.

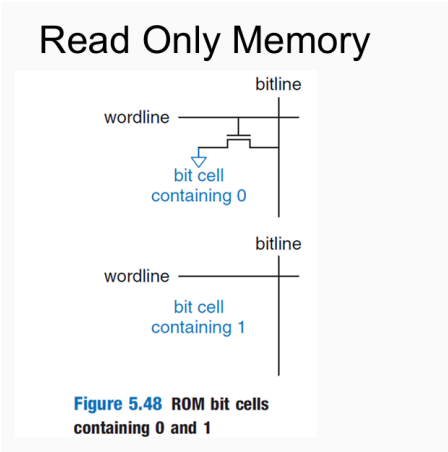
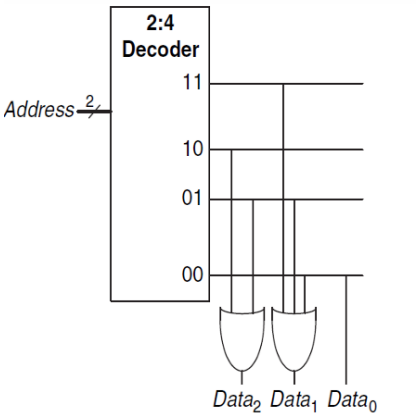


Figure 5.48 ROM bit cells containing 0 and 1

- ROM cells are **combinational circuits** (no state is maintained, so no data is "forgotten" when power is off).

ROM Representation and Dot Notation

- **Decoder Implementation:**
  - ROM can be built using a **decoder** and **OR gates**:
    - The decoder generates all possible address combinations (minterms).
    - Each address corresponds to a specific OR gate that outputs the stored data.



Programmable ROM (PROM)

- **What is PROM?**
  - A PROM has a transistor in every bit cell.
  - Connections to ground can be programmed (made or broken).

- **Reprogrammable PROMs:**
    - These allow you to reconnect or disconnect transistors reversibly.
- **Erasable PROMs (EPROMs):**
    - Use a **floating-gate transistor** to store data.
    - Data can be erased by exposing the chip to **ultraviolet (UV) light** for about 30 minutes.
- **Electrically Erasable PROMs (EEPROMs):**
    - Use electrical signals to erase and write data, eliminating the need for UV light.
    - **Key Feature:** Individual bits can be erased.

Flash Memory

- **What is Flash Memory?**
    - A type of **nonvolatile storage** that can be electrically erased and reprogrammed.
    - It is derived from EEPROM but is more cost-efficient.
- **Key Features:**
    - Flash memory erases data in **larger blocks**, unlike EEPROM, which can erase individual bits.
- **Types of Flash Memory:**
    - Named after their internal design:
        - **NAND Flash:** Resembles NAND gate characteristics.
        - **NOR Flash:** Resembles NOR gate characteristics.

Lecture 5

What are Instructions?

- A computer uses a specific "language" to operate.
- The words in this language are called **instructions**.

What is an Instruction Set?

- The **instruction set** is the computer's "vocabulary."
- All programs, whether simple or complex, are converted into this set of instructions. For example:
    - **Add**, **subtract**, **jump**, etc.
- An instruction tells the computer:
    - **What to do** (called the opcode).
    - **Which data to use** (called the operands).
    - Operands can come from:
        - Memory.
        - Registers.

        - The instruction itself.

Machine Language vs Assembly Language

Machine Language

- Computers understand only **binary numbers** (1's and 0's).
- Instructions are encoded in a format called **machine language**.
- Just as humans use letters for language, computers use binary numbers.

Microprocessors (CPUs)

- Microprocessors execute instructions written in **machine language**.
- Humans find machine language hard to read, so we use **assembly language** instead.

Assembly Language

- A human-readable version of machine language.
- Each assembly instruction:
    - Specifies the **operation** to perform (opcode).
    - Specifies the **data** to work on (operands).
- **Example:** Instead of binary, you write add to perform addition.

Instruction Sets and Architectures

Instruction Set Variety

- Different computer architectures have **different instruction sets**, like dialects of a language.
- However, all architectures include basic instructions like **add**, **subtract**, and **jump**.

Assembly Language Example

Addition Example

- Adding two variables, b and c, and storing the result in a.

Explanation:

- add: The operation (mnemonic/opcode).
- b and c: The **source operands** (data to be added).
- a: The **destination operand** (where the result is stored).

MIPS Architecture

MIPS (Microprocessor without Interlocked Pipeline Stage)

- Developed by John Hennessy at Stanford in the 1980s.
- Used in products by companies like Nintendo and Cisco.

RISC vs. CISC

- **RISC (Reduced Instruction Set Computer)**:
  - Focuses on simple, commonly used instructions.
  - Example: MIPS architecture.
  - Benefits:
    - Small and fast hardware.
    - Instructions are easier to decode.
  - Complex operations are done by combining multiple simple instructions.
- **CISC (Complex Instruction Set Computer)**:
  - Focuses on many complex instructions.
  - Example: Intel x86 architecture.
  - Downside:
    - Complex instructions make the hardware slower and larger, even if rarely used.

Instruction Encoding

- **RISC:**

    - Few instructions → fewer bits for encoding.
    - Example: 64 instructions need 6 bits (log2(64) = 6).
- **CISC:**
    - Many instructions → more bits for encoding.
    - Example: 256 instructions need 8 bits (log2(256) = 8).

Operands: Registers, Memory, and Constants

What are Operands?

- Operands are the data values an instruction operates on.
- Example: In a = b + c, the variables a, b, and c are all **operands**.

How Computers Handle Operands:

- Computers process data in the form of **1's and 0's**, not variable names.
- The binary data must come from specific physical locations like:
    1. **Registers** (fast storage inside the CPU).
    2. **Memory** (larger but slower storage).
    3. **Constants** (fixed values encoded directly in the instruction).

Speed vs. Storage

- **Registers and constants**:
    - Faster to access.
    - Can store only a small amount of data.
- **Memory**:
    - Slower to access.
    - Can store much larger amounts of data.

Example: MIPS-32

- Called a 32-bit architecture because it processes **32-bit data**.
- Some commercial versions now extend to **64 bits**.

Registers in MIPS

Why Use Registers?

- Accessing operands stored in **registers** is much faster than accessing them in memory.

- MIPS architecture provides **32 registers**, also called a **register set** or **register file**.

Key Design Principle:

- **Smaller is faster** – fewer registers mean faster access.

MIPS Register Naming

- **Registers** are named with a $ sign (e.g., $s0, $t0).
- Examples of register usage:
  - $s0: Holds variable a.
  - $s1: Holds variable b.
  - $s2: Holds variable c.

Example Code (Addition):

- **High-Level Code:** a = b + c
- **MIPS Assembly Code:**

  - This **adds** the values in $s1 (b) and $s2 (c) and **stores** the result in $s0 (a).

Saved and Temporary Registers

Saved Registers ($s)

- Used to store variables (e.g., a, b, c).
- These are preserved across function calls.

Temporary Registers ($t)

- Used for intermediate calculations.
- These are not preserved across function calls.

Example Code (Subtraction):

- **High-Level Code:** a = b + (c - d)
- **MIPS Assembly Code:**

```assembly
sub $t0, $s2, $s3  # $t0 = c - d
add $s0, $s1, $t0  # a = b + $t0
```

Register Details

MIPS Register Table Overview:

- **Registers for Variables:** $s0–$s7 (saved) and $t0–$t9 (temporary).
- **Special Registers:**
  - $0: Always holds the constant value **0**.
  - $gp: Global pointer.
  - $sp: Stack pointer (used in function calls).
  - $ra: Return address (stores the address to return to after a function call).

MIPS Memory Basics

- **32-bit Architecture**: MIPS uses **32-bit addresses** and **32-bit data words**.
- **Byte-Addressable Memory**:
  - Every byte in memory has a **unique address**.
  - A 32-bit word (4 bytes) is stored in **4 consecutive memory addresses**.
  - Example:
    - Word 0 starts at address 0 (00000000).
    - Word 1 starts at address 4 (00000004).

Word-Addressable vs. Byte-Addressable

1. **Word-Addressable Memory**:
   - Each 32-bit data word has its own unique address.
   - Example:
     - Word 0: Address 00000000.
     - Word 1: Address 00000001.

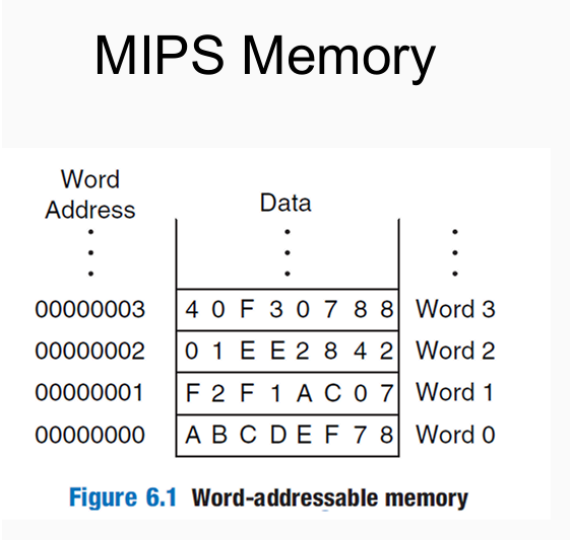


Figure 6.1 **Word-addressable memory**

- c

2. **Byte-Addressable Memory**:
   - Each **byte** has a unique address.
   - A word (4 bytes) is stored at consecutive addresses.
   - Example:
     - Word 0 (4 bytes): Addresses 00000000 to 00000003.
     - Word 1 (4 bytes): Addresses 00000004 to 00000007.

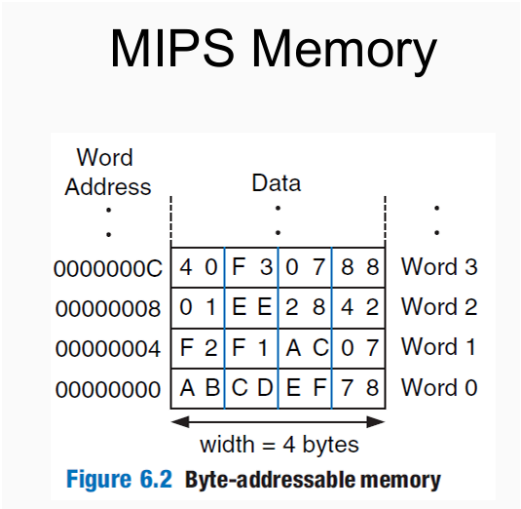


Figure 6.2 **Byte-addressable memory**

Loading and Storing Data (MIPS Instructions)

- **Load Word (lw)**: Reads a 32-bit word from memory into a register.

- Example:

```assembly
lw $s3, 4($0)
```

    - Reads the word starting at address 4 into register $s3.

Store Word (sw): Writes a 32-bit word from a register into memory.

- Example:

```assembly
sw $s3, 8($0)
```

    - Writes the value from $s3 into the word at address 8.

MIPS Memory and Endianness

MIPS memory follows **byte-addressable storage**, meaning each memory address holds **one byte (8 bits)**. However, computers store multi-byte data (like 32-bit words) in two different ways:

1. Big-Endian Storage (Used in Some Systems)

- **Stores the most significant byte (MSB) first** at the smallest memory address.
- The **byte order looks natural** (left to right) when reading memory.
- Example:
    - A **4-byte word** 23 45 67 89 is stored as:

```makefile
Address:   0   1   2   3
Data:     23  45  67  89
```

- **MIPS processors traditionally use big-endian format.**

2. Little-Endian Storage (Used in Intel Processors)

- **Stores the least significant byte (LSB) first** at the smallest memory address.
- The byte order is **reversed** compared to big-endian.
- Example:
    - The same **4-byte word** 23 45 67 89 is stored as:
    -

```
makefile                                   Copy  Edit

Address:   3   2   1   0
Data:     23  45  67  89
```

MIPS Immediate Operands (addi instruction)

What is an Immediate?

- An **immediate** is a **constant number** used directly in an instruction.
- It **does not require** fetching from memory or a register.

Example: MIPS addi Instruction

- addi stands for **"add immediate"**.
- It adds a constant **immediate value** to a register.

- Example:

```
assembly                                   Copy  Edit

addi $s0, $s0, 4   # a = a + 4
addi $s1, $s0, -12 # b = a - 12
```

    - $s0 = a
    - $s1 = b
    - First instruction adds 4 to $s0 (modifying a).
    - Second instruction subtracts 12 from $s0 and stores in $s1 (b).

Why Use Immediates?

- **Faster execution** (no need to load values from memory).
- **Reduces memory access**, making code more efficient.
- **Common in arithmetic and logical operations**.

MIPS Machine Language Explained Simply

MIPS has **three types of instructions**:

1. **R-type (Register-type)** → Uses three registers.
2. **I-type (Immediate-type)** → Uses two registers and a **16-bit immediate**.
3. **J-type (Jump-type)** → Uses a **26-bit address** for jumping.

R-Type Instructions (Register Instructions)

- **Uses three registers**:
  - **rs** (Source Register 1)
  - **rt** (Source Register 2)
  - **rd** (Destination Register)
- **Opcode is always 0**.
- **Operation is decided by the funct field**.

Instruction Format (Figure 6.5)

Field	Bits	Purpose
op (opcode)	6	Always `000000` for R-type
rs	5	First source register
rt	5	Second source register
rd	5	Destination register
shamt	5	Shift amount (used only for shift instructions)
funct	6	Specific operation code

Example: ADD Instruction (add $s0, $s1, $s2)

- **Meaning:** $s0 = $s1 + $s2
- **Machine code fields:**
  - rs = $s1 → Register 17
  - rt = $s2 → Register 18
  - rd = $s0 → Register 16
  - shamt = 0 (not a shift)
  - funct = 32 (add operation)

- **Machine Code:**

```
000000 10001 10010 10000 00000 100000
```

  - **(0x02328020 in hex)**

Example: SUB Instruction (sub $t0, $t3, $t5)

- **Meaning:** $t0 = $t3 - $t5
- **Machine code fields:**
  - rs = $t3 → Register 11
  - rt = $t5 → Register 13
  - rd = $t0 → Register 8
  - shamt = 0
  - funct = 34 (sub operation)

- **Machine Code:**

```
000000 01011 01101 01000 00000 100010
```

  - **(0x016D4022 in hex)**

Conditional Branching (beq instruction)

- **beq (Branch if Equal)** checks if two registers are equal.
- **If equal, it jumps to a target label** instead of executing the next instruction.

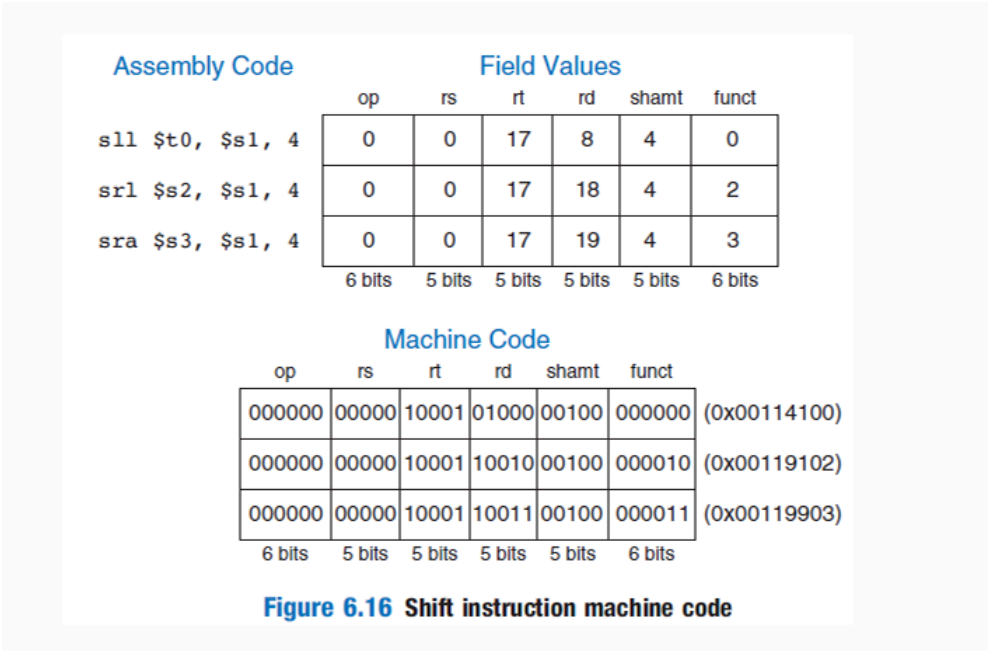
Example :

```assembly
addi $s0, $0, 4   # $s0 = 0 + 4 = 4
addi $s1, $0, 1   # $s1 = 0 + 1 = 1
sll  $s1, $s1, 2  # $s1 = 1 << 2 = 4
beq  $s0, $s1, target  # If $s0 == $s1, jump to target
addi $s1, $s1, 1  # (Not executed)
sub  $s1, $s1, $s0  # (Not executed)
target:
add  $s1, $s1, $s0  # $s1 = 4 + 4 = 8
```

- $s0 = 4, $s1 = 1
- $s1 is shifted left (1 << 2 → 4)
- **Branch Check**: $s0 == $s1 → **TRUE!**
- **Jumps to target, skipping the next two instructions.**
- target: executes → $s1 = 4 + 4 = 8

Shift Instructions

MIPS shift operations are sll (shift left logical), srl (shift right logical), and sra (shift right arithmetic).



Assembly Code / Field Values

Assembly Code	op	rs	rt	rd	shamt	funct
sll $t0, $s1, 4	0	0	17	8	4	0
srl $s2, $s1, 4	0	0	17	18	4	2
sra $s3, $s1, 4	0	0	17	19	4	3
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	00000	10001	01000	00100	000000	(0x00114100)
000000	00000	10001	10010	00100	000010	(0x00119102)
000000	00000	10001	10011	00100	000011	(0x00119903)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Figure 6.16 Shift instruction machine code

MIPS I-Type Instructions (Immediate Instructions)

MIPS uses **I-type (Immediate-type) instructions** when an **immediate value (constant)** is involved. These instructions **use two registers and a 16-bit immediate value**.

1. I-Type Instruction Format (Figure 6.8)

Field	Bits	Purpose
op (opcode)	6	Specifies the operation type
rs	5	Source register
rt	5	Target register (or destination)
imm	16	Immediate value (constant)

- **rs**: Always a source register.
- **imm**: Immediate value (a constant or memory offset).
- **rt**: Acts as:
  - A **destination register** (e.g., addi, lw).
  - A **source register** (e.g., sw).

Example 1: addi (Add Immediate)

Instruction:

assembly Copy Edit

```
addi $s0, $s1, 5  # $s0 = $s1 + 5
```

Field Values:

op	rs	rt	imm
8	17	16	5

Machine Code:

Copy Edit

```
001000 10001 10000 0000000000000101
```

- **Hexadecimal:** `0x22300005`

Example 2: addi with Negative Immediate

Instruction:

assembly Copy Edit

```
addi $t0, $s3, -12  # $t0 = $s3 - 12
```

Field Values:

op	rs	rt	imm
8	19	8	-12

Machine Code:

Copy Edit

```
001000 10011 01000 1111111111110100
```

- **Hexadecimal:** `0x2268FFF4`
- **Negative values use two's complement representation.**

Example 3: lw (Load Word)

Example 3: `lw` (Load Word)

Instruction:

assembly Copy Edit

```
lw $t2, 32($s0)  # Load from memory address ($s0 + 32) into $t2
```

Field Values:

op	rs	rt	imm
35	0	10	32

Machine Code:

Copy Edit

```
100011 00000 01010 0000000000100000
```

- **Hexadecimal:** `0x8C0A0020`
- **Adds 32 to the base address in `$s0` and loads from memory into `$t2`.**

Example 4: sw (Store Word)

Instruction:

assembly Copy Edit

```
sw $s1, 4($t1)  # Store $s1 into memory at address ($t1 + 4)
```

Field Values:

op	rs	rt	imm
43	9	17	4

Machine Code:

Copy Edit

```
101011 01001 10001 0000000000000100
```

- **Hexadecimal:** `0xAD310004`
- **Stores the value from** `$s1` **into memory at** `$t1 + 4`.

Two's Complement and Sign Extension

- **I-type immediates are 16-bit but used in 32-bit operations.**
- **Positive numbers:** Upper 16 bits are **filled with 0s**.
- **Negative numbers:** Upper 16 bits are **filled with 1s** (sign extension).
- **Example:** -24 in **16-bit two's complement** = 1111111111101000.

MIPS J-Type Instructions (Jump Instructions) - Simplified Explanation

MIPS uses **J-Type (Jump-Type) instructions only for jump operations**. These instructions are used to **jump to a specific memory address**.

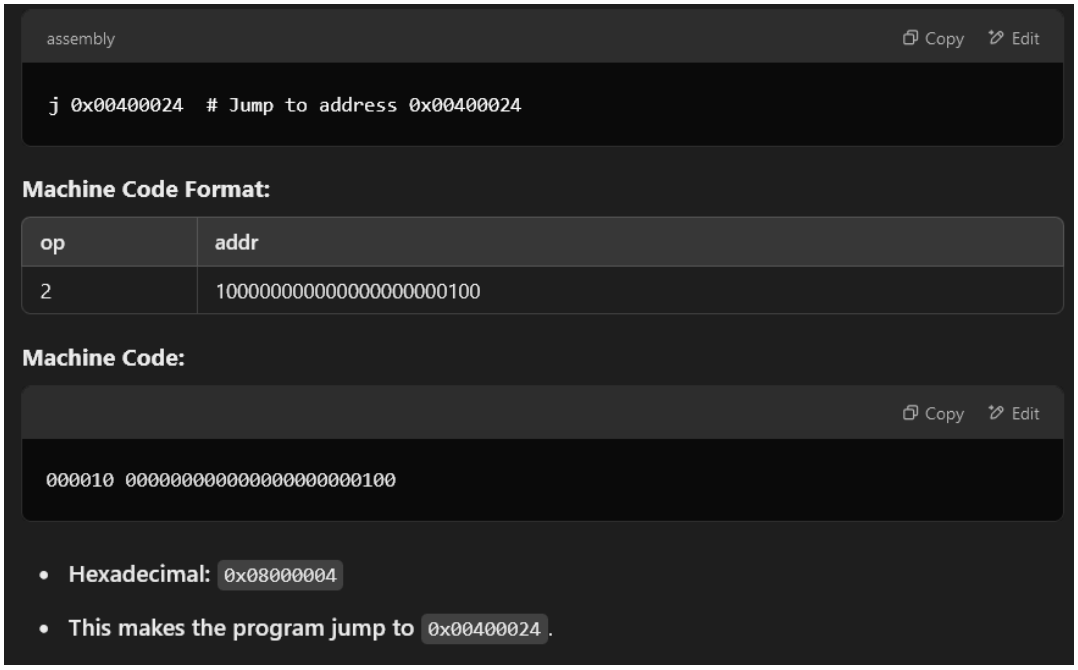
J-Type Instruction Format

Field	Bits	Purpose
op (opcode)	6	Specifies the operation type
addr	26	Jump address

- The addr field holds the target address (jump destination).
- The opcode tells what kind of jump it is (e.g., j or jal).

- Unlike R-type and I-type, J-type does not use registers.

Example 1: Jump Instruction (j target)



assembly Copy Edit

```
j 0x00400024  # Jump to address 0x00400024
```

Machine Code Format:

op	addr
2	10000000000000000000100

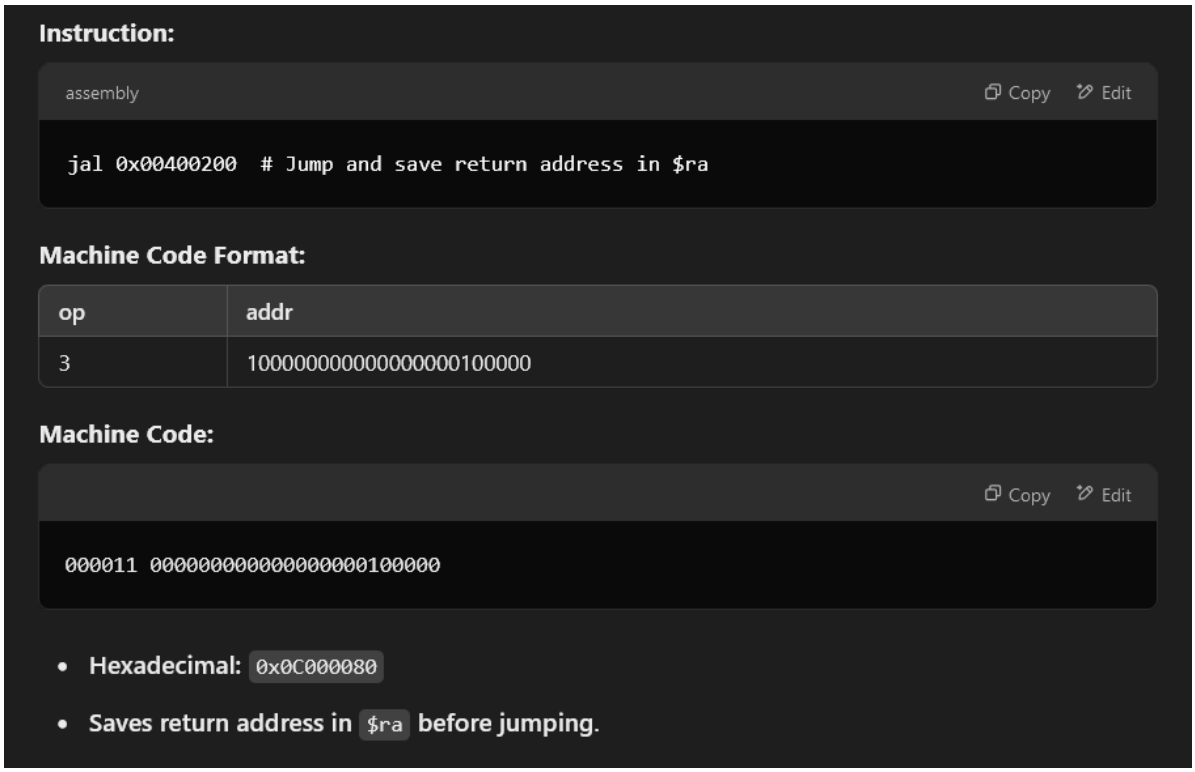
Machine Code:

Copy Edit

```
000010 00000000000000000000100
```

- **Hexadecimal:** `0x08000004`
- **This makes the program jump to** `0x00400024`.

Example 2: Jump and Link (jal target)



Instruction:

assembly Copy Edit

```
jal 0x00400200  # Jump and save return address in $ra
```

Machine Code Format:

op	addr
3	10000000000000000100000

Machine Code:

Copy Edit

```
000011 00000000000000000100000
```

- **Hexadecimal:** `0x0C000080`
- **Saves return address in** `$ra` **before jumping.**

Interpreting Machine Code

To **translate machine code into assembly**, we need to:

1. **Extract the opcode (first 6 bits).**
   - If **opcode = 0**, it is **R-type**.
   - If **opcode = 2 or 3**, it is **J-type**.
   - Otherwise, it is **I-type**.
2. **Decode the remaining bits**:
   - For **J-type**, the next **26 bits are the address**.

Example: Translating Machine Code

Machine Code 1 (0x2237FFF1)

Copy Edit

```
001000 10001 10111 1111111111110001
```

- **Opcode (`001000`) = 8 → `addi` instruction.**
- **Decoded Assembly:**

assembly Copy Edit

```
addi $s7, $s1, -15
```

Machine Code 2 (0x02F34022)

Copy Edit

```
000000 10111 10011 01000 00000 100010
```

- **Opcode (`000000`) = 0 → R-Type (`funct` determines operation).**
- **Funct (`100010`) = 34 → `sub` instruction.**
- **Decoded Assembly:**

assembly Copy Edit

```
sub $t0, $s7, $s3
```

The Power of the Stored Program

Computers work because of a **stored program concept**, which allows them to **store machine instructions in memory** and execute them sequentially. This **makes computers flexible and powerful**, as they can run different programs **without changing hardware**—just by **loading new instructions into memory**.

What is the Stored Program Concept?

- A **computer program is just a series of 32-bit machine instructions** stored in memory.
- These instructions **can be changed easily**, allowing a computer to run **many different programs**.
- **Before stored programs**, computers required **manual rewiring** to change their functions.
- **Now**, programs can be stored in memory, and the CPU can **fetch and execute them automatically**.

How the CPU Executes Instructions

Example :

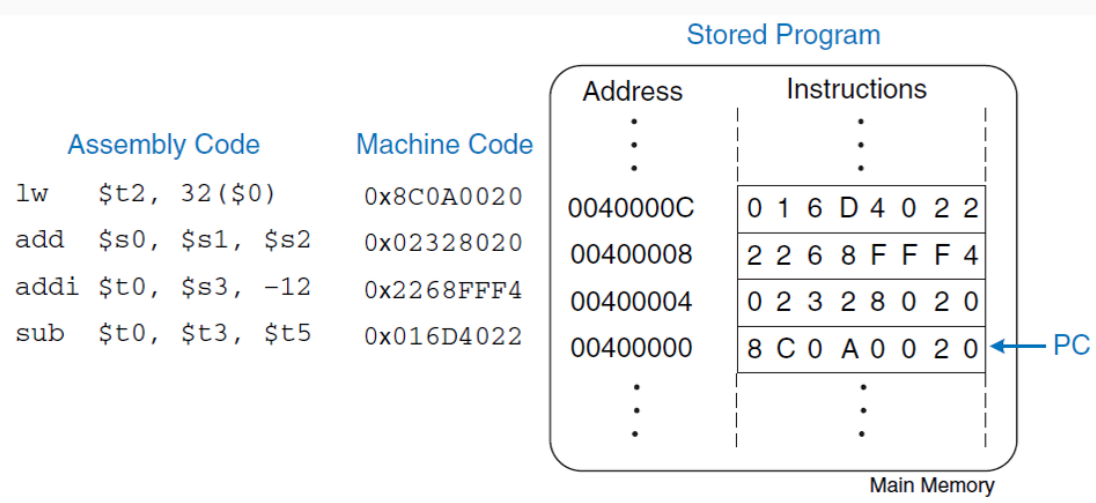


Figure 6.13 Stored program

- **The PC starts at 0x00400000 and moves forward by 4 each time.**
- **Each instruction is read, decoded, and executed one after another.**

Step-by-Step Execution:

- **The operating system sets the Program Counter (PC) to** 0x00400000 (starting address for MIPS programs).
- **The processor fetches the instruction at that address**, decodes it, and executes it.
- **After executing an instruction, the PC is increased by 4** (since MIPS instructions are **4 bytes each**).
- **This process repeats until the program ends**

The Role of the Program Counter (PC)

- The **Program Counter (PC)** is a special **32-bit register** that **stores the address of the next instruction** to execute.
- **After executing one instruction, the PC increases by 4** to fetch the next one.
- If a **jump or branch occurs**, the PC **changes to the target address** instead of simply increasing.

How Computers Handle Interruptions

- The **CPU state** consists of **registers** and the **PC**.
- If the **operating system saves this state**, it can:
  - **Pause the program**.
  - **Execute another task**.
  - **Restore the saved state** and **resume execution** without the program noticing.
- This allows **multitasking**, where a computer runs many processes at the same time.

High-Level vs. Low-Level Programming

- **High-Level Languages** (like C, Java) are **easier to read and write** but need to be **translated** into machine instructions.
- These languages include:
  - **Arithmetic & Logical Operations** (+, -, &&, ||).
  - **If-Else Conditions** (if, else).
  - **Loops** (for, while).
  - **Function Calls** (main(), printf()).
- **MIPS Assembly is a low-level language**, meaning it works **closer to machine instructions**.

Lecture 6

MIPS Branching

Computers can make decisions based on conditions, just like humans do. In MIPS assembly, we use **branching** to change the flow of execution depending on certain conditions. This allows us to create **if-else**, **loops**, and other control structures.

Conditional Branching: beq (Branch if Equal)

Code Example 6.12

- **Instruction Used:** beq $s0, $s1, target
- **What It Does:** If $s0 == $s1, jump to **target**. Otherwise, continue normally.

Step-by-Step Execution

Code Example 6.12 CONDITIONAL BRANCHING USING beq

MIPS Assembly Code

```
  addi  $s0, $0, 4          # $s0 = 0 + 4 = 4
  addi  $s1, $0, 1          # $s1 = 0 + 1 = 1
  sll   $s1, $s1, 2         # $s1 = 1 << 2 = 4
  beq   $s0, $s1, target    # $s0 == $s1, so branch is taken
  addi  $s1, $s1, 1         # not executed
  sub   $s1, $s1, $s0       # not executed

target:
  add   $s1, $s1, $s0       # $s1 = 4 + 4 = 8
```

1. addi $s0, $0, 4 → $s0 = 4
2. addi $s1, $0, 1 → $s1 = 1
3. sll $s1, $s1, 2 → $s1 = 1 << 2 = 4
4. beq $s0, $s1, target → Since $s0 == $s1, jump to **target**.
5. The next two instructions (addi and sub) are **not executed**.
6. At **target**, add $s1, $s1, $s0 → $s1 = 4 + 4 = 8

Key Takeaways

- beq checks if two registers hold the same value.
- If **true**, it skips instructions and jumps to the label.
- If **false**, execution continues normally.

Conditional Branching: bne (Branch if Not Equal)

Code Example 6.13

- **Instruction Used:** bne $s0, $s1, target
- **What It Does:** If $s0 != $s1, jump to **target**. Otherwise, continue normally.

Step-by-Step Execution

Code Example 6.13 CONDITIONAL BRANCHING USING bne

MIPS Assembly Code

```
  addi  $s0, $0, 4          # $s0 = 0 + 4 = 4
  addi  $s1, $0, 1          # $s1 = 0 + 1 = 1
  sll   $s1, $s1, 2         # $s1 = 1 << 2 = 4
  bne   $s0, $s1, target    # $s0 == $s1, so branch is not taken
  addi  $s1, $s1, 1         # $s1 = 4 + 1 = 5
  sub   $s1, $s1, $s0       # $s1 = 5 – 4 = 1

target:
  add   $s1, $s1, $s0       # $s1 = 1 + 4 = 5
```

1. addi $s0, $0, 4 → $s0 = 4
2. addi $s1, $0, 1 → $s1 = 1
3. sll $s1, $s1, 2 → $s1 = 1 << 2 = 4
4. bne $s0, $s1, target → Since $s0 == $s1, **branch is not taken**.
5. addi $s1, $s1, 1 → $s1 = 4 + 1 = 5
6. sub $s1, $s1, $s0 → $s1 = 5 - 4 = 1
7. At **target**, add $s1, $s1, $s0 → $s1 = 1 + 4 = 5

Key Takeaways

- bne checks if two registers hold **different** values.
- If **true**, it jumps to the label.
- If **false**, it executes the next instructions.

Unconditional Jump: j (Jump to Label)

Code Example 6.14

- **Instruction Used:** j target
- **What It Does:** Always jumps to **target**, skipping any instructions in between.

Step-by-Step Execution

Code Example 6.14 UNCONDITIONAL BRANCHING USING j

MIPS Assembly Code

```
  addi  $s0, $0, 4      # $s0 = 4
  addi  $s1, $0, 1      # $s1 = 1
  j     target          # jump to target
  addi  $s1, $s1, 1     # not executed
  sub   $s1, $s1, $s0   # not executed
target:
  add   $s1, $s1, $s0   # $s1 = 1 + 4 = 5
```

1. addi $s0, $0, 4 → $s0 = 4
2. addi $s1, $0, 1 → $s1 = 1
3. j target → Jump to **target**, skipping the next two instructions.
4. (Skipped) addi $s1, $s1, 1
5. (Skipped) sub $s1, $s1, $s0
6. At **target**, add $s1, $s1, $s0 → $s1 = 1 + 4 = 5

Key Takeaways

- j **always** jumps to the given label.
- All instructions between the j and the label **are skipped**.

Unconditional Jump: jr (Jump Register)

Code Example 6.15

- **Instruction Used:** jr $s0
- **What It Does:** Jumps to the **address stored in a register**.

Step-by-Step Execution

Code Example 6.15 UNCONDITIONAL BRANCHING USING `jr`

MIPS Assembly Code

```
0x00002000  addi  $s0, $0, 0x2010  # $s0 = 0x2010
0x00002004  jr    $s0              # jump to 0x00002010
0x00002008  addi  $s1, $0, 1       # not executed
0x0000200c  sra   $s1, $s1, 2      # not executed
0x00002010  lw    $s3, 44($s1)     # executed after jr instruction
```

1. addi $s0, $0, 0x2010 → $s0 = 0x2010 (address of next instruction)
2. jr $s0 → Jump to **address 0x2010**.
3. (Skipped) addi $s1, $0, 1
4. (Skipped) sra $s1, $s1, 2
5. At **0x2010**, lw $s3, 44($s1) → Loads data at $s1 + 44 into $s3.

Key Takeaways

- jr allows flexible jumps using register values.
- Useful for **function returns** (like return in C).

If Statements

An if statement runs a block of code **only if** a condition is **true**.

Example (High-Level Code)

```
if (i == j)
   f = g + h;
f = f - i;
```

MIPS Assembly Code

```
bne $s3, $s4, L1 # if i != j, skip if block
add $s0, $s1, $s2 # if block: f = g + h
L1:
sub $s0, $s0, $s3 # f = f - i
```

How It Works

1. $s3 = i, $s4 = j
2. bne $s3, $s4, L1 → **If i != j, skip the if block.**
3. If i == j, add $s0, $s1, $s2 runs (f = g + h).
4. Finally, sub $s0, $s0, $s3 executes (f = f - i).

Key Idea

- **MIPS tests the opposite condition** (bne instead of beq).
- If bne is **true**, it **skips** the if block.

If-Else Statements

An if-else statement **chooses between two blocks of code**.

Example

Code Example 6.17 `if/else` STATEMENT

High-Level Code

```
if (i == j)
  f = g + h;

else
  f = f - i;
```

MIPS Assembly Code

```
# $s0 = f, $s1 = g, $s2 = h, $s3 = i, $s4 = j
  bne $s3, $s4, else   # if i != j, branch to else
  add $s0, $s1, $s2    # if block: f = g + h
  j   L2               # skip past the else block
else:
  sub $s0, $s0, $s3    # else block: f = f - i
L2:
```

How It Works

1. **Branching with bne** → If i != j, jump to else.
2. **If block (i == j)** runs f = g + h, then skips else.
3. **Else block** runs if i != j.

Switch/Case Statements

A switch statement **chooses one block from many cases**.

Example

switch/case Statements

High-Level Code

```
switch (amount) {
  case 20:   fee = 2; break;

  case 50:   fee = 3; break;

  case 100:  fee = 5; break;

  default: fee = 0;
}

// equivalent function using
   if/else statements
if      (amount == 20)  fee = 2;
else if (amount == 50)  fee = 3;
else if (amount == 100) fee = 5;
else                    fee = 0;
```

MIPS Assembly Code

```
# $s0 = amount, $s1 = fee
case20:
  addi $t0, $0, 20          # $t0 = 20
  bne  $s0, $t0, case50     # amount == 20? if not,
                            # skip to case50
  addi $s1, $0, 2           # if so, fee = 2
  j    done                 # and break out of case
case50:
  addi $t0, $0, 50          # $t0 = 50
  bne  $s0, $t0, case100    # amount == 50? if not,
                            # skip to case100
  addi $s1, $0, 3           # if so, fee = 3
  j    done                 # and break out of case
case100:
  addi $t0, $0, 100         # $t0 = 100
  bne  $s0, $t0, default    # amount == 100? if not,
                            # skip to default
  addi $s1, $0, 5           # if so, fee = 5
  j    done                 # and break out of case
default:
  add  $s1, $0, $0          # fee = 0
done:
```

How It Works

1. Each case checks if amount matches a value.
2. If true, it sets fee and **jumps out** (j done).
3. If no case matches, it defaults to fee = 0.

Key Idea

- Each case **compares** a value (bne).
- **Jump (j done) avoids running extra cases**.

While Loops

A while loop **keeps repeating** until a condition becomes **false**.

Example

Code Example 6.19 `while` LOOP

High-Level Code

```
int pow = 1;
int x = 0;


while (pow != 128)
{
  pow = pow * 2;
  x = x + 1;
}
```

MIPS Assembly Code

```
# $s0 = pow, $s1 = x
  addi $s0, $0, 1       # pow = 1
  addi $s1, $0, 0       # x = 0

  addi $t0, $0, 128     # t0 = 128 for comparison
while:
  beq  $s0, $t0, done   # if pow == 128, exit while loop
  sll  $s0, $s0, 1      # pow = pow * 2
  addi $s1, $s1, 1      # x = x + 1
  j    while
done:
```

How It Works

1. $s0 = pow, $s1 = x, $t0 = 128.
2. If pow == 128, loop **exits** (beq).
3. Otherwise, pow = pow * 2, x = x + 1.
4. **Repeats** until pow == 128.

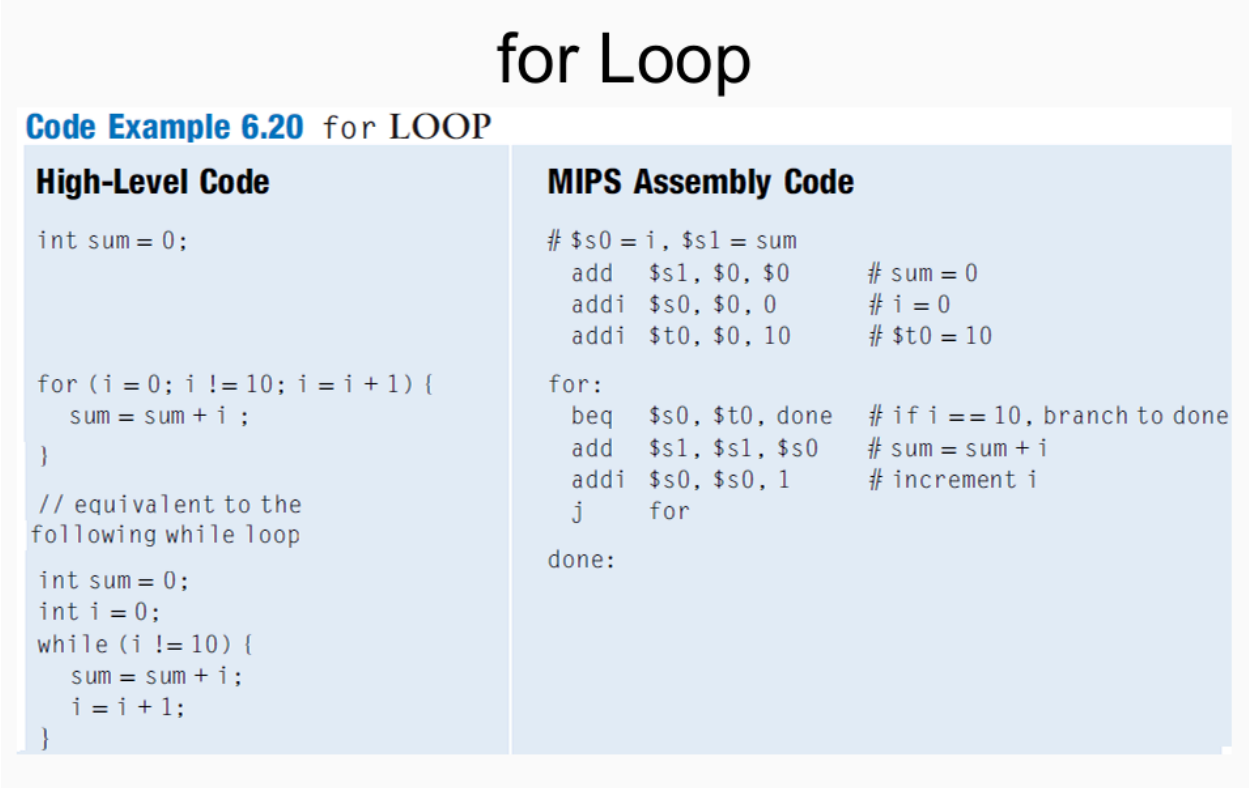
Key Idea

- **Branching (beq) exits** when the condition is met.
- **Jump (j while) repeats** the loop.

For Loops

A for loop **initializes a variable, checks a condition, and updates it** each iteration.

Example (High-Level Code)



How It Works

1. $s0 = i, $s1 = sum, $t0 = 10 (loop limit).
2. If i == 10, loop **exits** (beq).
3. Otherwise, adds i to sum, **increments** i, and repeats.

Key Idea

- **Same structure as while, but initializes i first**.
- j for **jumps back** to repeat.

Understanding Array Indexing in Memory

Example: Five-Element Array

Array Indexing

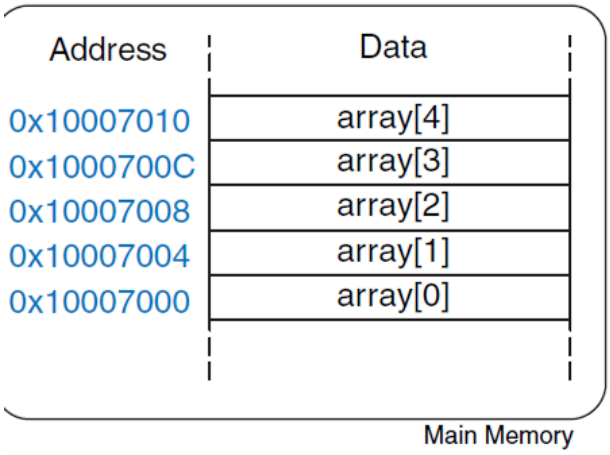


Figure 6.20 **Five-entry array with base address of 0x10007000**

- The array starts at **base address 0x10007000**.
- Each element is **4 bytes apart** (since integers take 4 bytes in memory).
- The memory layout:
  - array[0] → 0x10007000
  - array[1] → 0x10007004
  - array[2] → 0x10007008
  - array[3] → 0x1000700C
  - array[4] → 0x10007010

The **base address** of an array is the memory location of its first element.

Accessing Arrays in MIPS

Example: Multiplying Elements by 8

Code Example 6.21 ACCESSING ARRAYS

High-Level Code

```
int array[5];

array[0] = array[0] * 8;

array[1] = array[1] * 8;
```

MIPS Assembly Code

```
# $s0 = base address of array
  lui $s0, 0x1000       # $s0 = 0x10000000
  ori $s0, $s0, 0x7000  # $s0 = 0x10007000

  lw  $t1, 0($s0)       # $t1 = array[0]
  sll $t1, $t1, 3       # $t1 = $t1 << 3 = $t1 * 8
  sw  $t1, 0($s0)       # array[0] = $t1

  lw  $t1, 4($s0)       # $t1 = array[1]
  sll $t1, $t1, 3       # $t1 = $t1 << 3 = $t1 * 8
  sw  $t1, 4($s0)       # array[1] = $t1
```

How It Works:

1. Load the base address of the array into $s0.
2. Load array[0] into $t1.
3. Multiply it by 8 (sll shifts left by 3, which is the same as multiplying by 8).
4. Store the updated value back into memory.

Key Idea:

- lw (load word) gets an element.
- sw (store word) puts it back.
- Array elements are spaced **4 bytes apart**.

Accessing Arrays Using a Loop

For large arrays, we use loops instead of hardcoding each index.

Example: Multiplying 1000 Elements

Array Indexing

Code Example 6.22 ACCESSING ARRAYS USING A `for` LOOP

High-Level Code

```
int i;
int array[1000];

for (i = 0; i < 1000; i = i + 1)

  array[i] = array[i] * 8;
```

MIPS Assembly Code

```
# $s0 = array base address, $s1 = i
# initialization code
  lui  $s0, 0x23B8       # $s0 = 0x23B80000
  ori  $s0, $s0, 0xF000  # $s0 = 0x23B8F000
  addi $s1, $0, 0        # i = 0
  addi $t2, $0, 1000     # $t2 = 1000
loop:
  slt  $t0, $s1, $t2     # i < 1000?
  beq  $t0, $0, done     # if not, then done
  sll  $t0, $s1, 2       # $t0 = i*4 (byte offset)
  add  $t0, $t0, $s0     # address of array[i]
  lw   $t1, 0($t0)       # $t1 = array[i]
  sll  $t1, $t1, 3       # $t1 = array[i] * 8
  sw   $t1, 0($t0)       # array[i] = array[i] * 8
  addi $s1, $s1, 1       # i = i + 1
  j    loop              # repeat
done:
```

Dr.Anilkumar K.G 35

How It Works:

1. $s0 stores the **base address** of array[1000].
2. $s1 is the loop counter (i).
3. $t2 stores the **loop limit** (1000).
4. The loop:
    - Checks i < 1000.
    - Calculates the **memory address** of array[i] using i * 4 (bytes).
    - Loads, modifies, and stores array[i].
    - Increments i and repeats.

Key Idea:

- slt (set less than) checks the condition.
- beq **exits the loop** when i == 1000.
- sll is used to **multiply the index by 4**, since memory is **byte-addressed**.

ASCII Encoding

Computers store characters as numbers using the **ASCII table**.

ASCII values

Table 6.2 ASCII encodings

#	Char	#	Char	#	Char	#	Char	#	Char	#	Char
20	space	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y

Key ASCII Values:

- **Uppercase Letters:**
  - A = 0x41, B = 0x42, C = 0x43 ... Z = 0x5A
- **Lowercase Letters:**
  - a = 0x61, b = 0x62, c = 0x63 ... z = 0x7A
- **Digits:**
  - 0 = 0x30, 1 = 0x31, 2 = 0x32 ... 9 = 0x39
- **Special Characters:**
  - Space = 0x20, ! = 0x21, @ = 0x40, # = 0x23

Observations:

- **Uppercase and lowercase letters differ by 0x20**.
  - Example: A = 0x41, a = 0x61.
- **Digits start at 0x30**.
- **Special characters are mixed**.

ASCII in MIPS

Example: Convert Lowercase to Uppercase
To convert a **lowercase letter** to **uppercase**, we subtract 0x20.

How It Works:

1. Load 'b' (0x62) into $t0.
2. Subtract 0x20 → Result is 0x42 (B).

Key Idea:

- ASCII values allow simple arithmetic for **case conversion**.
- Useful for **text processing** in assembly.

Simple Function Call in MIPS

A basic function call jumps to a function, runs its code, and returns.

Example: Simple Function Without Arguments

Code Example 6.23 `simple` FUNCTION CALL

High-Level Code

```
int main() {
  simple();
  . . .
}
// void means the function returns no value
void simple() {
  return;
}
```

MIPS Assembly Code

```
0x00400200 main:    jal simple  # call function
0x00400204          . . .


0x00401020 simple:  jr $ra      # return
```

How It Works

1. jal simple jumps to the function simple and saves the return address in $ra.
2. The function simple executes.
3. jr $ra jumps back to the instruction after jal.

Key Idea

- **jal (jump and link)** saves the return address in $ra.
- **jr $ra (jump register)** returns to the caller.

Function with Arguments and Return Values

A function can **take inputs** and **return a result**.

Example: Compute Difference of Sums

Function Calls

Code Example 6.24 FUNCTION CALL WITH ARGUMENTS AND RETURN VALUES

High-Level Code

```
int main()
{
  int y;

  . . .

  y = diffofsums(2, 3, 4, 5);

  . . .
}

int diffofsums(int f, int g, int h, int i)
{
  int result;
  result = (f + g) - (h + i);
  return result; }
```

MIPS Assembly Code

```
# $s0 = y

main:
  . . .
  addi $a0, $0, 2    # argument 0 = 2
  addi $a1, $0, 3    # argument 1 = 3
  addi $a2, $0, 4    # argument 2 = 4
  addi $a3, $0, 5    # argument 3 = 5
  jal  diffofsums    # call function
  add  $s0, $v0, $0  # y = returned value
  . . .
# $s0 = result
diffofsums:
  add $t0, $a0, $a1  # $t0 = f + g
  add $t1, $a2, $a3  # $t1 = h + i
  sub $s0, $t0, $t1  # result = (f + g) - (h + i)
  add $v0, $s0, $0   # put return value in $v0
  jr  $ra            # return to caller
```

How It Works

1. Arguments are passed in registers $a0 - $a3.
2. The function performs the computation.
3. The result is stored in $v0 and returned.

Key Idea

- $a0 - $a3 store function **inputs**.
- $v0 stores the **return value**.
- **jal calls the function**, and **jr $ra returns**.

The Stack in MIPS

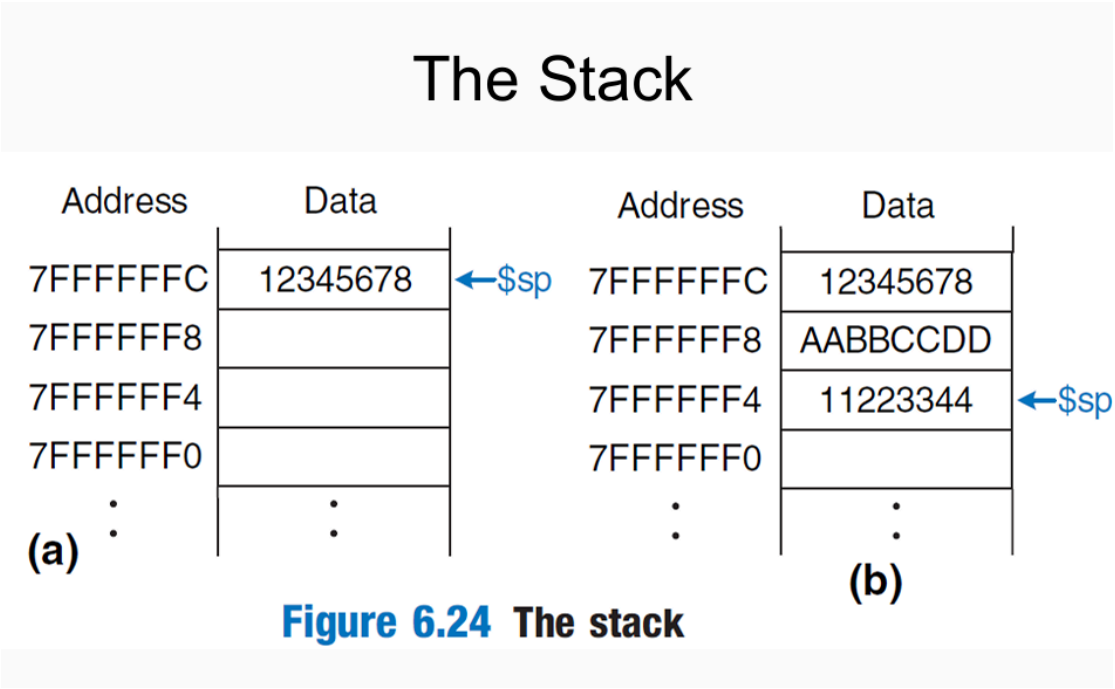


Figure 6.24 The stack

The **stack** is a memory section that stores **temporary data**, such as:

- Local variables
- Saved registers
- Return addresses

The **stack pointer ($sp)** keeps track of the top of the stack.

Stack Characteristics

- **LIFO (Last In, First Out)** – like a stack of plates.
- **Grows downward** in memory.
- **Managed manually in MIPS**.

Example: Stack Before and After Pushing Data

Before pushing data:

- $sp = 0x7FFFFFFC (top of the stack)

After pushing two values (AABBCCDD and 11223344):

- $sp moves to 0x7FFFFFF4
- New values are stored in memory.

Using the Stack to Save Registers

When a function uses registers, it must **save them** before modifying them.

Example: Improved Function Using the Stack

Function Saving Registers on the Stack: Code Example 6.25.

MIPS Assembly Code

```
# $s0 = result
diffofsums:
   addi $sp, $sp, -12 #  make space on stack to store three registers
   sw   $s0, 8($sp)   #  save $s0 on stack
   sw   $t0, 4($sp)   #  save $t0 on stack
   sw   $t1, 0($sp)   #  save $t1 on stack
   add  $t0, $a0, $a1 #  $t0 = f + g
   add  $t1, $a2, $a3 #  $t1 = h + i
   sub  $s0, $t0, $t1 #  result = (f + g) - (h + i)
   add  $v0, $s0, $0  #  put return value in $v0
   lw   $t1, 0($sp)   #  restore $t1 from stack
   lw   $t0, 4($sp)   #  restore $t0 from stack
   lw   $s0, 8($sp)   #  restore $s0 from stack
   addi $sp, $sp, 12  #  deallocate stack space
   jr   $ra           #  return to caller
```

Dr.Anilkumar K.G 50

How It Works

1. **Stack Space Allocation**
   - addi $sp, $sp, -12 reserves space for 3 registers.
2. **Saving Registers**
   - sw stores $s0, $t0, and $t1 on the stack.
3. **Function Computation**
   - Performs (f + g) - (h + i).
4. **Restoring Registers**
   - lw reloads the saved registers.
5. **Releasing Stack Space**
   - addi $sp, $sp, 12 frees memory.

Key Idea

- **Registers must be saved before modification.**
- **Stack space must be freed after use.**

Stack Behavior During Function Calls

Stack Before Function Call

- $sp points to the last used memory location.

Stack During Function Call

- Function **pushes registers** onto the stack.
- $sp moves **down**.

Stack After Function Call

- Function **restores registers**.
- $sp moves **back up**.

Addressing Modes in MIPS

Addressing modes determine how data is accessed. The common types are:

Register Addressing

- Operands are stored in registers.
- Example: Add R4, R3
- Meaning: Regs[R4] = Regs[R4] + Regs[R3]
- Used when data is already in registers.

Immediate Addressing

- A constant value is directly used.
- Example: Add R4, #3
- Meaning: Regs[R4] = Regs[R4] + 3
- Used when working with fixed values.

Displacement (Base) Addressing

- Access memory using a register and an offset.
- Example: Add R4, 100(R1)
- Meaning: Regs[R4] = Regs[R4] + Mem[100 + Regs[R1]]
- Used for local variables and arrays.

Register Indirect Addressing

- Access memory using a pointer stored in a register.
- Example: Add R4, (R1)
- Meaning: Regs[R4] = Regs[R4] + Mem[Regs[R1]]
- Used for dynamic memory access.

Indexed Addressing

- Uses two registers to calculate the memory address.
- Example: Add R3, (R1+R2)
- Meaning: Regs[R3] = Regs[R3] + Mem[Regs[R1] + Regs[R2]]
- Used in array processing.

Direct (Absolute) Addressing

- Uses a fixed memory address.
- Example: Add R1, (1001)
- Meaning: Regs[R1] = Regs[R1] + Mem[1001]
- Used for accessing static memory locations.

Memory Indirect Addressing

- A register contains the memory address of another pointer.
- Example: Add R1, 0(R3)
- Meaning: Regs[R1] = Regs[R1] + Mem[Mem[Regs[R3]]]
- Used when working with pointers.

Auto-Increment Addressing

- Accesses memory and then updates the register.
- Example: Add R1, (R2)+
- Meaning: Regs[R1] = Regs[R1] + Mem[Regs[R2]]
- Then: Regs[R2] = Regs[R2] + d
- Used for stepping through arrays.

Auto-Decrement Addressing

- Decreases the register before accessing memory.
- Example: Add R1, -(R2)
- Meaning: Regs[R2] = Regs[R2] - d
- Then: Regs[R1] = Regs[R1] + Mem[Regs[R2]]
- Used for stacks and recursion.

Scaled Addressing

- Multiplies an index register before adding it to a base.

- Example: Add R1, 100(R2)[R3]
- Meaning: Regs[R1] = Regs[R1] + Mem[100 + Regs[R2] + Regs[R3] * d]
- Used for fast array access.

MIPS Memory Map

The MIPS architecture organizes memory into **different segments**.

Memory Overview

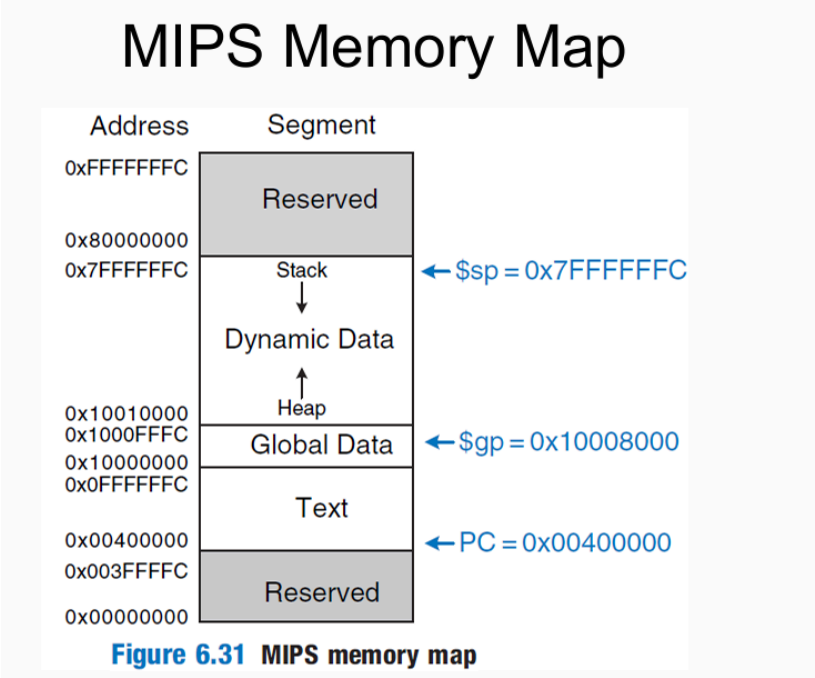


Figure 6.31 MIPS memory map

- **32-bit addresses → 4GB of memory**.
- Each **word (4 bytes)** is stored at an address divisible by 4.
- The memory is divided into **text, data, stack, heap, and reserved sections**.

Memory Segments

Text Segment

- Stores **machine code** (compiled program instructions).
- Starts at **0x00400000**.
- The program counter (**PC**) points to instructions here.

Global Data Segment

- Stores **global variables** (accessible by all functions).

- Starts at **0x10000000**.
- The global pointer (**$gp**) is set at **0x10008000** for easy access.
- Variables are stored **relative to $gp**.

Dynamic Data Segment

- Contains **heap and stack**.
- Heap:
  - Stores **dynamically allocated memory** (e.g., malloc in C).
  - Grows **upward** in memory.
- Stack:
  - Stores **local variables, function calls, and return addresses**.
  - Grows **downward** from **0x7FFFFFFC**.
  - The stack pointer (**$sp**) tracks the top of the stack.

Reserved Segments

- Memory areas **reserved for the OS**.
- Cannot be directly accessed by programs.

Memory Access in MIPS

Global Variables

- Accessed using **base addressing with $gp**.
- Example: lw $t0, 8($gp) loads a global variable.

Local Variables and Stack

- Accessed using **base addressing with $sp**.
- Example: sw $t0, -4($sp) saves a variable on the stack.

Dynamic Memory Allocation

- Uses **syscalls (sbrk, malloc)** to request heap memory.

Translating and Starting a Program

When a program is written in a high-level language like C, it needs to be translated into machine code before the computer can execute it. This process involves several steps:

Translating and Starting a Program

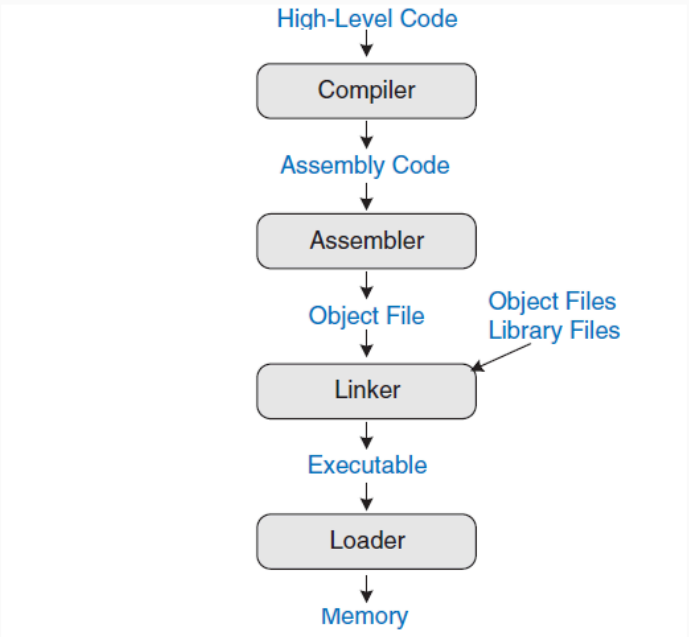


Figure 6.32 Steps for translating and starting a program

Compilation

The **compiler** converts high-level code into **assembly language**. Assembly language is human-readable but still closely tied to machine code.

For example, a C program:

Code Example 6.30 COMPILING A HIGH-LEVEL PROGRAM

High-Level Code

```
int f, g, y; // global variables

int main(void)
{
  f = 2;
  g = 3;
  y = sum(f, g);
  return y;
}

int sum(int a, int b) {
  return (a + b);
}
```

MIPS Assembly Code

```
.data
f:
g:
y:

.text
main:
  addi $sp, $sp, -4  # make stack frame
  sw   $ra, 0($sp)   # store $ra on stack
  addi $a0, $0, 2    # $a0 = 2
  sw   $a0, f        # f = 2
  addi $a1, $0, 3    # $a1 = 3
  sw   $a1, g        # g = 3
  jal  sum           # call sum function
  sw   $v0, y        # y = sum(f, g)
  lw   $ra, 0($sp)   # restore $ra from stack
  addi $sp, $sp, 4   # restore stack pointer
  jr   $ra           # return to operating system

sum:
  add  $v0, $a0, $a1 # $v0 = a + b
  jr   $ra           # return to caller
```

Assembling

The **assembler** converts assembly code into **machine code** (binary instructions that the computer can understand).

It does this in **two passes**:

1. **First pass:** It assigns addresses to instructions and labels.
2. **Second pass:** It translates assembly into machine instructions.

Example machine code generated after the first assembler pass:

- The code after the **first assembler pass** is shown here.

```
0x00400000 main: addi $sp, $sp, -4
0x00400004       sw   $ra, 0($sp)
0x00400008       addi $a0, $0, 2
0x0040000C       sw   $a0, f
0x00400010       addi $a1, $0, 3
0x00400014       sw   $a1, g
0x00400018       jal  sum
0x0040001C       sw   $v0, y
0x00400020       lw   $ra, 0($sp)
0x00400024       addi $sp, $sp, 4
0x00400028       jr   $ra
0x0040002C sum:  add  $v0, $a0, $a1
0x00400030       jr   $ra
```

This machine code is saved in an **object file**.

Linking

Programs often call functions stored in **libraries** (like printf in C). These library functions are in separate files. The **linker** combines object files and libraries into a single executable program.

To make this work, the assembler creates a **symbol table** to keep track of variables and function addresses. Example symbol table:

Table 6.4 Symbol table

Symbol	Address
f	0x10000000
g	0x10000004
y	0x10000008
main	0x00400000
sum	0x0040002C

Translating and Starting a Program

Executable file header	Text Size	Data Size
	0x34 (52 bytes)	0xC (12 bytes)
Text segment	**Address**	**Instruction**
	0x00400000	0x23BDFFFC
	0x00400004	0xAFBF0000
	0x00400008	0x20040002
	0x0040000C	0xAF848000
	0x00400010	0x20050003
	0x00400014	0xAF858004
	0x00400018	0x0C10000B
	0x0040001C	0xAF828008
	0x00400020	0x8FBF0000
	0x00400024	0x23BD0004
	0x00400028	0x03E00008
	0x0040002C	0x00851020
	0x00400030	0x03E00008
Data segment	**Address**	**Data**
	0x10000000	f
	0x10000004	g
	0x10000008	y

```
addi $sp, $sp, –4
sw   $ra, 0($sp)
addi $a0, $0, 2
sw   $a0, 0x8000($gp)
addi $a1, $0, 3
sw   $a1, 0x8004($gp)
jal  0x0040002C
sw   $v0, 0x8008($gp)
lw   $ra, 0($sp)
addi $sp, $sp, –4
jr   $ra
add  $v0, $a0, $a1
jr   $ra
```

Figure 6.33 Executable

The **linker adjusts addresses** so that function calls and memory references point to the right locations.

Loading

The **operating system (OS)** loads the program into memory and starts execution.

Translating and Starting a Program

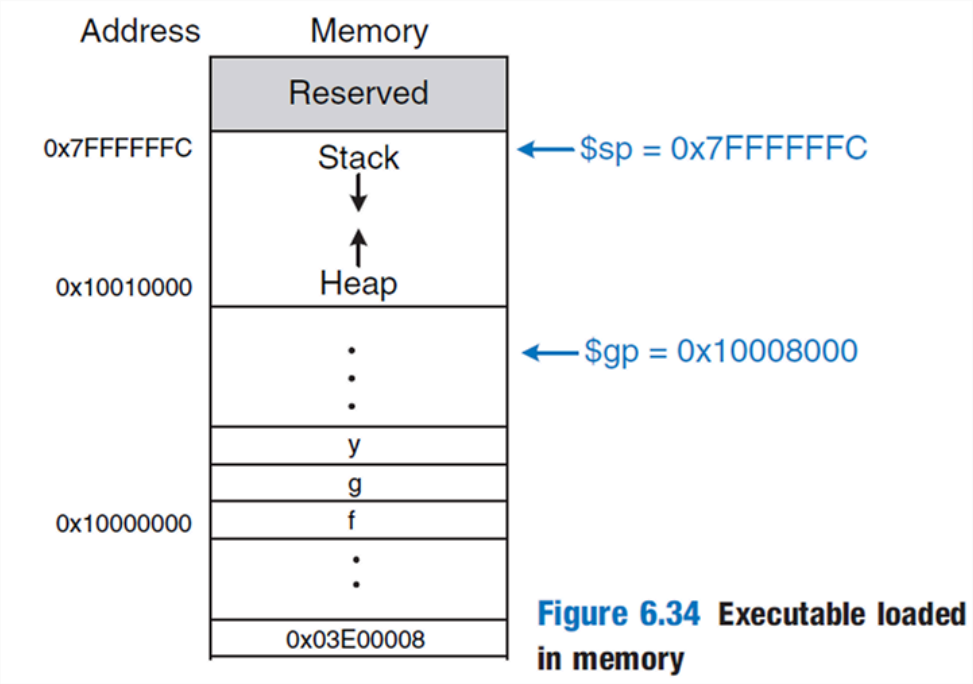


Figure 6.34 Executable loaded in memory

- It **reads the text segment** from the executable file and places it into memory.
- It **sets up registers**:
  - $gp (global pointer) is set to 0x10008000, which is in the middle of the **global data segment**.
  - $sp (stack pointer) is set to 0x7FFFFFFC, which is at the top of the **dynamic data segment**.
  - The **Program Counter (PC)** is set to 0x00400000, pointing to the first instruction.

Execution

Once the program is loaded, the **CPU fetches** the first instruction from memory at 0x00400000 and starts execution.

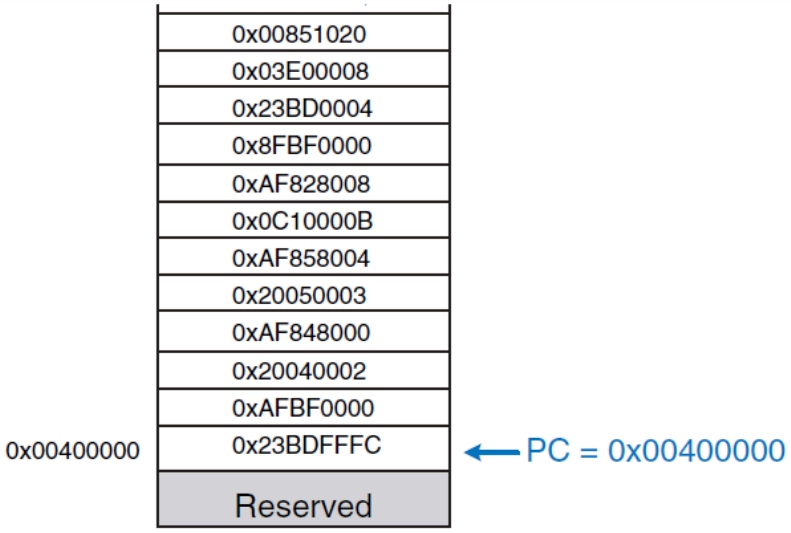


Figure 6.34 Executable loaded in memory

Dr.Anilkumar K.G

- If the instruction is a function call (jal sum), it jumps to the function's address (0x0040002C).
- The function runs, computes its result, stores it in $v0, and returns to the caller (jr $ra).
- Execution continues in **main** until it reaches the end, at which point the program terminates.

Lecture 7

Architectural State and Instruction Set

- The **architectural state** of a MIPS 32-bit system has:
  - A **program counter (PC)**.
  - **32 registers**, each storing 32 bits.
- An instruction uses the current state (values in PC and registers) to produce a **new state**.
- We focus on a **subset of MIPS instructions**:
  - **R-type** (e.g., **add**, **sub**, **and**, **or**).
  - **Memory** (e.g., **lw**, **sw**).
  - **Branch** (e.g., **beq**).
  - Later, we add **addi** and **j**.

Design Process

1. **Datapath**
    - Moves and processes **32-bit data**.
    - Has components like **memories**, **registers**, **ALUs**, and **multiplexers**.
2. **Control Unit**
    - Reads the **current instruction** from the datapath.
    - Generates signals (e.g., multiplexer selects, register enables) to guide the datapath.

MIPS System Components

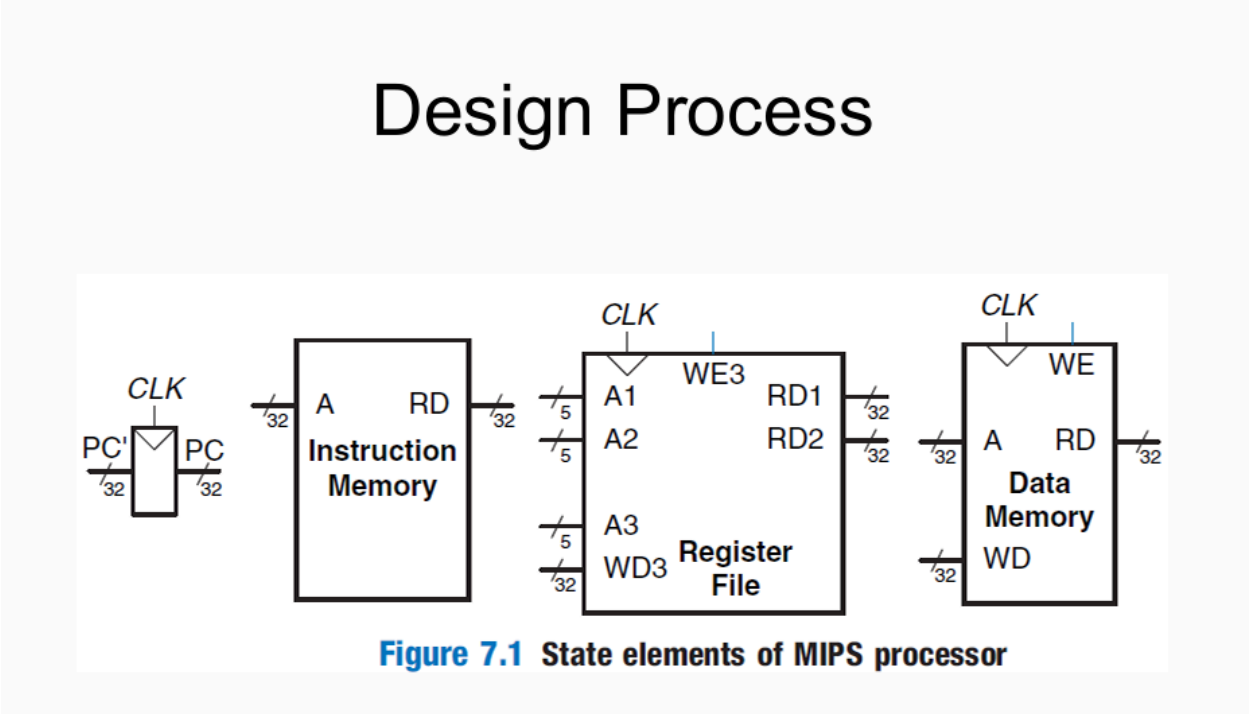
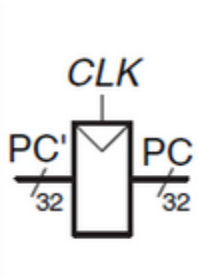


Figure 7.1 State elements of MIPS processor

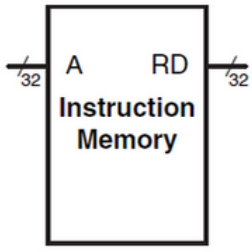
- Four **state elements**: **program counter**, **instruction memory**, **register file**, **data memory**.
- **32-bit data buses** are shown as heavy lines in diagrams.
- **5-bit address buses** (for registers) are medium lines.
- **Control signals** (like write enable) are narrow lines.

Program Counter (PC)



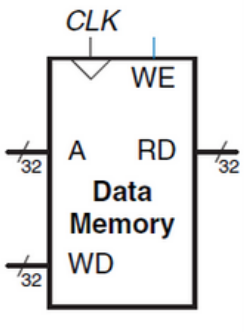
- A **32-bit register**.
- **Output** (PC) points to the current instruction.
- **Input** (PC′) points to the next instruction address.

Instruction Memory



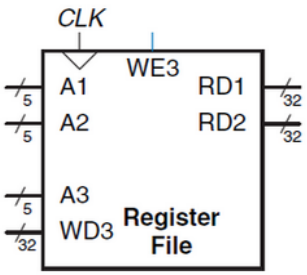
- Has **one read port**.
- Takes a **32-bit address** (A) and outputs a **32-bit instruction** (RD).

Data Memory



- Has **one read/write port**.
- **Write enable (WE)** controls if data is written or read.
- If WE = 1, data from **WD** is written to **address A** on the next clock edge.
- If WE = 0, data at **address A** is read onto **RD**.

Register File



- Holds **32 registers**, each **32 bits** wide.
- Has **two read ports** and **one write port**.
- **Read ports**:
  - **A1** and **A2** are 5-bit addresses.
  - **RD1** and **RD2** are the 32-bit outputs.
- **Write port**:
  - **A3** (5-bit address), **WD** (32-bit data), **WE3** (write enable).
  - If WE3 = 1, data in WD is written to register A3 on the clock's rising edge.

Single Cycle MIPS Micro-architecture

- Executes **one instruction** fully in **one clock cycle**.
- Easy to explain and has a **simple control unit**.
- The **clock period** must be long enough for the **slowest instruction** to finish.

Multi-cycle MIPS Micro-architecture

- Splits instruction execution into **multiple shorter cycles**.
- **Complex instructions** take more cycles; **simpler** ones take fewer.
- Reuses **hardware blocks** (like adders or memories) across different cycles.
- Needs **extra registers** to store intermediate results between cycles.

Pipelined MIPS Micro-architecture

- Adds **pipelining** to the single-cycle design.
- Divides execution into **five pipeline stages**.
- Several instructions run **at the same time** (one in each stage).
- Improves **throughput** (more instructions per second) but adds **extra pipeline registers** and **dependency handling**.

Single Cycle Datapath Overview

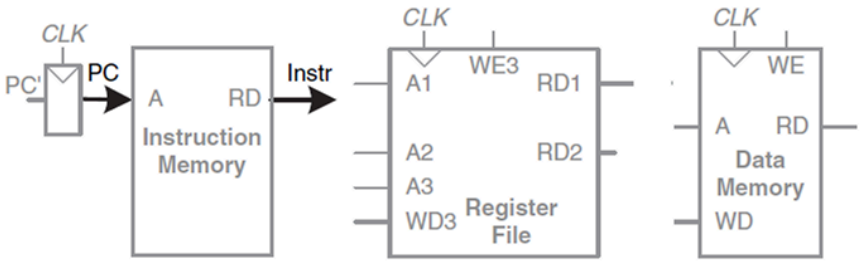


Figure 7.2 Fetch instruction from memory

Instruction Fetch

- The **Program Counter (PC)** points to the current instruction.
- The **Instruction Memory** uses the PC as an **address** input.
- It **outputs** a 32-bit **instruction** (called **Instr**).

Reading Registers

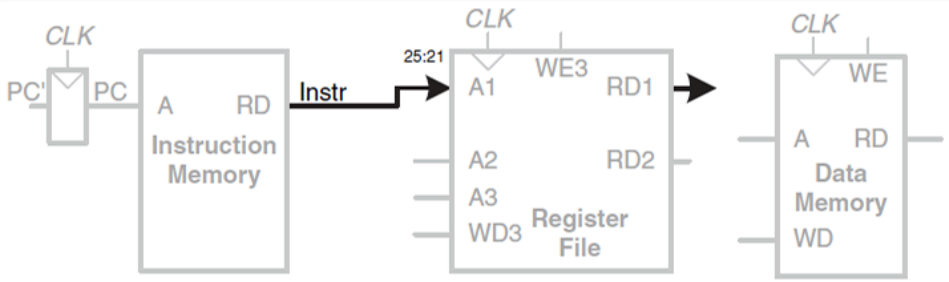


Figure 7.3 Read source operand from register file

- The instruction has **fields** that indicate which registers to read.
- For **lw**, the **rs** field (**Instr[25:21]**) selects the **base register** address.
- The **Register File** has two **read ports** (**A1**, **A2**) and one **write port**.
- The **register file** outputs the contents of the selected register onto **RD1** (and **RD2** if needed).

Sign Extension

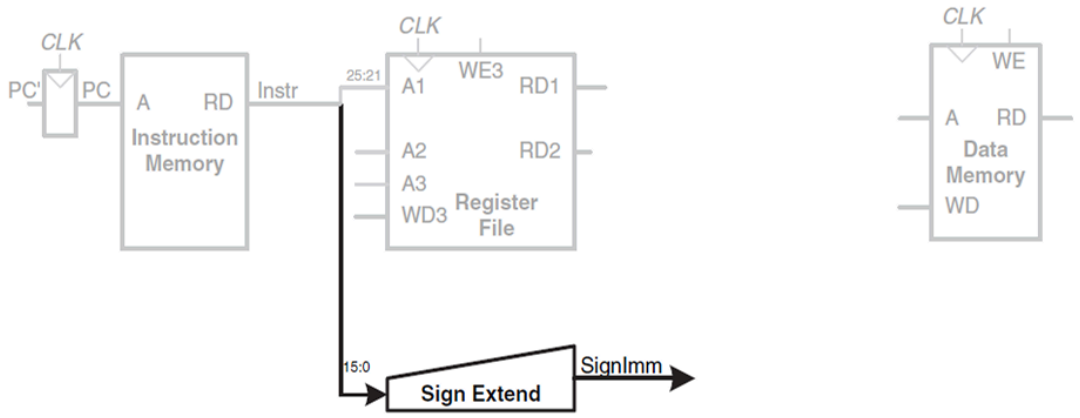


Figure 7.4 Sign-extend the immediate

- The **immediate** field (**Instr[15:0]**) is **16 bits**.
- For **lw**, we need to interpret this value as a **signed offset**.
- The **Sign-Extend** unit copies the sign bit (bit 15) into the upper 16 bits, creating a **32-bit** value (**SignImm**).

ALU Operation

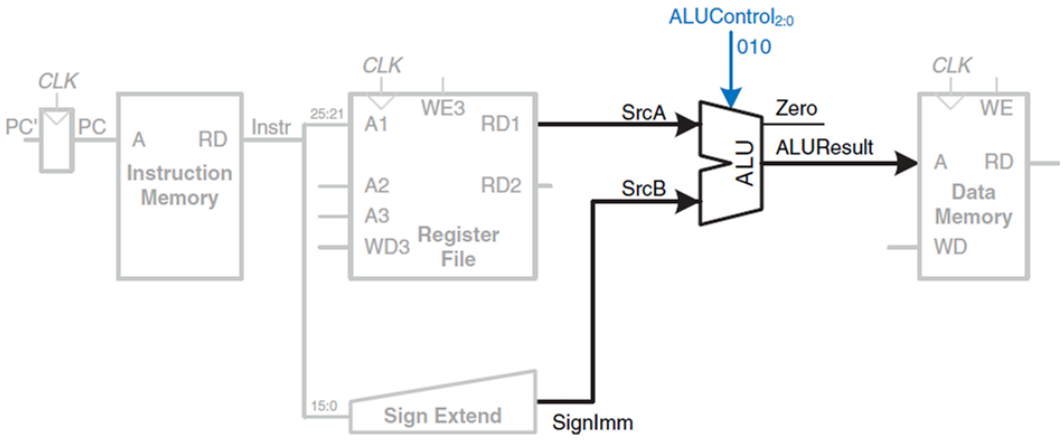


Figure 7.5 Compute memory address

- The **ALU** computes the **address** for memory access by adding:
  - **SrcA** (base register value)
  - **SrcB** (the sign-extended offset)
- The **ALUControl** signals tell the ALU which operation to do (e.g., **010** for add).

- The **ALU** produces a **32-bit result** (**ALUResult**) and a **Zero** flag.

Memory Access

- For **lw**, **ALUResult** is the **memory address**.
- The **Data Memory** reads from that address and places the data on **ReadData**.

Write Back

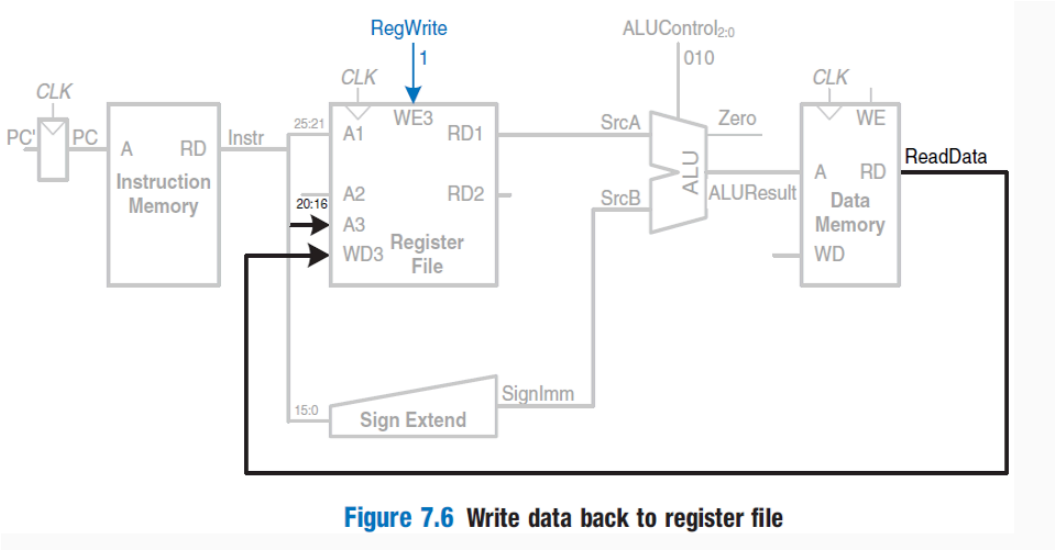


Figure 7.6 Write data back to register file

- The **ReadData** is then written into the **Register File**.
- **A3** (the write port address) comes from the **rt** field (**Instr[20:16]**) in the lw instruction.
- The **Write Enable** signal (**RegWrite**) ensures data is stored only if needed.

Control Unit

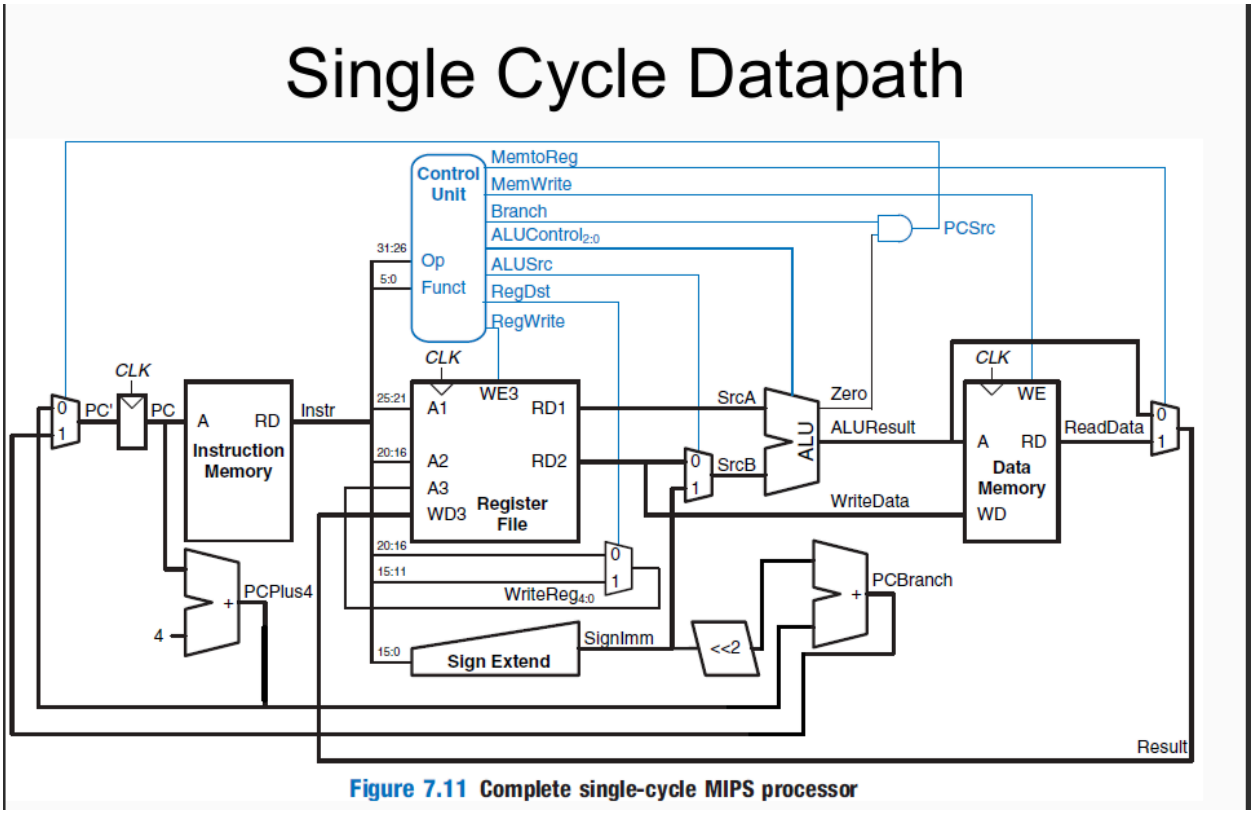


Figure 7.11 Complete single-cycle MIPS processor

- Looks at the **opcode** (**Instr[31:26]**) and, if necessary, the **funct** field (**Instr[5:0]**).
- Sets **control signals** (e.g., **MemWrite**, **MemtoReg**, **ALUSrc**, **RegDst**, **Branch**) so each instruction does the correct steps.

Lecture 8

- **10^{-3}** means **0.001**. This is called **milli**.
  Example: **1 millisecond (ms)** is **0.001 seconds**.
- **10^{-6}** means **0.000001**. This is called **micro**.
  Example: **1 microsecond (µs)** is **0.000001 seconds**.
- **10^{-9}** means **0.000000001**. This is called **nano**.
  Example: **1 nanosecond (ns)** is **0.000000001 seconds**.
- **10^{3}** means **1,000**. This is called **kilo**.
  Example: **1 kilobyte (KB)** is **1,000 bytes**.
- **10^{6}** means **1,000,000**. This is called **million**.
  Example: **1 megabyte (MB)** is **1,000,000 bytes**.
- **10^{9}** means **1,000,000,000**. This is called **giga** or **billion**.
  Example: **1 gigabyte (GB)** is **1,000,000,000 bytes**.

Understanding Computer Performance Evaluation

When designing a computer or a part of it, we need to check how well it works. There are two main ways to do this:

1. **Simulation** – This method runs a test model of the system to see how it performs. But it has a downside: it can hide the effects of different workloads (types of tasks) or system settings.
2. **Analytic Models** – These are mathematical models that calculate performance based on different factors like workload and system design. Unlike simulation, these models clearly show how each factor affects performance.

Sometimes, both methods are used together. A small part of the system is simulated, and an analytical model predicts the full system's performance.

What is Computer Performance?

To measure how good a computer is, we usually look at two things:

1. **Time taken to complete a task** – This tells us how long it takes to finish one job.
2. **Rate of task completion** – This tells us how many jobs are done per second.

For example:

- If a task takes 3 microseconds, it means each job needs 3 microseconds to finish.
- If a computer does 333,000 tasks per second (3.33×10^5), this is the rate at which it completes jobs.

Time and rate are opposites. If time per task goes down, the task rate goes up.

How the Computer's Clock Affects Performance

Every computer has a clock that controls how fast it works. We can measure performance using the number of clock cycles a task needs.

Example:

- **Clock Period**: 50 ns (means each clock cycle is 50 nanoseconds).
- **Clocks per Task**: 300 clocks are needed to finish Task A.
- **One Task in Clocks**: If 300 clocks are needed, then we have **1 / 300 = 0.0033 tasks** per clock.
- **Time per Task**: 300 clocks × 50 ns/clock = **15,000 ns** (or 15 µs) for one Task A.

This helps us understand how efficiently a processor is using its clock cycles.

Understanding CPI (Clocks Per Instruction)

A very common way to measure a processor's speed is CPI (Clocks Per Instruction). This tells us how many clock cycles are needed to complete one instruction.

Formula:
CPI = number of CPU clocks used ÷ number of instructions executed

The inverse of CPI is IPC (Instructions Per Clock):
IPC = 1 ÷ CPI

- CPI (Clocks Per Instruction) → Lower CPI means better performance.
- IPC (Instructions Per Clock) → Higher IPC means better performance.

Example of CPI Calculation

If a task needs **1,000,000 clock cycles** (1×10^6) and executes **500,000 instructions** (5×10^5), then:

CPI = 1,000,000 ÷ 500,000 = 2

This means, on average, each instruction takes 2 clock cycles to complete. Lower CPI is better because it means the processor is executing instructions faster.

Why CPI Varies?

- Not all instructions take the same number of clock cycles to execute.
- Some tasks need more processing power, so they take more cycles.
- CPI is usually an average value for all instructions executed.

Example 1 : Calculating CPI

- **Instruction Mix and Cycle Counts:**
    1. **R-type:** 50% of instructions, 4 cycles each
    2. **Load:** 15% of instructions, 5 cycles each
    3. **Store:** 25% of instructions, 4 cycles each
    4. **Branch:** 8% of instructions, 3 cycles each
    5. **Jump:** 2% of instructions, 3 cycles each
- **Steps to Calculate CPI:**
    1. **Multiply each percentage by its cycle count:**
        - R-type: 50 × 4 = 200
        - Load: 15 × 5 = 75
        - Store: 25 × 4 = 100
        - Branch: 8 × 3 = 24

        - Jump: 2 × 3 = 6
    2. **Add the results:**
        - Total = 200 + 75 + 100 + 24 + 6 = **405**
    3. **Divide by 100 (total percentage):**
        - CPI = 405 / 100 = **4.05** cycles per instruction

Example 2: Calculating CPI, CPU Time, and MIPS

- **Given Information:**
    - **Processor Clock Rate:** 3 GHz (3,000,000,000 cycles per second)
    - **Instruction Counts:**
        - **Integer arithmetic:** 45,000 instructions, 1 cycle each
        - **Data transfer:** 32,000 instructions, 2 cycles each
        - **Floating point:** 15,000 instructions, 3 cycles each
        - **Control transfer:** 8,000 instructions, 5 cycles each
- **Step 1: Total Clock Cycles**
    - Integer arithmetic: 45,000 × 1 = **45,000 cycles**
    - Data transfer: 32,000 × 2 = **64,000 cycles**
    - Floating point: 15,000 × 3 = **45,000 cycles**
    - Control transfer: 8,000 × 5 = **40,000 cycles**
    - **Total cycles:** 45,000 + 64,000 + 45,000 + 40,000 = **194,000 cycles**
- **Step 2: Calculate CPI**
    - **Total instructions:** 45,000 + 32,000 + 15,000 + 8,000 = **100,000 instructions**
    - **CPI:** 194,000 / 100,000 = **1.94 cycles/instruction**
- **Step 3: Calculate CPU Time**
    - **Clock cycle time:** 1 / 3,000,000,000 = **0.333×10^{-9} seconds** (about 0.333 nanoseconds per cycle)
    - **CPU Time:** 194,000 cycles × (1/3,000,000,000 seconds per cycle) ≈ **0.0000647 seconds**
        - (This is about 64.7(64.7 x **10^5** microseconds)
- **Step 4: Calculate MIPS Rate**
    - **MIPS** stands for **Million Instructions Per Second**
    - **Formula:** MIPS = (Clock Frequency) / (CPI × 1,000,000(**10^6**))
    - Calculation: 3,000,000,000 / (1.94 × 1,000,000) ≈ **1.546 x 10^3 MIPS**
        - (This means the processor can execute roughly 1,546 million instructions per second)

Speedup

- **Definition:**

    - Speedup measures how much faster one design (or system) is compared to another.
- **How to Calculate Speedup:**
    - **Time Method:**
        - Example:
            - Design A takes **10 seconds**.
            - Design B takes **5 seconds**.
            - **Speedup = Time of A / Time of B = 10 / 5 = 2.**
            - **Interpretation:** Design B is twice as fast.
    - **CPI Method (Clocks Per Instruction):**
        - Example:
            - Design A has a **CPI of 2**.
            - Design B has a **CPI of 1**.
            - **Speedup = CPI of A / CPI of B = 2 / 1 = 2.**
            - **Note:** Lower CPI means better (faster) performance.
    - **IPC Method (Instructions Per Cycle):**
        - Example:
            - Design A has an **IPC of 1**.
            - Design B has an **IPC of 2**.
            - **Speedup = IPC of B / IPC of A = 2 / 1 = 2.**
            - **Note:** Higher IPC means better performance.
    - **Rate Method:**
        - Example:
            - Design A completes **1 task per second**.
            - Design B completes **2 tasks per second**.
            - **Speedup = Rate of B / Rate of A = 2 / 1 = 2.**
- **How to Interpret Speedup:**
    - – If design B is an improvement, then speedup will be greater than 1.
    - – If design B hurts performance, the speedup will be less than 1.
    - – If speedup is equal to 1, there is no performance change

---

Percent Difference and Change

Definition:
Percent difference tells us by what percentage one system is better or worse than another.

Calculating Percent Difference Using Rate:

Formula:
Percent Difference = 100 × (Speedup - 1)

Example:
Processor A does 3 tasks per second.

Processor B does 2 tasks per second.
Speedup = 3 / 2 = 1.5
Percent Difference = 100 × (1.5 - 1) = 50%
Interpretation: Processor A is 50% faster than Processor B.

Calculating Percent Difference Using Time:

Formula:
Percent Difference = 100 × (1 - Time of A / Time of B)
or
Percent Difference = 100 × (1 - 1 / Speedup)

Example:
Processor A takes 3 seconds.
Processor B takes 4 seconds.
Percent Difference = 100 × (1 - 3 / 4) = 100 × 0.25 = 25%
Interpretation: Processor A is 25% faster than Processor B.

Understanding Means and Weighted Means in Performance Measurement

To compare different computers or evaluate design improvements, we need a **single number** that summarizes performance. This allows easy comparison between systems.

Three Types of Performance Measurements

1. **Time taken to perform a task** – Measures how long a task takes to complete.
2. **Rate of task execution** – Measures how many tasks a system completes per unit of time.
3. **Ratio of times or rates (Speedup)** – Compares the performance of one system against another to see the improvement.

Types of Means for Performance Analysis

Different types of averages (means) help summarize performance measurements:

- **Arithmetic Mean** → Used when measuring time values. It calculates the average time taken across multiple tasks.
- **Harmonic Mean** → Used when measuring rates. It provides a better representation when dealing with performance speeds.
- **Geometric Mean** → Used when comparing ratios, such as speedup between two systems.

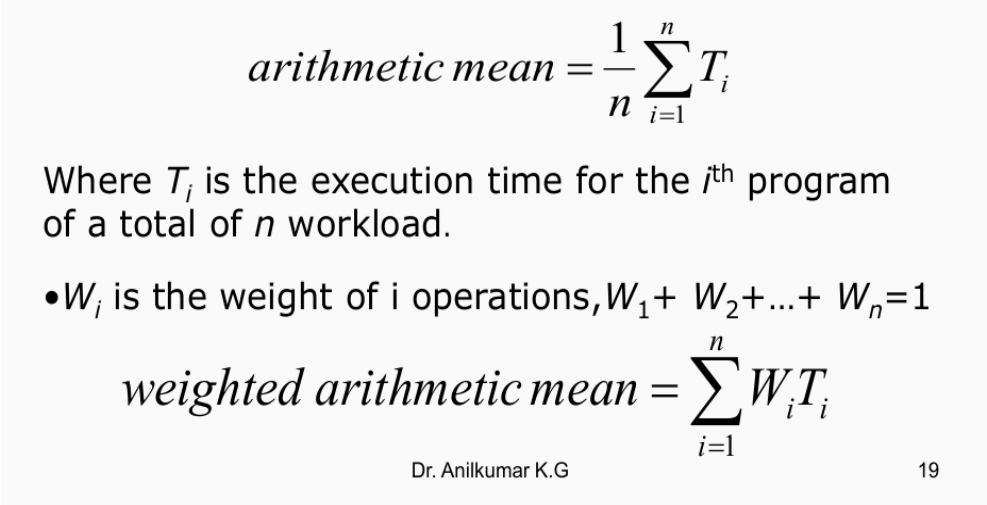
Understanding Weighted Means

Sometimes, different tasks do not happen with the same frequency. Some tasks are more common than others. To handle this, we use **weights** in calculations.

- **Weight** represents how often an event occurs.
- It is usually written as a fraction (x/100) or a percentage.
- The total of all weights must always add up to **1 (or 100%)**.

By using weighted means, we ensure that more frequent tasks have a larger impact on the performance measurement, giving a more **realistic** evaluation.

Time-Based Means



Arithmetic Mean (Simple Example)

- Suppose you run three programs, and their execution times are **10 seconds**, **15 seconds**, and **20 seconds**.
- To find the **arithmetic mean**, add them: 10 + 15 + 20 = **45**.
- Divide by the number of programs (3): 45 / 3 = **15 seconds**.

Weighted Arithmetic Mean (Simple Example)

- Imagine you have two tasks. Task A runs **3 times** more often than Task B.
- Task A's time is **10 seconds**. Task B's time is **20 seconds**.
- Total "weight" is 3 (for A) + 1 (for B) = 4.
- **Multiply** each time **by its weight** and **add them**: (10 × 3) + (20 × 1) = **30 + 20 = 50**.
- Divide **by total weight** (4): 50 / 4 = **12.5 seconds**.

Rate-Based Means

$$\text{harmonic mean} = \frac{1}{\frac{1}{n}\sum_{i=1}^{n}\frac{1}{R_i}} = \frac{n}{\sum_{i=1}^{n}\frac{1}{R_i}}$$

(Reciprocal of arithmetic mean)

Where $T_i = 1/R_i$

$$\text{weighted harmonic mean} = \frac{1}{\sum_{i=1}^{n}\frac{W_i}{R_i}}$$

(Reciprocal of weighted arithmetic mean)

W_i *is the fraction of total task.*

Harmonic Mean

- This is useful when dealing with **rates** (like jobs per hour).
- Suppose you can complete a task at **0.5 jobs/hour** and your friend can do **0.5 jobs/hour** too.
- The harmonic mean of these two rates:
    - Take reciprocals: 1 / 0.5 = 2 and 1 / 0.5 = 2.
    - Add them: 2 + 2 = 4.
    - Divide the number of rates (2) by 4 = 2 / 4 = **0.5 jobs/hour** **(add jobs/hours and divide them by the number of rates)** .

Weighted Harmonic Mean

- If one rate applies **60%** of the time, and another rate applies **40%** of the time, you weight the reciprocals by those percentages.
- Then you take the reciprocal of that weighted sum to get the final rate.

Example (Rate-Based Mean)

Let's assume these weights:

- w1 = 0.2
- w2 = 0.3
- w3 = 0.25
- w4 = 0.25
  (Sum: 0.2 + 0.3 + 0.25 + 0.25 = 1)

Steps:

1. Divide each weight by its corresponding rate:
    - w1 / R1 = 0.2 / 0.5 = 0.4
    - w2 / R2 = 0.3 / 0.45 ≈ 0.6667

    - w3 / R3 = 0.25 / 0.53 ≈ 0.4717
    - w4 / R4 = 0.25 / 0.43 ≈ 0.5814
2. Add them up:
   0.4 + 0.6667 + 0.4717 + 0.5814 ≈ 2.1198
3. Calculate the weighted harmonic mean:
   1 / 2.1198 ≈ **0.471 jobs/hour**.

So, the **weighted rate-based mean** is approximately **0.471 jobs/hour**.

Ratio-Based Means

Geometric Mean

$$geometric\ mean = \sqrt[n]{\prod_{i=1}^{n} ratio_i}$$

$$weighted\ geometric\ mean = \prod_{i=1}^{n} ratio^{w_i}$$

- Useful for **ratios**, like speedups (Computer A time / Computer B time).
- Example ratios: **1.95**, **1.96**, **2.08**, **2.38**.
- Multiply them: 1.95 × 1.96 × 2.08 × 2.38 ≈ **18.94**.
- Because there are 4 ratios, take the 4th root. That's about **2.08**.

Weighted Geometric Mean If each ratio has a **weight** (like a fraction of total work), you raise each ratio to its weight, then multiply everything together.

Weighted Geometric Mean Example

Scenario

- There are four loops: **loop1**, **loop2**, **loop3**, and **loop4**.
- **loop1** runs **20** times.
- **loop2** runs **30** times.
- **loop3** runs **50** times.
- **loop4** runs **100** times.
- Total number of loop executions = 20 + 30 + 50 + 100 = **200**.

Weights

- **Weight for loop 1** = 20 / 200 = **0.1**.
- **Weight for loop 2** = 30 / 200 = **0.15**.

- **Weight for loop 3** = 50 / 200 = **0.25**.
- **Weight for loop 4** = 100 / 200 = **0.5**.

Speedups

- **Speedup of loop1** = **1.95**.
- **Speedup of loop2** = **1.96**.
- **Speedup of loop3** = **2.08**.
- **Speedup of loop4** = **2.38**.

Weighted Geometric Mean Calculation

Formula:

- **Speedup of loop1)^(Weight of loop1) × (Speedup of loop2)^(Weight of loop2) × (Speedup of loop3)^(Weight of loop3) × (Speedup of loop4)^(Weight of loop4).**
- Substituting numbers: 1.95^0.1 × 1.96^0.15 × 2.08^0.25 × 2.38^0.5.
- Result = **2.19**.

Amdahl's Law

What It Means

- **Amdahl's Law** tells us how **overall performance** changes when we improve **one part** of a system.
- If the rest of the system is **not improved**, it becomes the **bottleneck**.

Basic Idea

- Imagine you have a **task** with two parts:
    1. **Part A** (not improved).
    2. **Part B** (can be made faster).
- Even if **Part B** is made **much faster**, the total time still depends on **Part A**.
- So, **overall speedup** is limited by the part that **does not change**.

Formula

- Let **a** = fraction of time that **can be sped up**.
- Let **n** = how many times faster that fraction becomes.
- The new execution time = **(1−a) + a/n**.
- **Speedup** = **1** / (new execution time) = $$\frac{1}{(1-a)+\frac{a}{n}}$$

Example 1: Serial and Parallel Portions

- 30s = time that cannot be sped up (serial part).
- 70s = time that can be sped up (parallel part).
- a = 70s = 0.7.
- n = 8 (speedup factor).

New Execution Time

- (1−0.7) + 0.7/8 = 0.3 + 0.0875 = 0.3875

Speedup

- 1 / 0.3875 ≈ 2.58.

Example 2: New CPU Speedup

- 40% = fraction of time doing computation (can be sped up).
- 60% = fraction of time doing I/O (cannot be sped up).
- a = 0.4.
- n = 10 (CPU is 10× faster for computation).

New Execution Time

- (1−0.4) + 0.4/10 = 0.6 + 0.04 = 0.64.

Speedup

- 1 / 0.64 ≈ 1.56.

Moore's Law

- **Key Idea**: The number of circuits (transistors) on a chip doubles roughly **every two years**.
- **Why It Matters**: Designers use **Moore's Law** so their designs won't be out of date when they finish.
- **More Devices per Die**: As we make transistors smaller, we fit **more** on the same chip area.

Die per Wafer

Suppose you have a **20 cm** wafer (circular) and each die is **1.5 cm × 1.5 cm**.

Formula :

$$\text{– Die/wafer} = \left[\frac{\pi(\text{wafer radius})^2}{\text{Die area}} - \frac{\pi(\text{wafer diameter})}{\sqrt{2x\ \text{Die area}}} \right]$$

Identify Wafer Radius and Die Area
Wafer diameter is 20 cm, so the wafer radius is 10 cm.
Die side is 1.5 cm, so the die area is 1.5 × 1.5 = 2.25 cm².

Approximate Wafer Area Contribution
The first part of the formula is:
(π × (wafer **radius**)^2) / (die area)
Substitute the values:
(3.14 × 10^2) / 2.25 ≈ 314 / 2.25 ≈ 139.6.

Subtract Edge Correction
The second part of the formula is:
(π × wafer **diameter**) / 2 × (die area)
Substitute the values:
(3.14 × 20) / 2 × 2.25 ≈ (62.8 / 2) × 2.25 ≈ 19.7.

Combine Results
Subtract the edge correction from the first part:
139.6 − 19.7 ≈ 119.9.
The final approximate answer is often rounded to 110 dies per wafer.

Grosch's Law

- **Key Idea**: To make computing "ten times cheaper," you must be "one hundred times faster."
- **Square Law**: **Computing power** (P) grows with the **square** of **cost** (C).
- **Example**: Spending **twice** the money gives you **four times** the computing power.

MFLOPS (Millions of Floating-Point Operations per Second)

- Example:
  A processor performs 10 million floating-point operations and 1 million overhead operations in 50 milliseconds. For MFLOPS, only floating-point operations are counted.
  10 million operations in 0.05 seconds:

$$\text{MFLOPS} = 10 \times 10^6 / 50 \times 10^{-3} \times 10^6 = 200$$

MFLOPS = (10,000,000 / 0.05) / 1,000,000 = 200 MFLOPS.

Lecture 9

What is Pipelining?

Pipelining makes a CPU work faster by handling **many instructions at the same time**. Instead of finishing one instruction before starting the next, the CPU **splits the work into steps** and processes different instructions at different steps **all at once**.

Example: Assembly Line

Think of a **car factory**:

- One worker **installs the engine**.
- Another worker **attaches the wheels**.
- A third worker **paints the car**.
- They all **work at the same time** on different cars.

This is **pipelining**—each instruction is like a car moving through different steps.

How Does It Work?

In a **MIPS 32-bit CPU**, an instruction goes through **five stages**:

1. **Fetch (IF)** – The CPU **gets** the instruction from memory.
2. **Decode (ID)** – The CPU **understands** what the instruction means.
3. **Execute (EX)** – The CPU **does** the actual work (like adding numbers).
4. **Memory (MEM)** – If needed, the CPU **reads/writes** data from memory.
5. **Write-back (WB)** – The CPU **stores** the result.

Each stage works on **one part of an instruction**, while other instructions **move forward**.

Why Is Pipelining Faster?

- A **normal CPU** (without pipelining) **finishes one instruction before starting the next**.
- A **pipelined CPU works on many instructions at once**—one at each stage.

- This **reduces the time per instruction** and **increases the number of instructions finished per second**.

Key Benefits of Pipelining

- **More instructions per second** → CPU **processes more tasks** at the same time.
- **Better performance** → The CPU does not **waste time waiting**.
- **Efficient resource use** → Each part of the CPU is **always working**.

Challenges in Pipelining

Pipelining **is not perfect**. Some problems can slow it down:

Hazards (Delays)

- **Data Hazard** → One instruction **depends on the result** of a previous one.
- **Control Hazard** → The CPU **does not know** what to do next (like in an "if" statement).
- **Structural Hazard** → Two instructions **need the same resource** at the same time.

Balancing the Stages

- If **one stage is too slow**, the whole pipeline slows down.
- The goal is to make **each stage take about the same time**.

Basic MIPS Instruction Set

MIPS is a **RISC (Reduced Instruction Set Computer) processor**. It follows **simple rules** to make execution fast.

Key Features of MIPS:

1. **All operations use registers.**
   - The CPU works with **data inside registers** (small storage areas inside the CPU).
   - Each register in MIPS **holds 32 bits**.
2. **Only two instructions affect memory:**
   - **Load (LW):** Takes data from memory and puts it in a register.
   - **Store (SW):** Takes data from a register and puts it in memory.
3. **Fixed instruction size:**
   - Every instruction is **exactly 4 bytes**.
   - This makes it **easy to design the CPU pipeline**.

Five Stages of the MIPS Pipeline

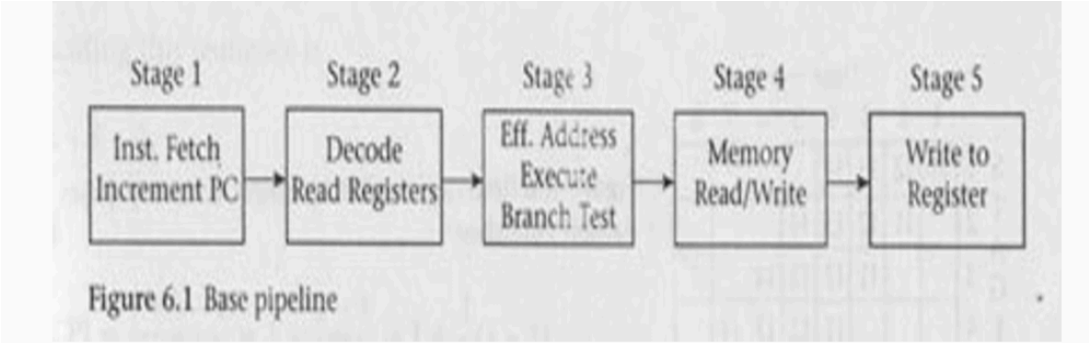
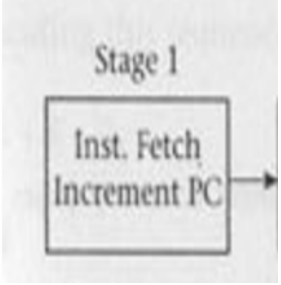


Figure 6.1 Base pipeline

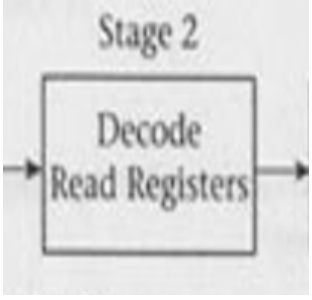
MIPS uses a **5-stage pipeline**. Each instruction **moves through these stages step by step**.

1. **IF (Instruction Fetch) Stage**



- The CPU **gets the instruction** from memory.
- The **Program Counter (PC)** stores the location of the current instruction.
- After fetching, the PC **moves to the next instruction** (adds 4 because each instruction is 4 bytes).

2. **ID (Instruction Decode) Stage**

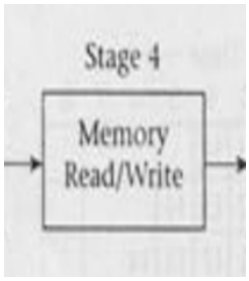


- The CPU **figures out what the instruction means**.
- It **reads the needed registers** from a storage area called the **register file**.
- If the instruction is a **branch (like an if-condition)**, the CPU **checks the condition** and **calculates the next address**.

3. **EX (Execution) Stage**

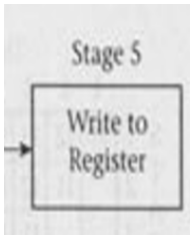
- The CPU **performs the actual operation** in this stage.
- There are three types of operations:
    1. **Memory operation** (Load/Store) → The CPU **calculates the memory address**.
    2. **Register operation** (Arithmetic, Logical) → The CPU **computes the result** using the ALU (math unit in CPU).
    3. **Immediate operation** (Using a fixed number) → The ALU **processes** the number directly.

4. MEM (Memory Access) Stage



- If the instruction is **Load (LW)** → The CPU **reads the data from memory**.
- If the instruction is **Store (SW)** → The CPU **writes data to memory**.

5. WB (Write Back) Stage



- This stage **saves the final result** in a register.
- If the instruction is **Load (LW)** → It **writes the data from memory into a register**.

- If it is an **ALU operation (like add, subtract, multiply)** → It **stores the result in a register**.

Why This Design is Simple and Fast?

- **Fixed instruction size (4 bytes)** → Easy to handle instructions.
- **Only Load and Store affect memory** → Reduces complexity.
- **Registers are read in parallel** while decoding → Makes execution faster.

How the Pipeline Works (Figure A.1)

Classic Five-stage Pipeline for RISC Processor

	Clock number								
Instruction number	1	2	3	4	5	6	7	8	9
Instruction *i*	IF	ID	EX	MEM	WB				
Instruction *i* + 1		IF	ID	EX	MEM	WB			
Instruction *i* + 2			IF	ID	EX	MEM	WB		
Instruction *i* + 3				IF	ID	EX	MEM	WB	
Instruction *i* + 4					IF	ID	EX	MEM	WB

Figure A.1 Simple RISC pipeline. On each clock cycle, another instruction is fetched and begins its 5-cycle execution. If an instruction is started every clock cycle, the performance will be up to five times that of a processor that is not pipelined. The names for the stages in the pipeline are the same as those used for the cycles in the unpipelined implementation: IF = instruction fetch, ID = instruction decode, EX = execution, MEM = memory access, and WB = write back.

- The diagram in **Figure A.1** shows time on the horizontal axis (clock cycles).
- Each instruction (i, i+1, i+2, etc.) uses one stage per clock cycle.
- For example, in clock cycle 1, instruction i is in **IF**. In clock cycle 2, instruction i moves to **ID**, while instruction i+1 enters **IF**.
- After 5 cycles, an instruction completes all stages, but a new instruction starts every cycle.

Why Separate the Stages?

- Each **functional unit** (like the **ALU** or the **memory**) is used in a different clock cycle by each instruction.

- This prevents **resource conflicts** (two instructions trying to use the same unit at the same time).
- The pipeline keeps hardware busy every cycle, so the CPU can execute one instruction after another efficiently.

Pipeline Diagram (Figure A.2)

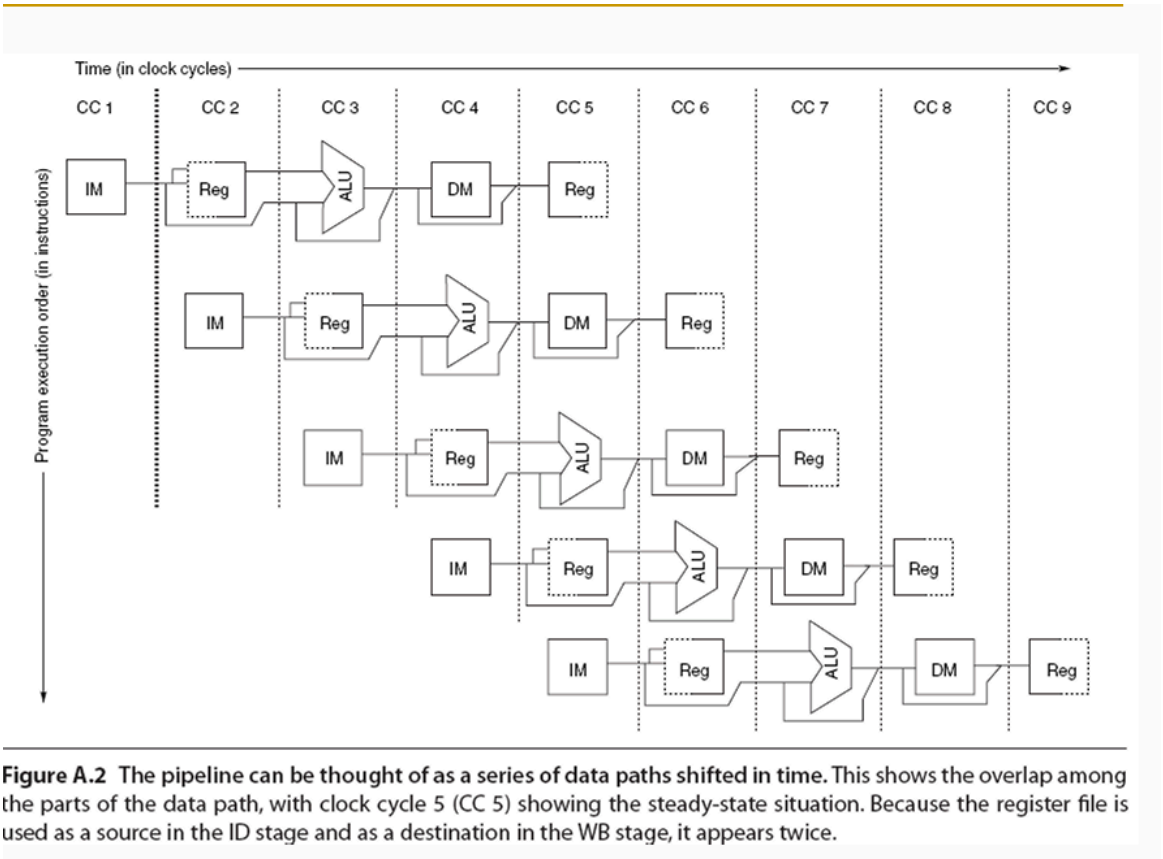


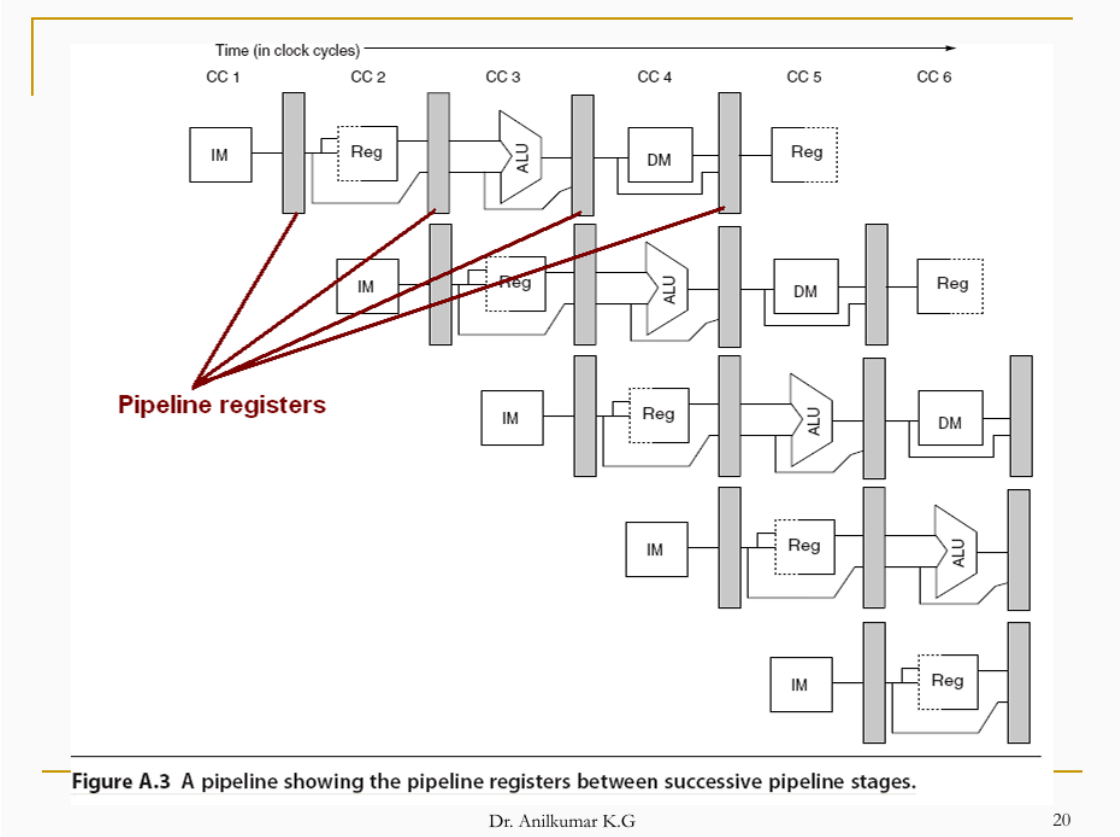
Figure A.2 The pipeline can be thought of as a series of data paths shifted in time. This shows the overlap among the parts of the data path, with clock cycle 5 (CC 5) showing the steady-state situation. Because the register file is used as a source in the ID stage and as a destination in the WB stage, it appears twice.

- **Figure A.2** shows how each **functional block** is used across the clock cycles:
  - **IM** (Instruction Memory) fetches the instruction.
  - **Reg** (Register File) is read to get source registers.
  - **ALU** does arithmetic or address calculations.
  - **DM** (Data Memory) reads or writes data.
  - **Reg** is used again to write back results.
- Notice how each block is used in a specific cycle so that multiple instructions do not clash.

Handling Resource Conflicts

1. **Separate Instruction and Data Memory**
    - Using **two caches** (one for instructions, one for data) avoids memory conflicts.
2. **Register File Access**
    - In the **ID** stage, the CPU reads registers.
    - In the **WB** stage, the CPU writes to a register.
    - To allow a read and a write in the same clock, the register file writes in the **first half** of the cycle and reads in the **second half**.
3. **Program Counter (PC) Updates**
    - The PC (which points to the next instruction) is updated in the **IF** stage every cycle.
    - Another adder in the **ID** stage calculates a branch target if needed.

Pipeline Registers (Figure A.3)



- At the **end of each stage**, results are stored in **pipeline registers** (also called buffers).
- These registers pass the outputs of one stage to the inputs of the next stage in the next clock cycle.

Putting It All Together

- **Each instruction** goes through **IF → ID → EX → MEM → WB** in order.
- **Pipeline registers** keep stage outputs safe so that the next stage can read them in the next clock.
- **Multiple instructions** flow like an assembly line, starting one per clock cycle.

Pipelining and Throughput

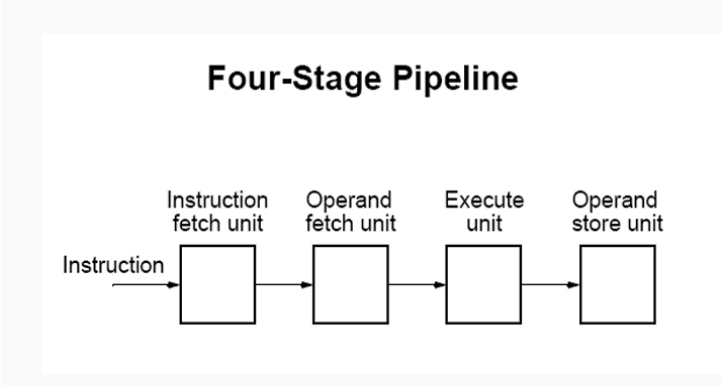
- **Pipelining** lets multiple instructions overlap in time.
- This increases the **throughput** (instructions finished per unit time).
- However, **individual instruction time** can actually get longer because of extra pipeline control steps.

Why Each Instruction Might Slow Down

- Adding pipeline **registers** between stages causes **overhead** (extra delay).
- **Clock skew** (the difference in clock arrival times) also adds to this overhead.
- If one stage is **slower** than the others, the clock must wait for it, reducing overall speed.

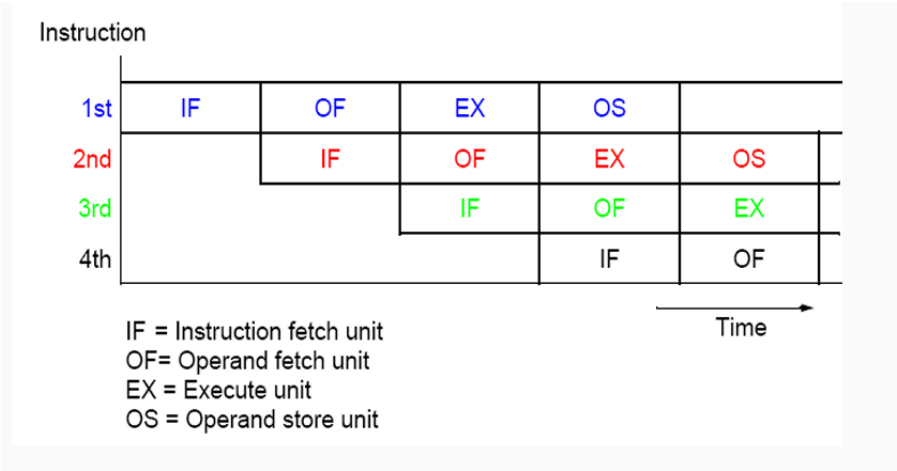
Four-Stage Pipeline

The Four Stages



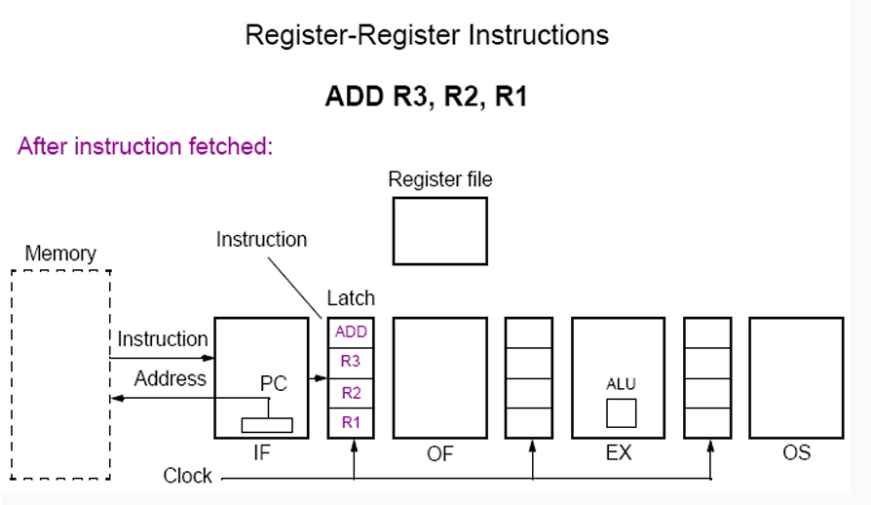
1. **IF (Instruction Fetch):** Get the instruction from memory using the **PC** (Program Counter).
2. **OF (Operand Fetch):** Collect the source data (like registers or immediate values) needed by the instruction.
3. **EX (Execute):** Perform the required operation (for example, add two numbers).
4. **OS (Operand Store):** Write the result back to a register or memory location.

Instruction-Time Diagram



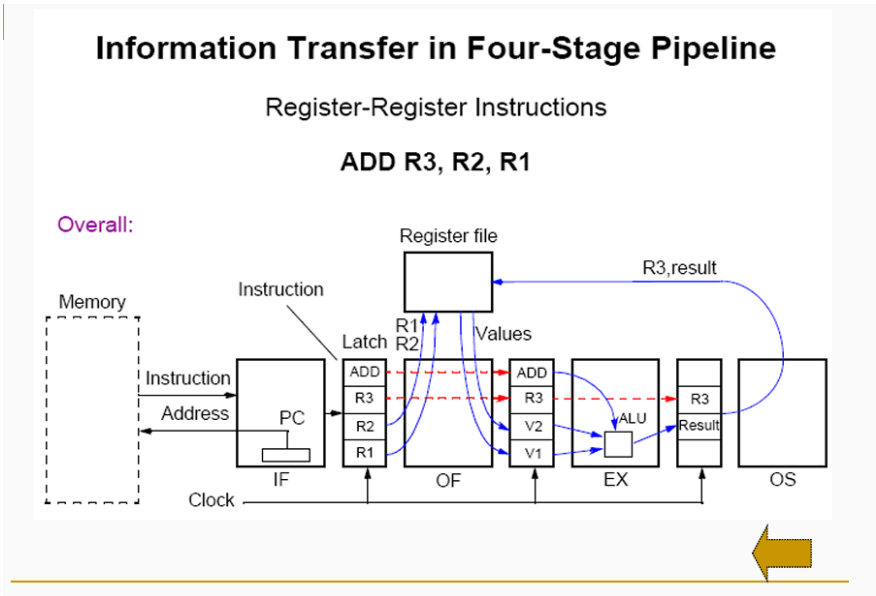
- The **1st instruction** goes to **IF** first, then **OF**, then **EX**, then **OS**.
- When the 1st instruction moves to **OF**, the **2nd instruction** starts its **IF** stage, and so on.
- This creates a “pipeline” where multiple instructions flow in parallel.

Pipeline Registers



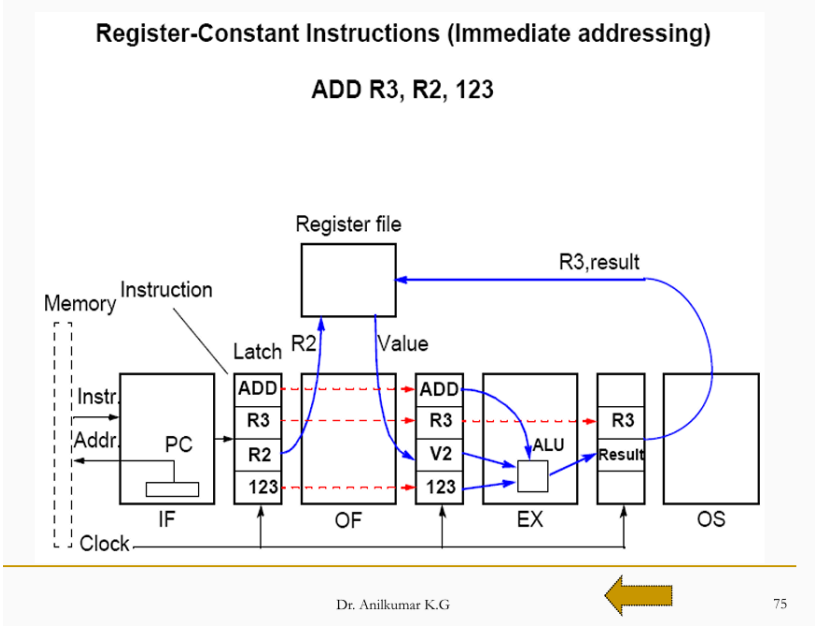
- Between each stage, there is a **pipeline register** (sometimes called a latch).
- These registers hold any important data (like operands or the ALU result) so the next stage can use it in the next clock cycle.
- This prevents the stages from interfering with each other.

Register-Register Example



- Take **ADD R3, R2, R1** as an example.
  - **IF:** The CPU fetches the **ADD** instruction (with register info R3, R2, R1).
  - **OF:** The CPU reads the values in **R2** and **R1** (call them **V2** and **V1**).
  - **EX:** The CPU adds **V2** and **V1**, producing a **result**.
  - **OS:** The CPU writes this **result** into **R3**.

Immediate Operands



- If the instruction has an **immediate** (like **ADD R3, R2, 123**), the **123** is also fetched in the **OF** stage.
- It goes to **EX** to be added to **R2**.

- Finally, the sum is written back in the **OS** stage.

Speedup from Pipelining

Consider the unpipelined processor in the previous section. Assume that it has a 1 ns clock cycle and that it uses 4 cycles for ALU operations and branches and 5 cycles for memory operations. Assume that the relative frequencies of these operations are 40%, 20%, and 40%, respectively. Suppose that due to clock skew and setup, pipelining the processor adds 0.2 ns of overhead to the clock. Ignoring any latency impact, how much speedup in the instruction execution rate will we gain from a pipeline?

Unpipelined Processor: Average Instruction Execution Time

- **Clock cycle:** 1 ns
- **Operation frequencies and CPI:**
  - ALU: 40%, **4 cycles**
  - Branches: 20%, **4 cycles**
  - Memory: 40%, **5 cycles**

Weighted CPI Calculation:
(0.40 × 4) + (0.20 × 4) + (0.40 × 5) = **4.4**

Average Instruction Time:
1 ns × 4.4 = **4.4 ns**

Pipelined Processor: New Clock Cycle Time

- **Base cycle:** 1 ns
- **Pipeline overhead:** +0.2 ns
- **New cycle time: 1.2 ns**

In an ideal pipeline, **CPI ≈ 1**, so:
Average Instruction Time = 1.2 ns

Speedup from Pipelining

Speedup = Execution time (unpipelined) ÷ Execution time (pipelined)
Speedup = 4.4 ns ÷ 1.2 ns ≈ **3.7**

The pipelined processor is **3.7× faster** than the unpipelined one.

What Are Pipeline Hazards?

- In a **pipelined CPU**, multiple instructions overlap.
- A **hazard** is a situation that prevents an instruction from running in its normal pipeline stage.
- Hazards force the CPU to **stall** (delay) some instructions.

Types of Pipeline Hazards

1. **Structural Hazards:**
    - It Happens when a hardware resource (like memory or register file) is **overused**.
    - For example, if two instructions need the **same resource** at the same time and there is only **one** of that resource.
2. **Data Hazards:**
    - Occur when one instruction needs a **result** from a previous instruction that hasn't finished yet.
3. **Control Hazards:**
    - Happen when the CPU **changes the PC** (Program Counter), such as with **branches**, and the pipeline is already fetching the next instruction.

Structural Hazards

- A **structural hazard** means the CPU can't pipeline properly because one stage is blocked by another.
- Common example: a processor has **only one memory port** for both instructions and data.
- If one instruction is **loading** from memory (needs data access), and another instruction wants to **fetch** at the same time, they **conflict**.

Figure A.4

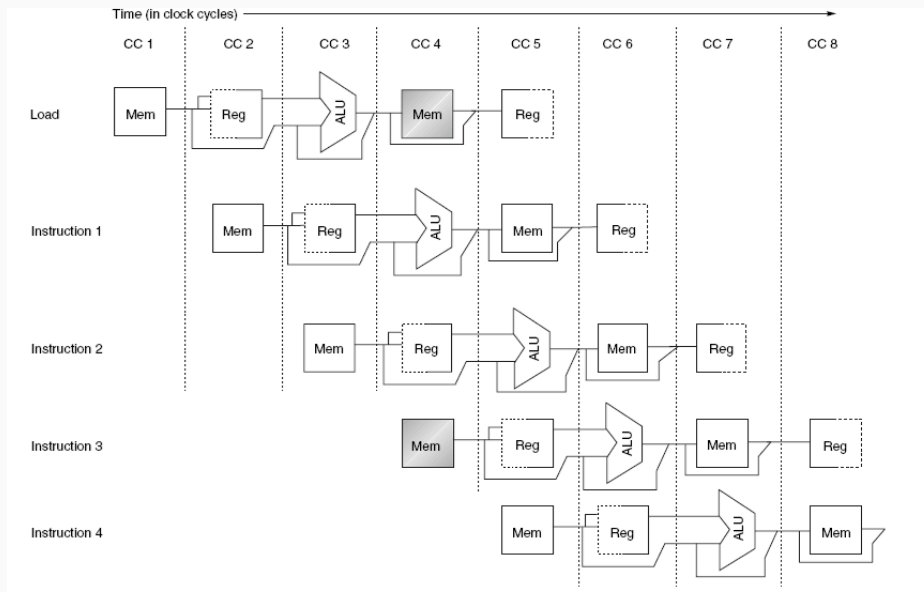


Figure A.4 A processor with only one memory port will generate a conflict whenever a memory reference occurs. In this example the load instruction uses the memory for a data access at the same time instruction 3 wants to fetch an instruction from memory.

- Shows a processor that has just **one memory port**.
- **Instruction 1** (a load) is using **memory** for data in one cycle.
- **Instruction 3** wants to **fetch** from memory in the same cycle.
- Because there's only **one port**, they **cannot** happen together, causing a **stall**.

Stalling the Pipeline (Figure A.5)

	Clock cycle number									
Instruction	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i + 1$		IF	ID	EX	MEM	WB				
Instruction $i + 2$			IF	ID	EX	MEM	WB			
Instruction $i + 3$				stall	IF	ID	EX	MEM	WB	
Instruction $i + 4$						IF	ID	EX	MEM	WB
Instruction $i + 5$							IF	ID	EX	MEM
Instruction $i + 6$								IF	ID	EX

Figure A.5 A pipeline stalled for a structural hazard—a load with one memory port. As shown here, the load instruction effectively steals an instruction-fetch cycle, causing the pipeline to stall—no instruction is initiated on clock cycle 4 (which normally would initiate instruction $i + 3$). Because the instruction being fetched is stalled, all other instructions in the pipeline before the stalled instruction can proceed normally. The stall cycle will continue to pass through the pipeline, so that no instruction completes on clock cycle 8.

- When a structural hazard occurs, the CPU **inserts a stall** (sometimes called a **bubble**).
- In **Figure A.5**, the pipeline stalls for one cycle so the load instruction can use memory.

- During that stall cycle, **no new instruction** moves forward.
- After the stall, instructions **resume** their normal flow, shifted to the right (delayed).

Impact on Performance

- Each stall **increases** the **average CPI** (cycles per instruction).
- **Fewer stalls → better performance**.
- If a design **duplicates** or **splits** resources (like using **separate instruction and data caches**), structural hazards can be avoided or reduced.

What Are Data Hazards?

- **Data hazards** happen when one instruction **depends** on the result of a previous instruction.
- In a pipeline, the **order** of reading and writing registers can get **mixed** because instructions **overlap**.

Example of a Data Hazard

- Instructions like:
    1. **DADD R1, R2, R3**
    2. **DSUB R4, R1, R5**
    3. **AND R6, R1, R7**
    4. **OR R8, R1, R9**
    5. **XOR R10, R1, R11**
- The **DADD** writes its result into **R1** at the **WB** stage (later in time).
- The **DSUB** tries to **read R1** earlier (during **ID**).
- Because the pipeline overlaps, the DSUB might see the **old** value in **R1**, not the new one.

How This Hazard Happens (Figure A.6)

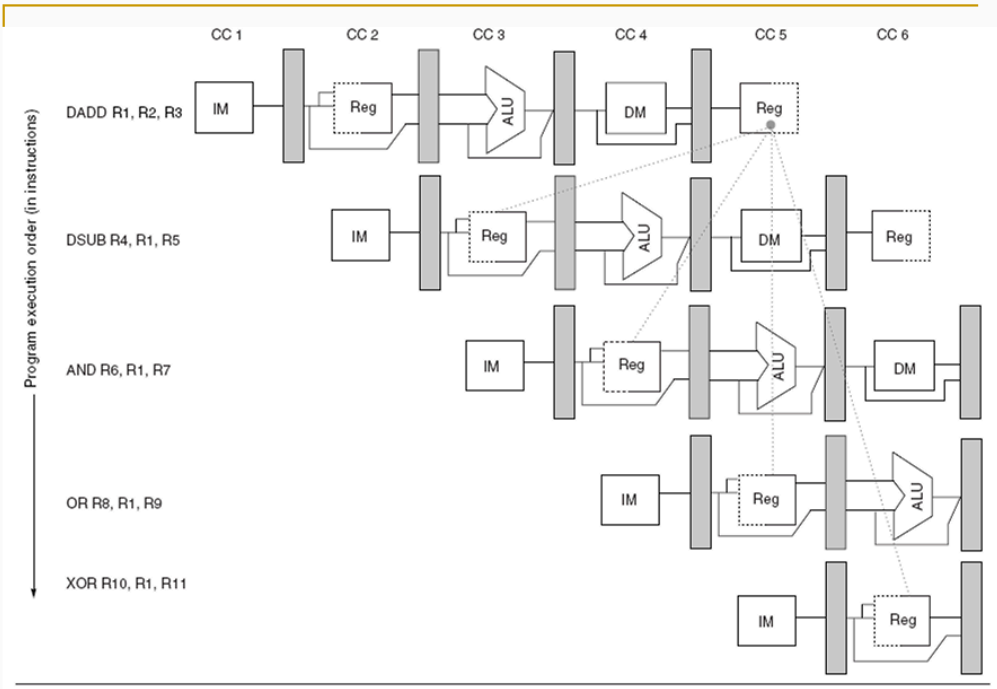


Figure A.6 The use of the result of the DADD instruction in the next three instructions causes a hazard, since the register is not written until after those instructions read it.

- Each instruction moves through stages (IF, ID, EX, MEM, WB).
- **DADD** writes to **R1** at the end of **clock cycle 5**.
- **DSUB** needs to read **R1** at the **start of clock cycle 3** (its ID stage).
- That's **too soon**, so it reads the **wrong** (old) value.

. Impact on Other Instructions

- **AND R6, R1, R7 also suffers** if it reads **R1** before the new value is written.
- **XOR R10, R1, R11 avoids** the hazard because it reads **R1** later (in a later clock cycle).
- **OR R8, R1, R9 might avoid** the hazard if the CPU's register file writes in the **first half** of the clock and reads in the **second half** (so it gets the updated value in time).

Minimizing Data Hazard Stalls by Forwarding

What Is Forwarding (Bypassing)?

- **Forwarding** is a hardware technique to **send** a result **directly** from where it is **produced** to where it is **needed**—without waiting for it to be written to a register first.
- For example, if **DADD** finishes computing a result in the **EX** stage, the CPU can **forward** it straight to the **EX** stage of **DSUB** in the next cycle.

How Forwarding Works

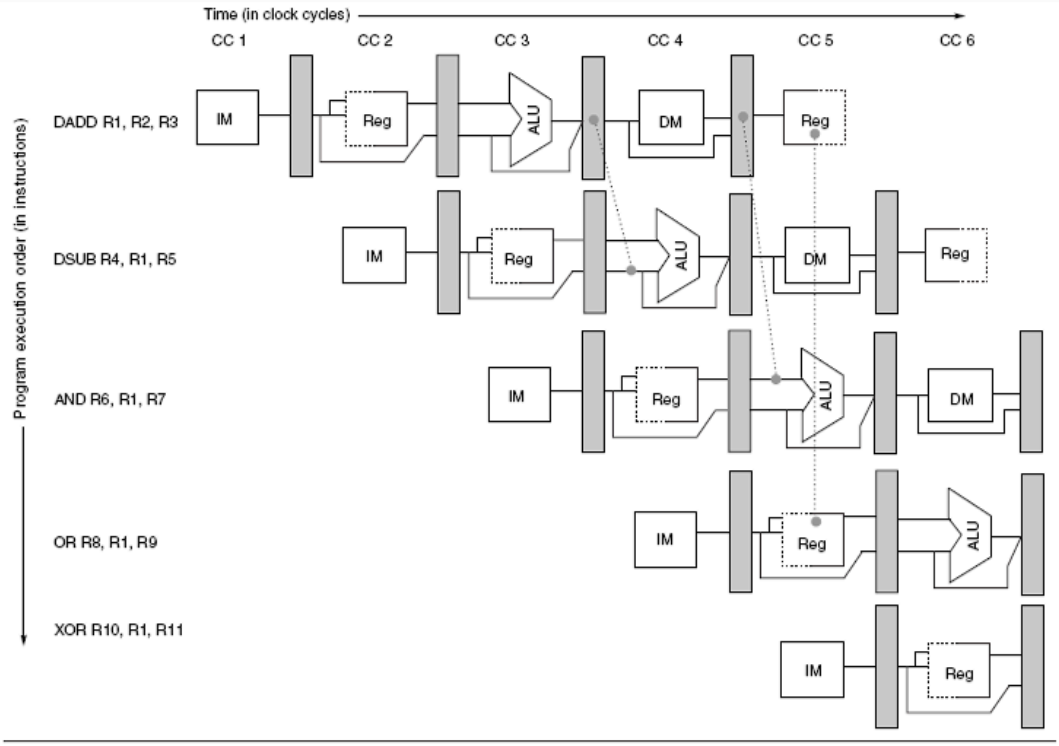


Figure A.7 A set of instructions that depends on the DADD result uses forwarding paths to avoid the data hazard. The inputs for the DSUB and AND instructions forward from the pipeline registers to the first ALU input. The OR receives its result by forwarding through the register file, which is easily accomplished by reading the registers in the second half of the cycle and writing in the first half, as the dashed lines on the registers indicate. Notice that the forwarded result can go to either ALU input; in fact, both ALU inputs could use forwarded inputs from either the same pipeline register or from different pipeline registers. This would occur, for example, if the AND instruction was AND R6,R1,R4.

- The **ALU result** is fed back to the **ALU input** in the next cycle.
- If the hardware detects that an instruction needs a **just-computed** result, it picks that forwarded value **instead of** the old value in the register file.
- **Figure A.7** shows these **bypass paths** that skip writing and reading from the register file.

Forwarding More Than One Cycle Later

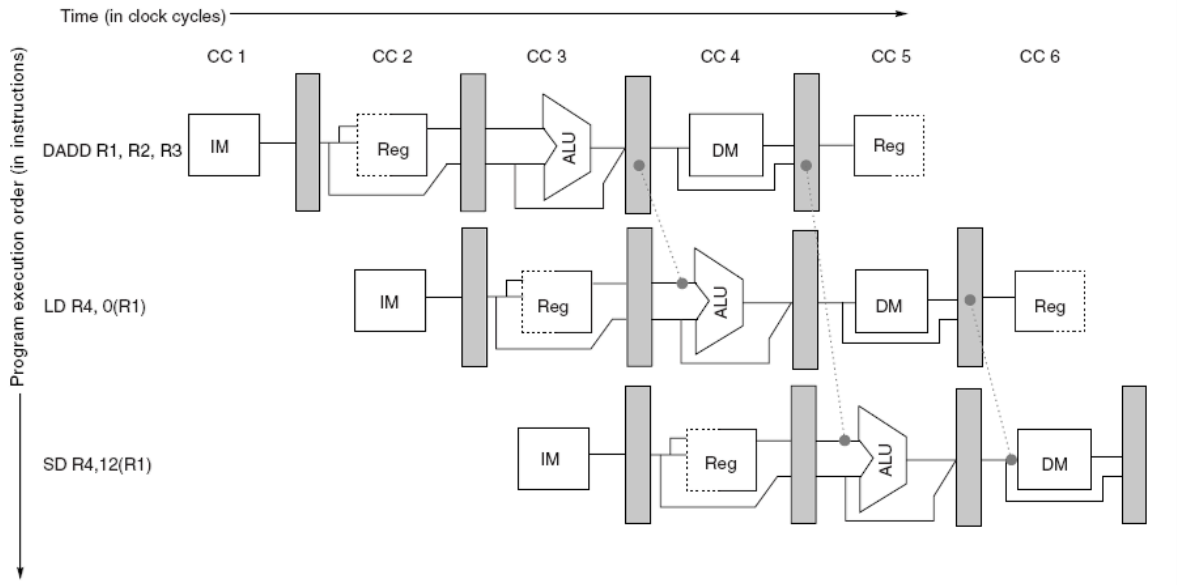


Figure A.8 Forwarding of operand required by stores during MEM. The result of the load is forwarded from the memory output to the memory input to be stored. In addition, the ALU output is forwarded to the ALU input for the address calculation of both the load and the store (this is no different than forwarding to another ALU operation). If the store depended on an immediately preceding ALU operation (not shown above), the result would need to be forwarded to prevent a stall.

- Sometimes you need to **forward** results from an instruction that started **two cycles** before the current instruction.
- The CPU sets up **multiple** bypass paths so it can handle different timing overlaps.

Forwarding for Loads and Stores

Example with LW and SW

- **DADD R1, R2, R3**
- **LW R4, 0(R1)** (Load word from memory using R1's address)
- **SW R4, 12(R1)** (Store word into memory at R1 + 12)
- Here, the **LW** and **SW** need the address from **R1**.
- Also, the **SW** might need the data from **R4**, which was just loaded.
- **Forwarding** sends the **memory output** (the loaded data) directly to the next stage that needs it.

Figure A.9

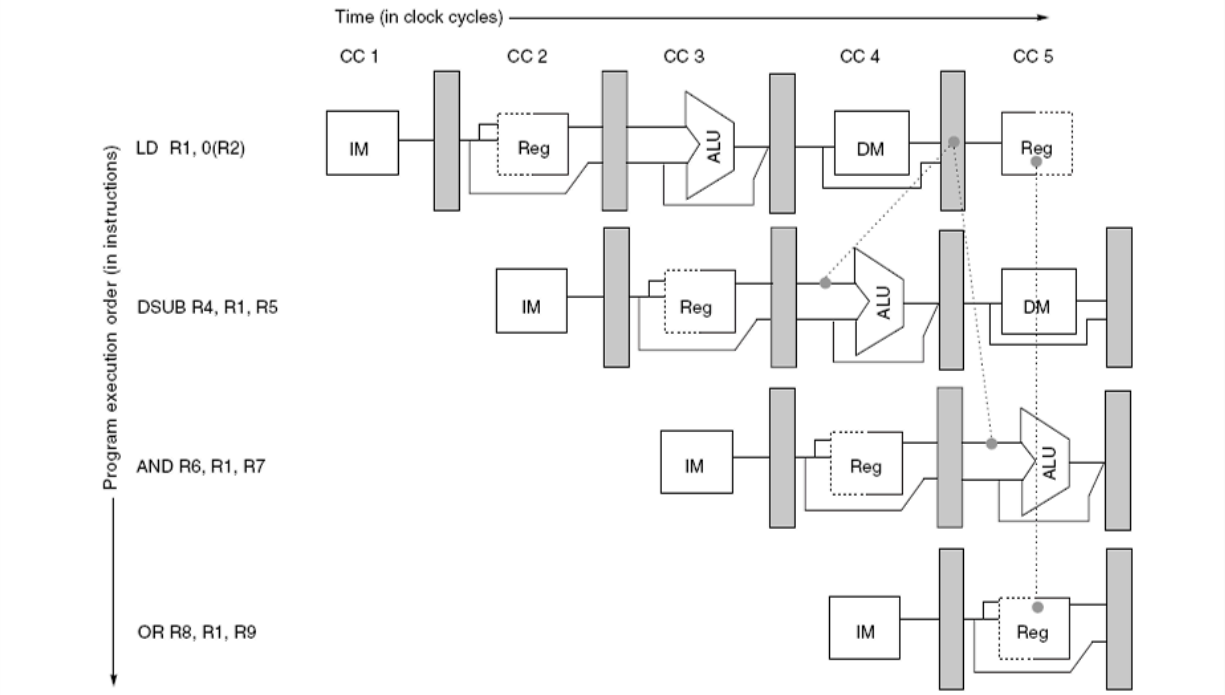


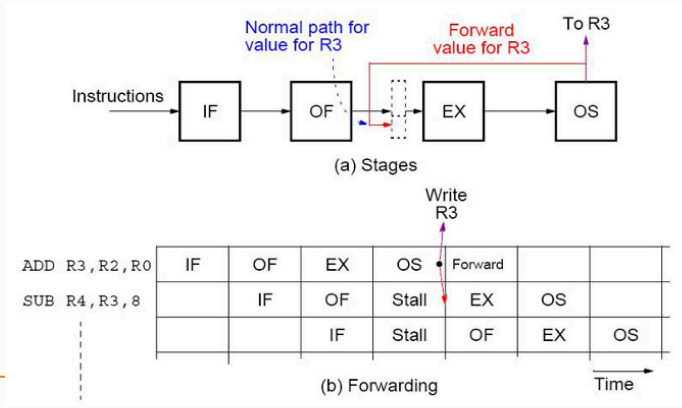
Figure A.9 The load instruction can bypass its results to the AND and OR instructions, but not to the DSUB, since that would mean forwarding the result in "negative time."

- Shows how the **loaded data** from **MEM** is forwarded to the **EX** stage if needed (for an address calculation or another ALU operation).
- This **avoids** extra stalls by providing the data as soon as it's available.

Another Example of 4 stages Pipeline

ADD R3, R2, R0
SUB R4, R3, 8

SUB instruction requires R3, which
Is by the ADD instruction (RAW-hazard)



Dr. Anilkumar K.G 87

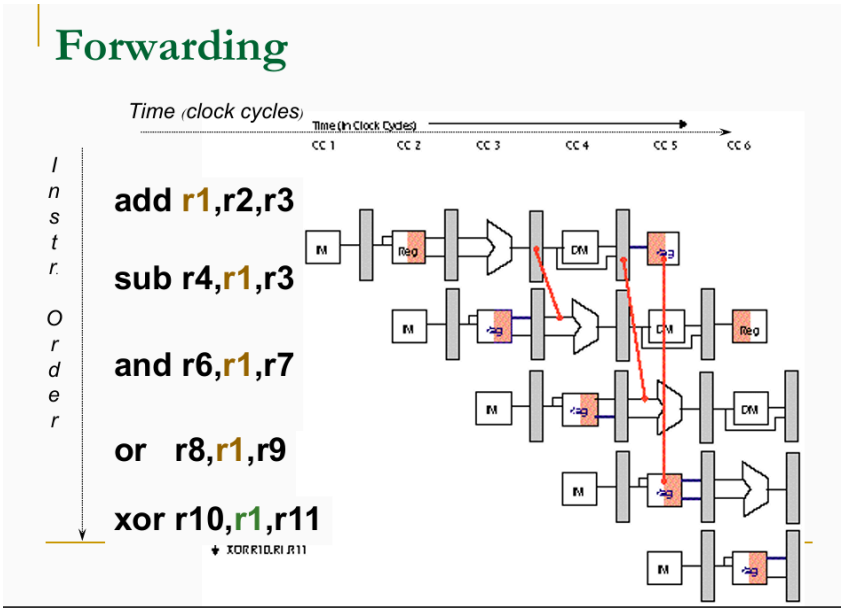
Why Forwarding Is Needed

- **ADD R3, R2, R0** produces a new value for **R3**.
- **SUB R4, R3, 8** immediately needs **R3** as an input.
- If the **SUB** reads **R3** from the register file, it might get the **old** value, not the new one.
- **Forwarding** fixes this by **sending** the new value of **R3** directly from the pipeline stage that computes it to the pipeline stage that needs it.

How Forwarding Works

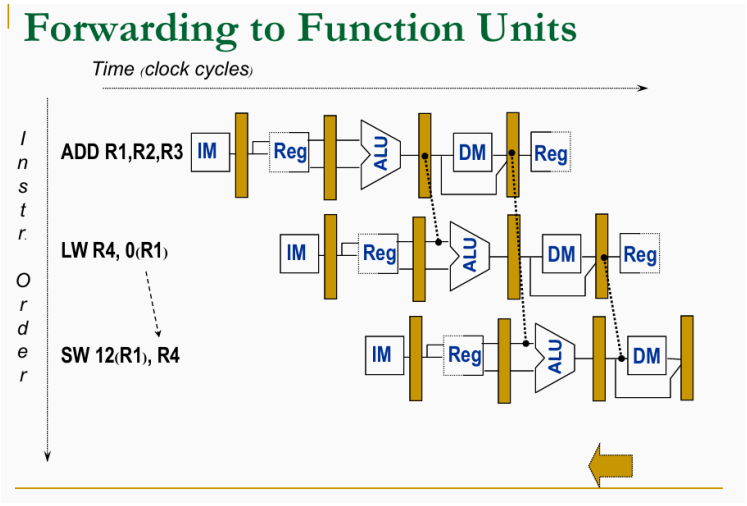
- The **ADD** instruction calculates **R3** in its **EX** stage.
- In the **next** cycle, **SUB** also reaches its **EX** stage.
- The hardware **forwards** the **ADD** result from the **EX** or **MEM** pipeline register to the **SUB**'s **EX** input.
- This way, **SUB** sees the **fresh** value of **R3** without waiting for the register file to be updated.

Multiple Instructions



- In sequences like:
  - `add r1, r2, r3`
  - `sub r4, r1, r3`
  - `and r6, r1, r7`
  - `or r8, r1, r9`
  - `xor r10, r1, r11`
- Each subsequent instruction may need **r1**.
- Forwarding lines (highlighted in diagrams) show where the **CPU** provides the **fresh** data to avoid a stall.

Forwarding to Function Units



- The CPU can forward results to different places:

    - **ALU** input (for arithmetic instructions).
    - **Memory** input (for store instructions).
    - **Register file** write port (normal path).
- By providing these **bypass paths**, the pipeline keeps moving without unnecessary delays.

The Problem with Load Instructions

- When an instruction **loads** data from memory (like `LW R1, 0(R2)`), the **result** is only ready at the **end** of the **MEM** stage.
- Another instruction that needs **R1** (for example, `DSUB R4, R1, R5`) might need that value **earlier** (at the start of its **EX** stage).
- If the **EX** stage happens **before** the load finishes **MEM**, the CPU cannot just forward the data.

Why Forwarding Alone Fails

- **Forwarding** takes a value from a **later** pipeline stage and feeds it **directly** to an **earlier** stage in the **next** instruction.
- But in this **LW → DSUB** scenario, the load's value isn't ready until the **end** of clock cycle 4. The DSUB wants it **at the beginning** of clock cycle 4.
- You would need to "forward data **backwards in time**," which is impossible.

Example Sequence

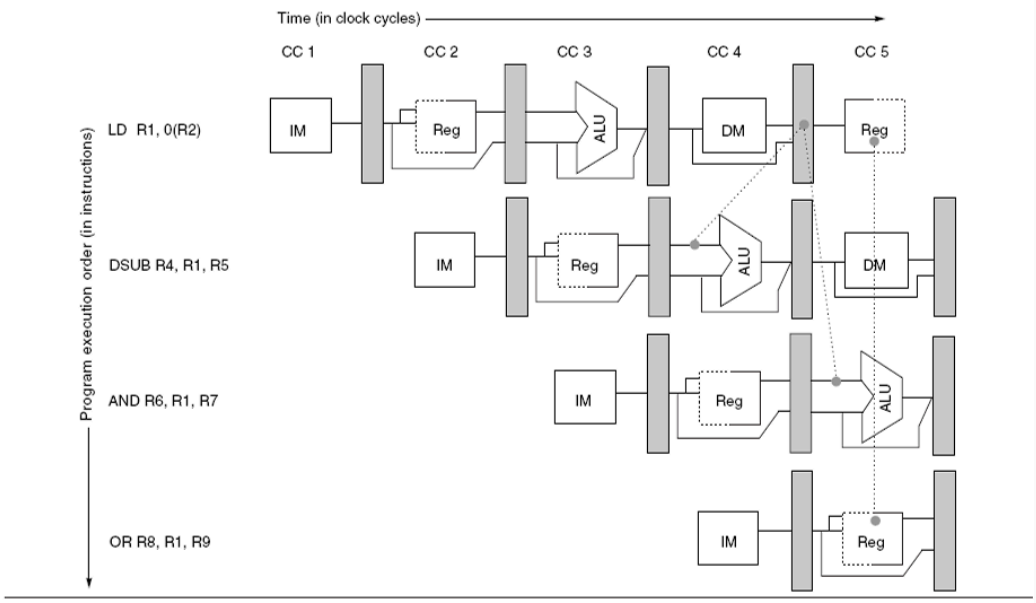


Figure A.9 The load instruction can bypass its results to the AND and OR instructions, but not to the DSUB, since that would mean forwarding the result in "negative time."

1. `LW R1, 0(R2)`
2. `DSUB R4, R1, R5`
3. `AND R6, R1, R7`
4. `OR R8, R1, R9`

- **AND** and **OR** can get the data in time because they start **two** or **three** cycles after the load.
- **DSUB** starts **one** cycle after the load, so it can't get the new data in time without a **stall**.

Result: Pipeline Stall

- The CPU **inserts** a stall (bubble) to wait until the **LW** finishes the MEM stage.
- After the stall, **DSUB** sees the correct value in **R1**.
